



**Politechnika
Śląska**

Dokumentacja projektowa

Zarządzanie Systemami Informatycznymi

Race Engineer Ecosystem and Cloud Usage in AI Modelling

Kierunek: Informatyka

Członkowie zespołu:

Marcel Bieńko

Michał Kierzkowski

Gliwice, 2024/2025

Spis treści

1	Wprowadzenie	4
1.1	Założenia	4
1.2	Technologie	6
1.3	Podział zadań	6
2	Implementacja aplikacji desktopowej	7
2.1	Struktura projektu	7
2.1.1	EngineerDashboard.App	7
2.1.2	EngineerDashboard.Database	8
2.1.3	EngineerDashboard.Telemetry	8
2.2	EngineerDashboard.Telemetry	9
2.2.1	Klient	9
2.2.2	Pakiety	13
2.3	EngineerDashboard.Database	18
2.4	EngineerDashboard.App	25
2.4.1	Services	29
2.4.1.1	TelemetryProvider	29
2.4.1.2	DatabaseService	30
2.4.2	Helpers	35
2.4.2.1	FontHelper	35
2.4.2.2	ColorHelper	36
2.4.2.3	Formatter	37
2.4.3	Converters	38
2.4.4	MainWindow	40
2.4.5	Views & ViewModels	44
2.4.6	Services	53
2.4.6.1	TelemetryProvider	53
2.4.6.2	TelemetryLogger	55
2.4.6.3	DatabaseService	61
2.4.7	Helpers	65
2.4.7.1	FontHelper	65
2.4.7.2	ColorHelper	66
2.4.7.3	Formatter	67
2.4.8	Converters	68
2.4.9	MainWindow	70
2.4.10	Views & ViewModels	74
3	Instrukcja obsługi aplikacji desktopowej	83
3.1	Pasek statusu sesji	83
3.2	Tabela kierowców	84

3.3	Informacje o samochodzie	85
3.4	Wykresy	86
3.5	Ranking kierowców	87
4	Podsumowanie aplikacji desktopowej	89
4.1	Potencjał rozwoju	89
4.2	Wnioski	89
5	Implementacja bazy danych	90
5.1	Struktura projektu	90
5.1.1	EngineerDashboard.Database	90
5.2	Schemat związków encji	91
5.3	Normalizacja	94
5.3.1	Pierwsza postać normalna	95
5.3.2	Druga postać normalna	95
5.3.3	Trzecia postać normalna	96
5.3.4	Postać normalna Boyce'a-Codda	97
5.3.5	Pozostałe postaci normalne	97
5.4	EngineerDashboard.Database	98
5.5	EngineerDashboard.App	111
5.5.1	Services	112
5.5.1.1	TelemetryProvider	112
5.5.1.2	DatabaseService	113
6	Podsumowanie bazy danych	118
6.1	Potencjał rozwoju	118
6.2	Wnioski	118
7	Implementacja modelu sztucznej inteligencji	119
7.1	Zbieranie danych	119
7.2	Przygotowanie danych	121
7.3	Wybór modelu	122
7.4	Model matematyczny	123
7.5	Klasa modelu	124
8	Testy modelu sztucznej inteligencji	125
8.1	Opis testów	125
8.2	Wyniki testów	142
8.2.1	Nienormalizowane dane, nieoptymalizowane parametry	142
8.2.2	Nienormalizowane dane, zoptymalizowane parametry	142
8.2.3	Znormalizowane dane, nieoptymalizowane parametry	143

8.2.4	Znormalizowane dane, zoptymalizowane parametry	143
8.3	Podsumowanie	144
9	Podsumowanie modelu sztucznej inteligencji	149
9.1	Potencjał rozwoju	149
9.2	Wnioski	149
10	Zarządzanie projektem	150
10.1	Analiza projektu	150
10.2	Decyzje	151
10.3	Model pracy	151
10.4	Sprint 1 - przygotowania	152
10.5	Sprint 2 - telemetria i baza danych	154
10.6	Sprint 3 - interfejs i model	157
10.7	Sprint 4 - integracja bazy, dokumentacja i zbieranie danych	160
10.8	Sprint 5 - doskonalenie modelu, dokumentacja i ostatnie szlify	165
10.9	Sprint 6 - zakończenie projektu	168
11	Podsumowanie projektu	175
12	Bibliografia	176

1 Wprowadzenie

Celem projektu jest stworzenie ekosystemu inżyniera wyścigowego obejmującego aplikację desktopową, pozwalającą monitorować dane telemetryczne w czasie rzeczywistym z gry *F1 23*; bazę danych, pozwalającą przechowywać historię wyścigów kierowców i ich ranking ELO; oraz model sztucznej inteligencji, przewidujący pozycję kierowcy na mecie wyścigu i umieszczenie go w chmurze.

Aplikacja stworzona w ramach *Programowania Obiektowego i Graficznego* bezpośrednio łączy się z bazą danych stworzoną na potrzeby *Baz Danych*. Pozwala również na zbieranie danych w celu szkolenia i testowania *Bayesowskiej Regresji Liniowej* na *Systemy Sztucznej Inteligencji*. Wszystkie te projekty spięte są w jeden wspólny dzięki *Zarządzaniu Systemami Operacyjnymi*, dzięki któremu wykorzystujemy ważne narzędzia zarządzania projektem.

1.1 Założenia

Aplikacja desktopowa obejmuje:

- pasek statusu (nazwa sesji i toru, aktualne okrążenie/ilość okrążeń, status toru, pogoda i temperatury),
- główna zakładka z tabelą kierowców (zawierająca pozycję, nazwisko, zespół, delta czasowa do kierowcy z przodu, delta czasowa do lidera wyścigu, rodzaj opon, zużycie opon, status systemu DRS, czas ostatniego okrążenia, czas sektorów),
- zakładka z danymi o bolidzie (stan opon, opony dostępne w pitstopie, uszkodzenia),
- zakładka z wykresami czasów okrążeń, zużycia opon, zużycia paliwa, zużycia baterii,
- zakładka z wykresami aplikacji gazu, aplikacji hamulca, prędkości, biegów i systemu DRS na przestrzeni okrążenia,
- zakładka z rankingiem kierowców pobranym z bazy; możliwość przejrzania statystyk, historii wyścigów i historii okrążeń każdego kierowcy.
- loggowanie części danych do bazy w celu utworzenia rankingu kierowców.
- loggowanie części danych do pliku w celu szkolenia i testowania modelu.

Baza danych ma za zadanie przechowywać:

- listę kierowców,
- listę wyścigów,
- listę kierowców biorących udział w konkretnym wyścigu,
- wyniki wyścigów
- informacje o okrążeniach kierowców,
- listę pit stopów kierowców,
- zespoły kierowców,
- rangi kierowców.

Skrypt stworzony w trakcie projektu ma za zadanie:

- złączyć wszystkie pliki *.csv* w jeden wspólny zestaw danych (telemetria każdego wyścigu znajduje się w oddzielnym pliku),
- zapewnić prawidłowość danych,
- znormalizować dane,
- podzielić dane na zestaw treningowy i testowy,
- wyszkolić odpowiedni model,
- przetestować dokładność modelu przy wielu przebiegach losowego podziału danych.

1.2 Technologie

Do stworzenia projektu wykorzystano:

- język C# z frameworkiem WPF[1]
- .NET w wersji 9.0
- bibliotekę interfejsu użytkownika WPF-UI[2]
- bibliotekę do tworzenia wykresów LiveCharts2[3]
- PostgreSQL[4] jako system bazodanowy,
- Docker[5] do self-hostingu bazy danych,
- Notebook Jupyter[6] w języku Python,
- bibliotekę numpy[7],
- bibliotekę pandas[8],
- bibliotekę optuna[9],
- bibliotekę scikit-learn[10],
- bibliotekę matplotlib[11],
- repozytorium GitHub[12] do zarządzania kodem,
- Jirę[13] do zarządzania projektem.

1.3 Podział zadań

Zadania dla tego projektu zostały rozłożone następująco:

Marcel Bieńko:

- zarządzanie projektem,
- stworzenie klienta telemetry,
- stworzenie backendu aplikacji desktopowej,
- stworzenie interfejsu użytkownika,
- zbieranie danych telemetrycznych,
- wybór modelu sztucznej inteligencji,
- stworzenie testów modelu,
- przygotowanie dokumentacji.

Michał Kierzkowski:

- stworzenie bazy danych,
- stworzenie wzoru obliczania ELO,
- stworzenie zapytań, funkcji, wyzwalaczy w bazie,
- implementacja modelu sztucznej inteligencji,
- analiza wyników testowania modelu,
- przygotowanie prezentacji,
- przygotowanie dokumentacji.

2 Implementacja aplikacji desktopowej

2.1 Struktura projektu

Implementacja została podzielona na trzy projekty znajdujące się w jednym rozwiązaniu:

```
.
├─ EngineerDashboard.App
├─ EngineerDashboard.Database
├─ EngineerDashboard.Telemetry
└─ EngineerDashboard.sln
```

EngineerDashboard.App zawiera pełną implementację aplikacji desktopowej.

EngineerDashboard.Database zawiera pliki konfiguracyjne bazy danych oraz implementację modeli, usług i kontekstu bazodanowych.

EngineerDashboard.Telemetry zawiera implementację klienta telemetrii.

2.1.1 EngineerDashboard.App

```
EngineerDashboard.App
├─ App.xaml
├─ App.xaml.cs
├─ Assets
├─ Converters
├─ Helpers
├─ MainWindow.xaml
├─ MainWindow.xaml.cs
├─ Services
├─ ViewModels
└─ Views
```

App.xaml, *App.xaml.cs*, *MainWindow.xaml* i *MainWindow.xaml.cs* - pliki wejściowe projektu.

Assets - folder zawierający czcionki i ikony.

Converters - folder zawierający konwertery WPF.

Helpers - folder zawierający klasy pomocnicze.

Services - folder zawierające usługi telemetrii, loggera i bazy danych.

ViewModels i *Views* - foldery zawierające View i ViewModele.

2.1.2 EngineerDashboard.Database

EngineerDashboard.Database

- |— AppDbContext.cs
- |— Models
- |— Services
- |— docker-compose.yml
- |— init

AppDbContext.cs - plik tworzący kontekst bazy danych.

Models - folder zawierający klasy modeli bazodanowych.

Services - folder zawierający usługi realizacji zapytań.

init - folder zawierający pliki konfiguracyjne bazy danych.

docker-compose.yml - plik tworzący kontener z wykorzystaniem plików konfiguracyjnych.

2.1.3 EngineerDashboard.Telemetry

EngineerDashboard.Telemetry

- |— Data
- |— Enums.cs
- |— Packets
- |— TelemetryClient.cs

Data - folder zawierający struktury danych pochodzących z telemetrii.

Packets - folder zawierający pakiety pochodzące z telemetrii, implementujący struktury z folderu *Data*.

Enums.cs - plik zawierający wszystkie enumy pochodzące z telemetrii.

TelemetryClient.cs - plik zawierający implementację klienta telemetrii.

2.2 EngineerDashboard.Telemetry

Projekt *Telemetry* korzysta z klasy *UdpClient*, aby połączyć się z portem telemetry skonfigurowanym w grze *F1 23*.

```
private UdpClient _client;
private IPEndPoint _peerEndPoint;
private Timer _timeoutTimer;
private GCHandle _handle;
private Packet[] _defaultPackets = new Packet[] {
    Packet.CAR_DAMAGE, Packet.CAR_STATUS, Packet.CAR_TELEMETRY,
    Packet.LAP_DATA, Packet.SESSION, Packet.SESSION_HISTORY,
    Packet.EVENT, Packet.FINAL_CLASSIFICATION, Packet.PARTICIPANTS,
    Packet.MOTION, Packet.MOTION_EX, Packet.TYRE_SET,
    Packet.CAR_SETUPS, Packet.LOBBY_INFO
};
```

2.2.1 Klient

Klient nasłuchuje każdego adresu w sieci lokalnej, na wybranym porcie:

```
public TelemetryClient(int port, Packet[] enabledPackets = null)
{
    _client = new UdpClient(port);
    _peerEndPoint = new IPEndPoint(IPAddress.Any, port);

    _timeoutTimer = new Timer(TIMEOUT)
    {
        AutoReset = true
    };
    _timeoutTimer.Elapsed += TimeoutEvent;

    Connected = true;
    OnConnectedStatusChange?.Invoke(true);

    EnabledPackets = enabledPackets ?? _defaultPackets;

    _client.BeginReceive(new AsyncCallback(ReceiveCallback), null);
}
```

Tworzy delegaty i eventy dla każdego nadchodzącego pakietu:

```
// Delegates
public delegate void MotionDataReceiveDelegate(MotionPacket packet);
public delegate void LapDataReceiveDelegate(LapDataPacket packet);
public delegate void EventDetailsReceiveDelegate(EventPacket
packet);
public delegate void SessionDataReceiveDelegate(SessionPacket
packet);
public delegate void
ParticipantsDataReceiveDelegate(ParticipantsPacket packet);
public delegate void CarSetupDataReceiveDelegate(CarSetupPacket
packet);
public delegate void
CarTelemetryDataReceiveDelegate(CarTelemetryPacket packet);
public delegate void CarStatusDataReceiveDelegate(CarStatusPacket
packet);
public delegate void
FinalClassificationDataReceiveDelegate(FinalClassificationPacket
packet);
public delegate void LobbyInfoDataReceiveDelegate(LobbyInfoPacket
packet);
public delegate void CarDamageDataReceiveDelegate(CarDamagePacket
packet);
public delegate void
SessionHistoryDataReceiveDelegate(SessionHistoryPacket packet);
public delegate void TyreSetDataReceiveDelegate(TyreSetPacket
packet);
public delegate void MotionExDataReceiveDelegate(MotionExPacket
packet);
```

```
// Events
public event MotionDataReceiveDelegate OnMotionDataReceive;
public event LapDataReceiveDelegate OnLapDataReceive;
public event EventDetailsReceiveDelegate OnEventDetailsReceive;
public event SessionDataReceiveDelegate OnSessionDataReceive;
public event ParticipantsDataReceiveDelegate
OnParticipantsDataReceive;
public event CarSetupDataReceiveDelegate OnCarSetupDataReceive;
public event CarTelemetryDataReceiveDelegate
OnCarTelemetryDataReceive;
public event CarStatusDataReceiveDelegate OnCarStatusDataReceive;
public event FinalClassificationDataReceiveDelegate
OnFinalClassificationDataReceive;
public event LobbyInfoDataReceiveDelegate OnLobbyInfoDataReceive;
public event CarDamageDataReceiveDelegate OnCarDamageDataReceive;
public event SessionHistoryDataReceiveDelegate
OnSessionHistoryDataReceive;
public event TyreSetDataReceiveDelegate OnTyreSetDataReceive;
public event MotionExDataReceiveDelegate OnMotionExDataReceive;
```


Odbiera nadchodzące pakiety:

```
private void ReceiveCallback(IAsyncResult result)
{
    try
    {
        byte[] data = _client.EndReceive(result, ref _peerEndPoint);

        _handle = GCHandle.Alloc(data, GCHandleType.Pinned);

        var header = (PacketHeader)Marshal
            .PtrToStructure(
                _handle.AddrOfPinnedObject(),
                typeof(PacketHeader));

        if (EnabledPackets.Contains(header.packetId))
        {
            switch (header.packetId)
            {
                case Packet.MOTION:
                    var motionPacket = (MotionPacket)Marshal
                        .PtrToStructure(
                            _handle.AddrOfPinnedObject(),
                            typeof(MotionPacket));
                    OnMotionDataReceive?.Invoke(motionPacket);
                    break;

                case Packet.MOTION_EX:
                    var motionExPacket = (MotionExPacket)Marshal
                        .PtrToStructure(
                            _handle.AddrOfPinnedObject(),
                            typeof(MotionExPacket));
                    OnMotionExDataReceive?.Invoke(motionExPacket);
                    break;
            }
        }
    }
    catch {}
    finally {
        if (_handle.IsAllocated) _handle.Free();
        _client?.BeginReceive(new AsyncCallback(ReceiveCallback), null);
    }
}
```

2.2.2 Pakiety

Poniżej znajduje się lista wszystkich pakietów:

EngineerDashboard.Telemetry/Packets

- |— CarDamagePacket.cs
- |— CarSetupPacket.cs
- |— CarStatusPacket.cs
- |— CarTelemetryPacket.cs
- |— EventPacket.cs
- |— FinalClassificationPacket.cs
- |— LapDataPacket.cs
- |— LobbyInfoPacket.cs
- |— MotionExPacket.cs
- |— MotionPacket.cs
- |— PacketHeader.cs
- |— ParticipantsPacket.cs
- |— SessionHistoryPacket.cs
- |— SessionPacket.cs
- |— TyreSetPacket.cs

Struktura przykładowego pakietu *CarDamagePacket* wygląda następująco:

```
namespace EngineerDashboard.Telemetry.Packets;

[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct CarDamagePacket
{
    public PacketHeader header;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 22)]
    public CarDamageData[] carDamageData;
}
```

Każdy pakiet zawiera nagłówek *header* i tablicę obiektów konkretnych danych. Rozmiar wynosi 22 z uwagi na maksymalną ilość pojazdów w sesji.

Struktura nagłówka pakietu *PacketHeader*:

```
namespace EngineerDashboard.Telemetry.Packets;

[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct PacketHeader
{
    public ushort packetFormat;
    public byte gameYear;
    public byte gameMajorVersion;
    public byte gameMinorVersion;
    public byte packetVersion;
    public Packet packetId;
    public ulong sessionUID;
    public float sessionTime;
    public uint frameIdentifier;
    public uint overallFrameIdentifier;
    public byte playerCarIndex;
    public byte secondaryPlayerCarIndex;
}
```

Struktura przykładowych danych *CarDamageData* dla pakietu *CarDamage-
Packet* wygląda następująco:

```
namespace EngineerDashboard.Telemetry.Data;

[StructLayout(LayoutKind.Sequential, Pack = 1)]
public struct CarDamageData
{
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public float[] tyresWear;
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public byte[] tyresDamage;
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public byte[] brakesDamage;
    public byte frontLeftWingDamage;
    public byte frontRightWingDamage;
    public byte rearWingDamage;
    public byte floorDamage;
    public byte diffuserDamage;
    public byte sidepodDamage;
    public byte drsFault;
    public byte ersFault;
    public byte gearBoxDamage;
    public byte engineDamage;
    public byte engineMGUHWear;
    public byte engineESWear;
    public byte engineCEWear;
    public byte engineICEWear;
    public byte engineMGUKWear;
    public byte engineBlown;
    public byte engineSeized;
}
```

Typ każdej zmiennej musi być dokładnie dopasowany do typów pochodzących z dokumentacji telemetry [14] aby klient mógł prawidłowo odczytywać dane i nie przesunął się w przypadku dobrania zbyt dużego/małego typu.

Poniżej przykładowy wycinek z dokumentacji dla pakietu *CarDamagePacket*:

This packet details car damage parameters for all the cars in the race.

Frequency: 10 per second

Size: 953 bytes

Version: 1

```
struct CarDamageData
{
    float      m_tyresWear[4];           // Tyre wear (%)
    uint8      m_tyresDamage[4];         // Tyre damage (%)
    uint8      m_brakesDamage[4];        // Brakes damage (%)
    uint8      m_frontLeftWingDamage;    // Front left wing damage (%)
    uint8      m_frontRightWingDamage;   // Front right wing damage (%)
    uint8      m_rearWingDamage;         // Rear wing damage (%)
    uint8      m_floorDamage;            // Floor damage (%)
    uint8      m_diffuserDamage;         // Diffuser damage (%)
    uint8      m_sidepodDamage;          // Sidepod damage (%)
    uint8      m_drsFault;               // DRS fault, 0=OK, 1=fault
    uint8      m_ersFault;               // ERS fault, 0=OK, 1=fault
    uint8      m_gearBoxDamage;          // Gear box damage (%)
    uint8      m_engineDamage;           // Engine damage (%)
    uint8      m_engineMGUHWear;         // Engine wear MGU-H (%)
    uint8      m_engineESWear;           // Engine wear ES (%)
    uint8      m_engineCEWear;           // Engine wear CE (%)
    uint8      m_engineICEWear;          // Engine wear ICE (%)
    uint8      m_engineMGUKWear;         // Engine wear MGU-K (%)
    uint8      m_engineTCWear;           // Engine wear TC (%)
    uint8      m_engineBlown;            // Engine blown, 0=OK, 1=fault
    uint8      m_engineSeized;           // Engine seized, 0=OK, 1=fault
}

struct PacketCarDamageData
{
    PacketHeader m_header;               // Header
    CarDamageData m_carDamageData[22];
};
```

Niektóre pakiety przesyłają jedynie identyfikatory danych, dlatego należało stworzyć enumy, na przykład identyfikatory pogody:

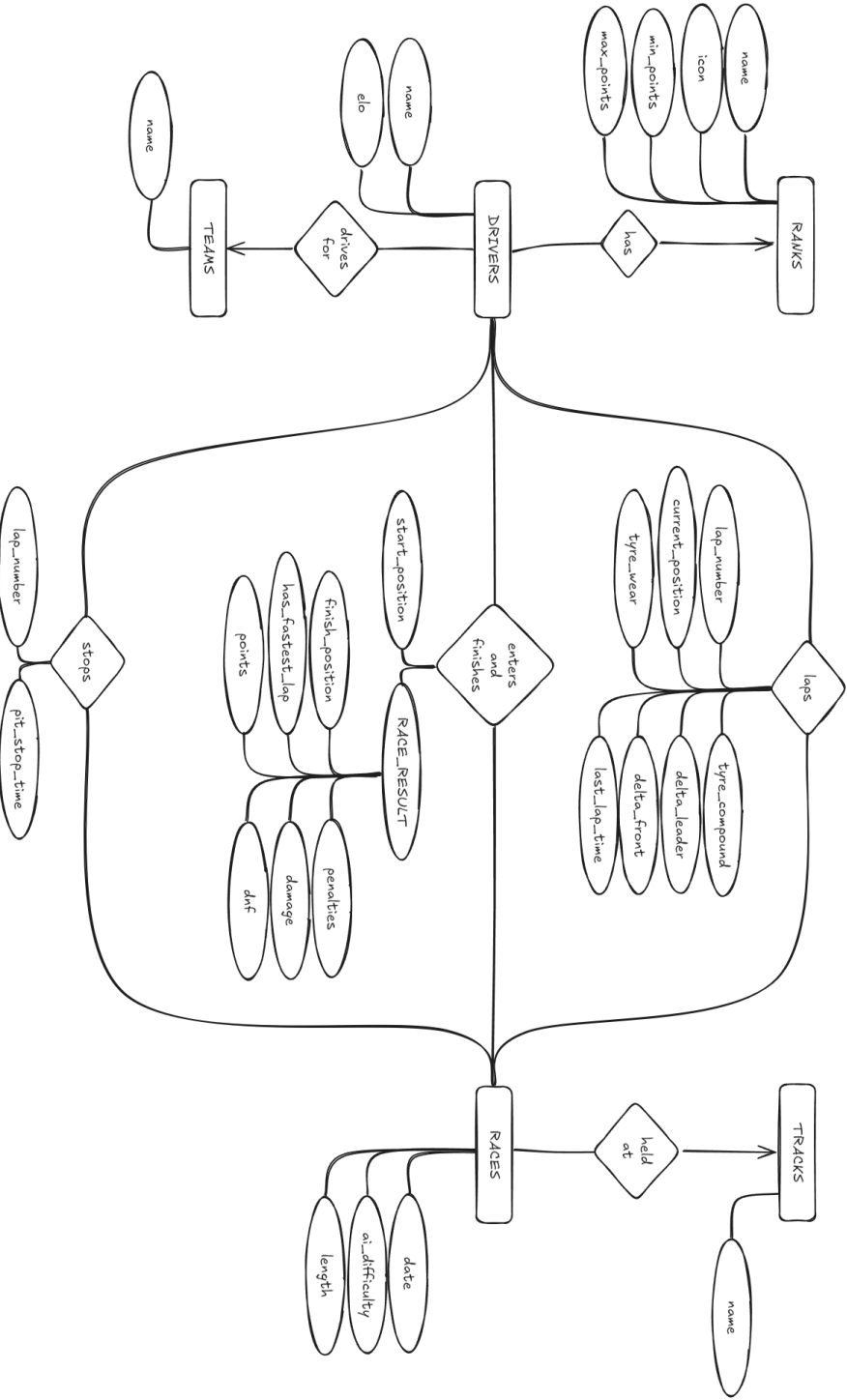
```
public enum Weather : byte
{
    CLEAR = 0,
    LIGHT_CLOUD = 1,
    OVERCAST = 2,
    LIGHT_RAIN = 3,
    HEAVY_RAIN = 4,
    STORM = 5
}
```

czy zespołów:

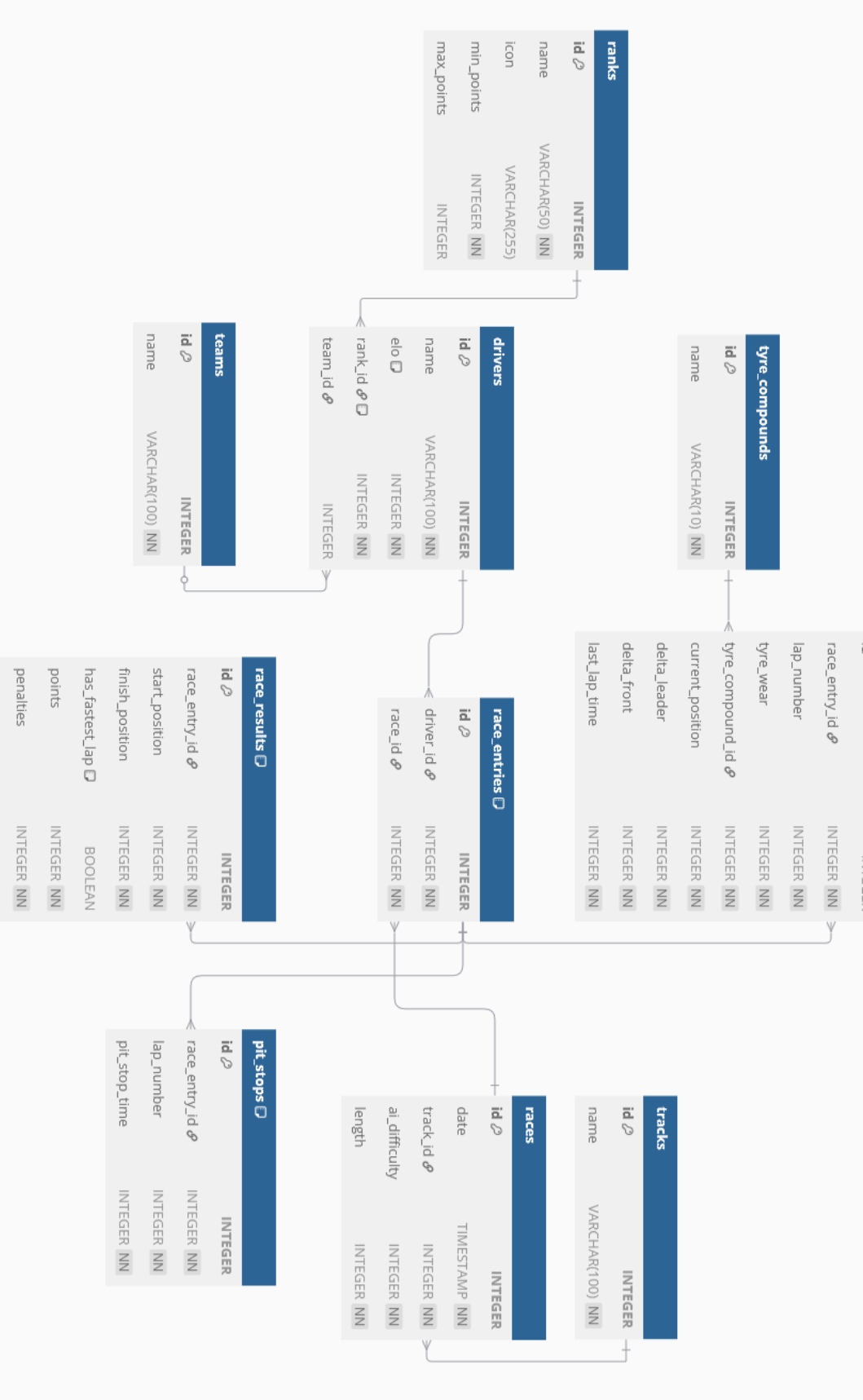
```
public enum Team : byte
{
    MERCEDES = 0,
    FERRARI = 1,
    REDBULL = 2,
    WILLIAMS = 3,
    ASTONMARTIN = 4,
    ALPINE = 5,
    ALPHATAURI = 6,
    HAAS = 7,
    MCLAREN = 8,
    ALFAROMEIO = 9,
    F1WORLD = 104
}
```

2.3 EngineerDashboard.Database

Projekt Database implementuje poniższe schematy (szczegóły w Sekcja 5):



Rysunek 1: Schemat związków encji bazy danych



Rysunek 2: Schemat relacyjnej bazy danych

Baza danych tworzona jest lokalnie w kontenerze z wykorzystaniem *docker-compose.yml*:

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    container_name: telemetry-postgres
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d
    ports:
      - "5432:5432"
    restart: always
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U admin -d telemetry" ]
      interval: 10s
      retries: 5
      start_period: 10s
      timeout: 5s

  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "8080:80"
    depends_on:
      - postgres
    restart: always

volumes:
  postgres_data:
```

W folderze *init* znajdują się 4 pliki:

```
EngineerDashboard.Database/init
├─ functions.sql
├─ init.sql
├─ procedures.sql
└─ views.sql
```

init.sql zawiera cały schemat bazy danych, wraz z użytkownikami.

functions.sql, *procedures.sql* i *views.sql* zawierają przykładowe funkcje, procedury i widoki, które można wykonać na bazie.

Baza danych posiada 3 użytkowników:

- *fia* - organ zarządzający serią wyścigową, może zmieniać zestaw torów, zespoły, kierowców oraz ich rangi,
- *stewards* - sędziowie, którzy mogą nakładać kary i zmieniać pozycje końcowe wyścigu,
- *telemetry_client* - użytkownik, z którego korzysta aplikacja, aby wprowadzać dane do bazy.

W folderze *Models* znajdują się klasy modeli dla każdej istniejącej tabeli:

```
EngineerDashboard.Database/Models
├─ Driver.cs
├─ Lap.cs
├─ PitStop.cs
├─ Race.cs
├─ RaceEntry.cs
├─ RaceResult.cs
├─ Rank.cs
├─ Team.cs
├─ Track.cs
└─ TyreCompound.cs
```

W folderze *Services* znajdują się usługi zapytań na konkretnych tabelach:

```
EngineerDashboard.Database/Services
├─ DriverService.cs
├─ LapService.cs
├─ PitStopService.cs
├─ RaceEntryService.cs
├─ RaceResultService.cs
└─ RaceService.cs
```

Przykładowa implementacja modelu *RaceEntry*, która wyznacza kolumny i relacje z innymi tabelami:

```
namespace EngineerDashboard.Database.Models;

public class RaceEntry
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int id { get; set; }

    [Column("driver_id")]
    public int driver_id { get; set; }

    [Column("race_id")]
    public int race_id { get; set; }

    [ForeignKey("driver_id")]
    public virtual Driver driver { get; set; } = null!;

    [ForeignKey("race_id")]
    public virtual Race race { get; set; } = null!;

    public virtual ICollection<Lap> laps { get; set; } =
        new List<Lap>();
    public virtual ICollection<PitStop> pit_stops { get; set; } =
        new List<PitStop>();
    public virtual RaceResult? race_result { get; set; }
}
```

Przykładowa implementacja usługi zapytań na tabeli *RaceEntries*:

```
namespace EngineerDashboard.Database.Services;

public class RaceEntryService
{
    public static async Task<RaceEntry>
    CreateRaceEntryAsync(AppDbContext context, RaceEntry raceEntry)
    {
        if (!await context.Drivers.AnyAsync(d =>
            d.id == raceEntry.driver_id))
            throw new InvalidOperationException("Driver not found");

        if (!await context.Races.AnyAsync(r =>
            r.id == raceEntry.race_id))
            throw new InvalidOperationException("Race not found");

        context.RaceEntries.Add(raceEntry);
        await context.SaveChangesAsync();

        return raceEntry;
    }

    public static async Task<List<RaceEntry>>
    GetRaceEntriesByDriverAsync(AppDbContext context, int driverId)
    {
        if (!await context.Drivers.AnyAsync(d => d.id == driverId))
            throw new InvalidOperationException("Driver not found");

        return await context.RaceEntries
            .Where(entry => entry.driver_id == driverId)
            .Include(entry => entry.race)
            .Include(entry => entry.race_result)
            .Include(entry => entry.race.track)
            .OrderByDescending(re => re.race.date)
            .ToListAsync();
    }
}
```

Metoda *CreateRaceEntryAsync* tworzy nowe *Race Entry* w bazie danych i zwraca je jako obiekt.

Metoda *GetRaceEntriesByDriverAsync* zwraca listę obiektów *RaceEntry* pobraną z bazy dla danego kierowcy, wraz z wybranymi danymi z innych tabel.

Automatyczne tworzenie zapytań umożliwia implementacja kontekstu bazy danych *AppDbContext*, która zawiera wszystkie mapowania każdej tabeli, wszystkich ich atrybutów i relacji:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Team>(entity =>
    {
        entity.ToTable("teams");
        entity.HasKey(e => e.id);
        entity.Property(e => e.id)
            .ValueGeneratedNever();
        entity.Property(e => e.name)
            .IsRequired().HasMaxLength(100);
    });
}
```

(pozostałe tabele...)

```
modelBuilder.Entity<Driver>(entity =>
{
    entity.ToTable("drivers");
    entity.HasKey(e => e.id);
    entity.Property(e => e.id)
        .ValueGeneratedNever();
    entity.Property(e => e.name)
        .IsRequired().HasMaxLength(100);
    entity.Property(e => e.elo)
        .IsRequired().HasDefaultValue(1000);
    entity.Property(e => e.rank_id)
        .IsRequired().HasDefaultValue(1);
    entity.HasOne(e => e.rank)
        .WithMany(r => r.drivers)
        .HasForeignKey(e => e.rank_id)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasOne(e => e.team)
        .WithMany(t => t.drivers)
        .HasForeignKey(e => e.team_id)
        .OnDelete(DeleteBehavior.SetNull);
    entity.HasMany(e => e.race_entries)
        .WithOne(re => re.driver)
        .HasForeignKey(re => re.driver_id)
        .OnDelete(DeleteBehavior.Cascade);
});
```

(pozostałe tabele...)

```
}
```

2.4 EngineerDashboard.App

Punktem startowym aplikacji desktopowej jest plik *App.xaml.cs*, w którym zaimplementowany został wzorzec projektowy *dependency injection*. Na jego początku zarejestrowany zostaje kontekst bazy danych i *singleton* usługi *DatabaseService*:

```
DotNetEnv.Env.TraversePath().Load();

ServiceCollection services = new ServiceCollection();

services.AddSingleton<TelemetryProvider>();
services.AddSingleton<TelemetryLoggerService>();

services.AddDbContextFactory<AppDbContext>(options =>
{
    var connectionString =
        $"Host={Environment.GetEnvironmentVariable("DATABASE_ADDRESS")};" +
        $"Port={Environment.GetEnvironmentVariable("DATABASE_PORT")};" +
        $"Database={Environment.GetEnvironmentVariable("DATABASE_NAME")};" +
        $"Username={Environment.GetEnvironmentVariable("DATABASE_USER")};" +
        $"Password={Environment.GetEnvironmentVariable("DATABASE_PASSWORD")};";

    options.UseNpgsql(connectionString);
});
services.AddSingleton<DatabaseService>();
```

Z uwagi na naturę aplikacji, w której żadne zapytanie nie jest wykonywane równoległe do innego i nie korzysta z tego samego kontekstu, możemy zarejestrować *DatabaseService* jako *singleton*, a nie jako *transient*.

Następnie rejestrowane są wszystkie stworzone *ViewModele* oraz *Modele*, z których później korzystają *View*:

```
services.AddSingleton<MainWindowViewModel>();
services.AddSingleton<SessionInfoViewModel>();
services.AddSingleton<DriversRowViewModel>();
services.AddSingleton<DriversTableViewModel>();
services.AddSingleton<TyreCardViewModel>();
services.AddSingleton<LapTimeChartViewModel>();
services.AddSingleton<TyreWearChartViewModel>();
services.AddSingleton<FuelChartViewModel>();
services.AddSingleton<BatteryChartViewModel>();
services.AddSingleton<InputsChartViewModel>();
services.AddSingleton<TelemetryChartViewModel>();
services.AddSingleton<CarInfoCardViewModel>();
services.AddSingleton<DamageCardViewModel>();
services.AddSingleton<DriversRankingViewModel>();
services.AddSingleton<DriverInfoViewModel>();
services.AddSingleton<RaceLapsViewModel>();

services.AddSingleton<Driver>();
services.AddSingleton<RaceEntry>();
```

Kolejne rejestrowane są *View*, a na sam koniec główne okno *MainWindow* i *serviceProvider*:

```
services.AddSingleton<SessionInfoView>(sp =>
    new SessionInfoView {
        DataContext = sp.GetRequiredService<SessionInfoViewModel>()
    });
services.AddSingleton<DriversRowView>(sp =>
    new DriversRowView {
        DataContext = sp.GetRequiredService<DriversRowViewModel>()
    });
services.AddSingleton<DriversTableView>(sp =>
    new DriversTableView {
        DataContext = sp.GetRequiredService<DriversTableViewModel>()
    });

(pozostałe View...)

services.AddSingleton<RaceLapsView>(sp =>
    new RaceLapsView {
        DataContext = sp.GetRequiredService<RaceLapsViewModel>()
    });

services.AddSingleton<MainWindow>();

_serviceProvider = services.BuildServiceProvider();
```


Przy uruchomieniu aplikacji, uruchamiane są także dwie usługi działające w tle - *TelemetryLoggerService* oraz *DatabaseService* i pokazywane jest główne okno aplikacji. Przed zamknięciem aplikacji czyszczony jest *serviceProvider*:

```
protected override void OnStartup(StartupEventArgs e)
{
    base.OnStartup(e);

    _ = _serviceProvider
        .GetRequiredService<TelemetryLoggerService>();
    _ = _serviceProvider
        .GetRequiredService<DatabaseService>();

    using var dbContext = _serviceProvider
        .GetRequiredService<IDbContextFactory<AppDbContext>>()
        .CreateDbContext();

    (...)

    var mainWindow =
    _serviceProvider.GetRequiredService<MainWindow>();
    var mainViewModel =
    _serviceProvider.GetRequiredService<MainWindowViewModel>();
    mainWindow.DataContext = mainViewModel;

    mainWindow.Show();
}

protected override void OnExit(ExitEventArgs e)
{
    if (_serviceProvider is IDisposable disposable)
    {
        disposable.Dispose();
    }
    base.OnExit(e);
}
```

2.4.1 Services

W tym projekcie stworzone zostały cztery usługi:

```
EngineerDashboard.App/Services
├─ DatabaseService.cs
├─ PartialTelemetryRow.cs
├─ TelemetryLoggerService.cs
└─ TelemetryProvider.cs
```

DatabaseService.cs zawierający implementację zapisu danych telemetrycznych do bazy i będący pośrednikiem dla zapytań pochodzących z aplikacji.

PartialTelemetryRow.cs i *TelemetryLoggerService.cs* zawierające implementację zapisu danych telemetrycznych do pliku *.csv*.

TelemetryProvider.cs będący pośrednikiem między *TelemetryClient* a aplikacją.

2.4.1.1 TelemetryProvider

W pierwszej kolejności dostawca telemetry *TelemetryProvider* tworzy instancję klienta telemetry *TelemetryClient*, pobierając port z pliku *.env*:

```
private readonly TelemetryClient _telemetryClient =
    new(int.Parse(
        Environment.GetEnvironmentVariable("TELEMETRY_CLIENT_PORT")
    ));
```

Następnie tworzy *subject* dla każdego pakietu, czyli obiekt który jednocześnie jest obserwatorem i może być obserwowany:

```
private readonly Subject<CarDamagePacket> _carDamageSubject = new();
private readonly Subject<CarSetupPacket> _carSetupSubject = new();
private readonly Subject<CarStatusPacket> _carStatusSubject = new();
(pozostałe subjecty...)
```

Tworzy *strumienie* dla każdego pakietu, które będą powiadamiać ich obserwatorów:

```
public IObservable<CarDamagePacket> CarDamageStream =>
    _carDamageSubject;
public IObservable<CarSetupPacket> CarSetupStream =>
    _carSetupSubject;
public IObservable<CarStatusPacket> CarStatusStream =>
    _carStatusSubject;
(pozostałe pakiety...)
```

Podpina do nich *eventy* pochodzące z *TelemetryClient*:

```
public TelemetryProvider()
{
    _telemetryClient.OnCarDamageDataReceive += packet =>
        _carDamageSubject.OnNext(packet);
    _telemetryClient.OnCarSetupDataReceive += packet =>
        _carSetupSubject.OnNext(packet);
    _telemetryClient.OnCarStatusDataReceive += packet =>
        _carStatusSubject.OnNext(packet);
    (pozostałe pakiety...)
}
```

Dzięki temu, usługa bazy danych może nasłuchiwać na wszystkie pakiety przychodzące na port i wykonywać z nimi własne operacje.

2.4.1.2 DatabaseService

Ta usługa korzysta ze stworzonego wcześniej *TelemetryProvider* i *AppDbContext*, więc dzięki *dependency injection* dostawca telemetrii i kontekst bazy danych mogą zostać „wstrzyknięci” bezpośrednio do konstruktora usługi:

```
private readonly CompositeDisposable _telemetrySubscription = new();
private readonly AppDbContext _context;

(...)

public DatabaseService(
    AppDbContext context,
    TelemetryProvider telemetryProvider
)
{
    (...)
}
```

Wewnątrz konstruktora do zmiennej prywatnej przypisywany jest kontekst bazy danych, a do grupy *telemetrySubscription* dodawane są *strumienie*, których *DatabaseService* będzie nasłuchiwał. Te strumienie dostarczają dane, które są wysyłane w trakcie wyścigu w tle:

```
_context = context;

_telemetrySubscription.Add(
    telemetryProvider.ParticipantsStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(OnParticipantsDataReceived));

_telemetrySubscription.Add(
    telemetryProvider.SessionStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(OnSessionDataReceived));
```

(pozostałe strumienie: *LapDataStream*, *CarDamageStream*, *CarStatusStream*, *EventStream*, *FinalClassificationStream*)

Przykładowa obsługa pakietu *SessionPacket*, który odbierany jest tylko raz, na początku wyścigu:

```
private void OnSessionDataReceived(SessionPacket packet)
{
    if (!_hasRaceInfo) return;

    _race = new Race
    {
        track_id = (int)packet.trackId,
        date = DateTime.UtcNow,
        ai_difficulty = packet.aiDifficulty,
        length = packet.totalLaps,
    };

    _hasRaceInfo = true;
    TrySaveToDatabase();
}
```

Funkcja *TrySaveToDatabase* to jedna z wielu, która korzysta z usług zapytań stworzonych wcześniej w projekcie *Database*:

```
private void TrySaveToDatabase()
{
    if (_hasSavedToDatabase) return;
    if (!_hasDrivers || !_hasRaceInfo || !_hasStartPositions) return;
    _hasSavedToDatabase = true;
    Task.Run(async () =>
    {
        for (int i = 0; i < _numDrivers; i++)
            _drivers[i].id = (await DriverService
                .CreateDriverAsync(_context, _drivers[i])).id;

        _raceId = (await RaceService
            .CreateRaceAsync(_context, _race)).id;

        for (int i = 0; i < _drivers.Count; i++)
        {
            var raceEntry = new RaceEntry
            { driver_id = _drivers[i].id,
              race_id = _raceId.Value };

            _raceEntries.Add(raceEntry);
            _raceEntries[i].id = (await RaceEntryService
                .CreateRaceEntryAsync(_context, raceEntry)).id;
        }
    });
}
```

Ta metoda ma za zadanie zebrać dane o wyścigu, listę kierowców i informacje o ich pozycjach startowych, a następnie zapisać je do bazy danych korzystając z metod *DriverService.CreateDriverAsync()*, *RaceService.CreateRaceAsync()* i *RaceEntryService.CreateRaceEntryAsync()*.

Wykonanie tych zapytań spowoduje stworzenie podstaw do umieszczenia w bazie kolejnych danych (okrążenia, pit stopy, wyniki).

Przykładowa obsługa pakietu *LapDataPacket*, który odbierany jest cały czas, ale zapisywany do bazy co okrążenie:

```
private void OnLapDataReceived(LapDataPacket packet)
{
    if (!_hasStartPositions)
    {
        foreach (var entry in packet.lapData)
            _startPositions.Add(entry.carPosition);

        _hasStartPositions = true;
        TrySaveToDatabase();
    }

    for (int i = 0; i < _numDrivers; i++)
    {
        var lapData = packet.lapData[i];
        var currentLapNum = lapData.currentLapNum;

        if (lapData.pitStopTimerInMS != 0)
            _latestPitStopTime[i] = lapData.pitStopTimerInMS;

        if (currentLapNum < 2)
            continue;

        if (_lastLoggedLapNums.TryGetValue(i, out var lastLoggedLap) &&
            lastLoggedLap == currentLapNum)
            continue;

        var lap = new Lap
        {
            race_entry_id = _raceEntries[i].id,
            lap_number = lapData.currentLapNum - 1,
            tyre_wear = (int)_latestTyreWear[i],
            tyre_compound_id = _latestVisualTyreCompound[i],
            current_position = lapData.carPosition,
            delta_leader = lapData.deltaToRaceLeaderInMS,
            delta_front = lapData.deltaToCarInFrontInMS,
            last_lap_time = (int)lapData.lastLapTimeInMS,
        };

        Task.Run(async () =>
            await LapService.CreateLapAsync(_context, lap);
        );

        _lastLoggedLapNums[i] = currentLapNum;
    }
}
```

Dodatkowo *DatabaseService* jest pośrednikiem między aplikacją a usługami bazy danych (wrapperem), dlatego implementuje własne metody zwracające wyniki zapytań, na przykład:

```
public async Task<Collection<Driver>> GetDrivers()
{
    var drivers = await DriverService
        .GetDrivers(_context);

    return drivers;
}

public async Task<int> GetDriverRacesCount(int driverId)
{
    var wins = await RaceEntryService
        .GetDriverRaceEntriesCount(_context, driverId);

    return wins;
}

public async Task<int> GetDriverWinsCount(int driverId)
{
    var wins = await RaceResultService
        .GetDriverWinsCount(_context, driverId);

    return wins;
}
```

2.4.2 Helpers

Folder *Helpers* zawiera klasy pomocnicze:

EngineerDashboard.App/Helpers

- |— ColorHelper.cs
- |— FontHelper.cs
- |— Formatter.cs

ColorHelper zwraca kolory *Brush* w zależności od podanych argumentów.

FontHelper ładuje czcionkę z pliku.

Formatter formatuje czasy do odpowiednich stringów.

2.4.2.1 FontHelper

W tej klasie istnieją dwie metody, pierwsza ładuje czcionkę z domyślnego pliku:

```
public static SKTypeface LoadCustomFont()
{
    try
    {
        var uri = new Uri("pack://application:,,,/Assets/Fonts/
JetBrainsMono-Regular.ttf");
        var info = System.Windows.Application.GetResourceStream(uri);

        if (info != null) return SKTypeface.FromStream(info.Stream);
    }
    catch (Exception ex)
    {
        Debug.WriteLine($"Failed to load custom font: {ex.Message}");
    }

    return SKTypeface.FromFamilyName("Consolas") ??
        SKTypeface.Default;
}
```

Druga, przeciążona, ładuje czcionkę z pliku podanego w argumencie:

```
public static SKTypeface LoadCustomFont(string fontFileName)
{
    (...)
}
```


2.4.2.2 ColorHelper

Przykładowa metoda zwracająca *Brush* w zależności od podanego zespołu kierowcy *GetTeamBrush*:

```
public static Brush GetTeamBrush(Team team) => team switch
{
    Team.MERCEDES => Brushes.Aqua,
    Team.FERRARI => Brushes.Red,
    Team.REDBULL => Brushes.RoyalBlue,
    Team.WILLIAMS => Brushes.DodgerBlue,
    Team.ASTONMARTIN => Brushes.ForestGreen,
    Team.ALPINE => Brushes.DeepSkyBlue,
    Team.ALPHATAURI => Brushes.SteelBlue,
    Team.HAAS => Brushes.LightGray,
    Team.MCLAREN => Brushes.DarkOrange,
    Team.ALFAROMEO => Brushes.DarkRed,
    Team.F1WORLD => Brushes.Goldenrod,
    _ => Brushes.Indigo
};
```

Przykładowy słownik mapujący temperaturę opon na kolor *Brush*:

```
[TyreCompound.C3] = new TyreGripProfile
{
    TemperatureToGrip = new Dictionary<byte, Brush>
    {
        [75] = Brushes.CornflowerBlue,
        [80] = Brushes.LightSkyBlue,
        [85] = Brushes.LightSkyBlue,
        [90] = Brushes.PaleGreen,
        [95] = Brushes.MediumSeaGreen,
        [100] = Brushes.MediumSeaGreen,
        [105] = Brushes.MediumSeaGreen,
        [110] = Brushes.Yellow,
        [115] = Brushes.Orange,
        [120] = Brushes.OrangeRed,
        [125] = Brushes.Red
    }
},
```

2.4.2.3 Formatter

Przykładowa metoda formatująca czas w milisekundach na minuty, sekundy i milisekundy:

```
public static string FormatMsToLapTimeString(uint ms)
{
    TimeSpan ts = TimeSpan.FromMilliseconds(ms);

    string formatted = string.Format("{0}:{1:D2}.{2:D3}",
        (int)ts.TotalMinutes,
        ts.Seconds,
        ts.Milliseconds);

    return formatted;
}
```

Przykładowa metoda formatująca czas w milisekundach na różnicę między dwoma samochodami:

```
public static string FormatMsToDeltaString(ushort ms)
{
    TimeSpan ts = TimeSpan.FromMilliseconds(ms);

    string formatted = string.Format("+{0}.{1:D3}",
        (int)ts.TotalSeconds,
        ts.Milliseconds);

    return formatted;
}
```

Przykładowa metoda formatująca czas w milisekundach na czas pit stopu:

```
public static string FormatMsToPitStopString(uint ms)
{
    TimeSpan ts = TimeSpan.FromMilliseconds(ms);

    return string.Format("{0}.{1:D3}",
        (int)ts.TotalSeconds,
        ts.Milliseconds);
}
```

2.4.3 Converters

Na frontendzie całej aplikacji wykorzystywane są różne *convertery*:

```
EngineerDashboard.App/Converters
├─ BoolToVisibilityConverter.cs
├─ PositionToBrushConverter.cs
├─ RankToBrushConverter.cs
├─ TeamToBrushConverter.cs
├─ TyreCompoundConverter.cs
├─ TyreToBrushConverter.cs
├─ TyreVisibilityConverter.cs
└─ WearToTextDecorationConverter.cs
```

BoolToVisibilityConverter konwertuje wartość zmiennej typu *bool* na widoczność elementu w interfejsie użytkownika.

PositionToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor pozycji kierowcy na mecie.

RankToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor rangi kierowcy w rankingu.

TeamToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor zespołu kierowcy.

TyreCompoundConverter konwertuje obiekt klasy *VisualTyreCompound* na jej skrótowe oznaczenie.

TyreToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor mieszanki opon.

TyreVisibilityConverter konwertuje wartości zmiennych typu *bool* na widoczność opon wśród dostępnych mieszanek.

WearToTextDecorationConverter konwertuje wartość zużycia opon na dekorator skrótowego oznaczenia opon wśród dostępnych mieszanek.

Przykładowy konwerter widoczności *BoolToVisibility*:

```
public object Convert(object value, Type targetType,
                    object parameter, CultureInfo culture)
{
    return (bool)value ?
        Visibility.Visible :
        Visibility.Collapsed;
}
```

Przykładowy konwerter koloru *RankToBrushConverter*:

```
public object Convert(object value, Type targetType,
                    object parameter, CultureInfo culture)
{
    if (value is string rankName) return ColorHelper
        .GetRankBrush(rankName);
    return Brushes.Gray;
}
```

Przykładowy konwerter oznaczenia *TyreCompoundConverter*:

```
public object Convert(object value, Type targetType,
                    object parameter, CultureInfo culture)
{
    if (value is VisualTyreCompound compound)
    {
        return compound switch
        {
            VisualTyreCompound.SOFT => "(S)",
            VisualTyreCompound.MEDIUM => "(M)",
            VisualTyreCompound.HARD => "(H)",
            VisualTyreCompound.INTER => "(I)",
            VisualTyreCompound.WET => "(W)",
            _ => string.Empty
        };
    }
    return string.Empty;
}
```

2.4.4 MainWindow

Plik *MainWindow.xaml* zawiera trzy główne komponenty - panel z informacjami o sesji, przyciski nawigacyjne i główną zawartość.

Każdy przycisk w menu nawigacyjnym ma przypisaną komendę, która zmienia zawartość głównej części ekranu:

```
<ui:Button Command="{Binding ShowDriversPageCommand}"
Padding="10,10" Margin="0 10">
  <StackPanel Orientation="Horizontal">
    <Image
      Source="pack://application:,,,
              /Assets/Icons/drivers-table-icon.png"
      Width="16"
      Height="16" />
  </StackPanel>
</ui:Button>

<Button Command="{Binding ShowCarSetupCommand}">
  (...)
</Button>

<Button Command="{Binding ShowChartsPageCommand}">
  (...)
</Button>

<Button Command="{Binding ShowInputsAndTelemetryPageCommand}">
  (...)
</Button>

<Button Command="{Binding ShowRankingPageCommand}">
  (...)
</Button>

Panel z informacjami o sesji, tak jak menu nawigacyjne, jest zawsze widoczny.
CurrentPageView jest kontenerem na zmienną zawartość:

<ContentControl Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="4"
  Margin="0 10"
  Content="{Binding SessionInfoView}" />

<ContentControl Grid.Row="1" Grid.Column="1" Grid.ColumnSpan="3"
  Content="{Binding CurrentPageView}" />
```

MainWindowViewModel zarządza wszystkimi *View* i *ViewModelami*. Tak samo jak w przypadku usług, wystarczy umieścić wszystkie zależności w konstruktorze *MainWindowViewModelu* a *dependency injection* nam je podepnie. Używane są w nim również niektóre *ViewModele* i *DatabaseService*, a do *driversRankingViewModel* podpięty jest *event* wybrania kierowcy z rankingu. Na koniec uruchamiana jest główna strona z tabelą kierowców:

```
public MainWindowViewModel(
    SessionInfoView sessionInfoView,
    DriversTableView driversTableView,
    TyreCardView tyreCardView,
    LapTimeChartView lapTimeChartView,
    TyreWearChartView tyreWearChartView,
    (pozostałe view...)
    DriversRankingViewModel driversRankingViewModel,
    DriverInfoViewModel driverInfoViewModel,
    RaceLapsViewModel raceLapsViewModel,
    DatabaseService databaseService)
{
    SessionInfoView = sessionInfoView;

    _driversTableView = driversTableView;
    _tyreCardView = tyreCardView;
    _lapTimeChartView = lapTimeChartView;
    _tyreWearChartView = tyreWearChartView;
    (pozostałe view...)

    _driversRankingViewModel = driversRankingViewModel;
    _driverInfoViewModel = driverInfoViewModel;
    _raceLapsViewModel = raceLapsViewModel;

    _databaseService = databaseService;

    _driversRankingViewModel.DriverSelected +=
        ShowDriverInfoPage;

    ShowDriversPage();
}
```

Przykładowa komenda wybrania zawartości *ShowDriversPage* na głównej stronie interfejsu użytkownika:

```
[RelayCommand]
private void ShowDriversPage()
{
    if (_currentPage == Page.Drivers)
        return;

    _currentPage = Page.Drivers;
    CurrentPageView = _driversTableView;
}
```

Przykładowa komenda wybrania zawartości *ShowInputsAndTelemetryPage* wraz z utworzeniem dynamicznej siatki:

```
[RelayCommand]
private void ShowInputsAndTelemetryPage()
{
    if (_currentPage == Page.InputsAndTelemetry)
        return;

    _currentPage = Page.InputsAndTelemetry;

    RemoveFromParent(_inputsChartView);
    RemoveFromParent(_telemetryChartView);

    var telemetryGrid = new Grid();
    telemetryGrid.RowDefinitions.Add(new RowDefinition());
    telemetryGrid.RowDefinitions.Add(new RowDefinition());

    telemetryGrid.SetValue(
        Grid.MarginProperty, new Thickness(30, 0, 30, 0)
    );

    _inputsChartView.SetValue(Grid.RowProperty, 0);
    _telemetryChartView.SetValue(Grid.RowProperty, 1);

    telemetryGrid.Children.Add(_inputsChartView);
    telemetryGrid.Children.Add(_telemetryChartView);

    CurrentPageView = telemetryGrid;
}
```

Przykładowa komenda wybrania zawartości *ShowRankingPage* wraz z uruchomieniem asynchronicznej metody pobrania danych o kierowcach z bazy:

```
[RelayCommand]
private async Task ShowRankingPage()
{
    _currentPage = Page.Ranking;
    await _driversRankingViewModel.LoadDriversAsync();
    CurrentPageView = _driversRankingView;
}
```

Przykładowa komenda wybrania zawartości *ShowDriverInfoPage* z obsługą eventu *GoBackRequested* dla usprawnienia nawigacji między panelami:

```
[RelayCommand]
private void ShowDriverInfoPage(Driver driver)
{
    if (driver == null) return;

    _currentPage = Page.DriverInfo;
    var driverInfoViewModel = new(_databaseService, driver);

    if (_driverInfoView.DataContext is
        DriverInfoViewModel previousViewModel)
    {
        previousViewModel.RaceSelected -= ShowLapsPage;
        previousViewModel.GoBackRequested -= async () =>
        {
            _currentPage = Page.Ranking;
            _driversRankingView.DataContext = _driversRankingViewModel;
            await _driversRankingViewModel.LoadDriversAsync();
            CurrentPageView = _driversRankingView;
        };
    }

    driverInfoViewModel.RaceSelected += ShowLapsPage;
    driverInfoViewModel.GoBackRequested += async () =>
    {
        _currentPage = Page.Ranking;
        _driversRankingView.DataContext = _driversRankingViewModel;
        await _driversRankingViewModel.LoadDriversAsync();
        CurrentPageView = _driversRankingView;
    };

    _driverInfoView.DataContext = driverInfoViewModel;
    CurrentPageView = _driverInfoView;
}
```


2.4.5 Views & ViewModels

Projekt zawiera następujące *ViewModele*:

EngineerDashboard.App/ViewModels

- |— BatteryChartViewModel.cs
- |— CarInfoCardViewModel.cs
- |— DamageCardViewModel.cs
- |— DriverInfoViewModel.cs
- |— DriversRankingViewModel.cs
- |— DriversRowViewModel.cs
- |— DriversTableViewModel.cs
- |— FuelChartViewModel.cs
- |— InputsChartViewModel.cs
- |— LapTimeChartViewModel.cs
- |— MainWindowViewModel.cs
- |— RaceLapsViewModel.cs
- |— SessionInfoViewModel.cs
- |— TelemetryChartViewModel.cs
- |— TyreCardViewModel.cs
- |— TyreSetViewModel.cs
- |— TyreWearChartViewModel.cs

Oraz odpowiadające im *View*:

EngineerDashboard.App/Views

- |— BatteryChartView.xaml
- |— CarInfoCardView.xaml
- |— DamageCardView.xaml
- |— DriverInfoView.xaml
- |— DriversRankingView.xaml
- |— DriversRowView.xaml
- |— DriversTableView.xaml
- |— FuelChartView.xaml
- |— InputsChartView.xaml
- |— LapTimeChartView.xaml
- |— RaceLapsView.xaml
- |— SessionInfoView.xaml
- |— TelemetryChartView.xaml
- |— TyreCardView.xaml
- |— TyreWearChartView.xaml

BatteryChart - wykres zużycia baterii na przestrzeni całego wyścigu.

CarInfoCard - dane o samochodzie i dostępnych oponach w pit stopie.

DamageCard - uproszczony schemat uszkodzeń pojazdu.

DriverInfo - szczegółowe informacje o kierowcy i historia jego wyścigów.

DriversRanking - ranking kierowców w formie listy.

DriversRow - pojedynczy wiersz tabeli kierowców w czasie rzeczywistym.

DriversTable - tabela wszystkich kierowców w czasie rzeczywistym.

FuelChart - wykres zużycia paliwa na przestrzeni całego wyścigu.

InputsChart - wykres aplikacji gazu i hamulca na przestrzeni pojedynczego okrążenia.

LapTimeChart - wykres czasów okrążeń na przestrzeni całego wyścigu.

RaceLaps - lista wszystkich okrążeń wybranego kierowcy w wybranym wyścigu.

SessionInfo - pasek informacji o aktualnej sesji.

TelemetryChart - wykres prędkości, biegów i aktywacji systemu DRS na przestrzeni pojedynczego okrążenia.

TyreCard - uproszczony schemat danych o oponach.

TyreWearChart - wykres zużycia opon na przestrzeni wyścigu.

Przykładowy *TyreCardViewModel* korzystający z *TelemetryProvidera*, znów w taki sam sposób jak usługi korzysta z *dependency injection*:

```
private readonly CompositeDisposable _telemetrySubscription = new();

public TyreCardViewModel(TelemetryProvider telemetryProvider)
{
    (...)
}
```

Wewnątrz konstruktora uruchamiana jest usługa podpinająca wszystkie *strumienie* do grupy *telemetrySubscription* oraz zainicjowane zostają wszystkie niezbędne zmienne:

```
public TyreCardViewModel(TelemetryProvider telemetryProvider)
{
    HookEvents(telemetryProvider);

    TyresWear = [0, 0, 0, 0];
    TyresInnerTemperature = [0, 0, 0, 0];
    TyresSurfaceTemperature = [0, 0, 0, 0];
    TyresPressure = [0, 0, 0, 0];
    VisualTyreCompound = VisualTyreCompound.SOFT;
    ActualTyreCompound = TyreCompound.C5;
    TyresAgeLaps = 0;
}

private void HookEvents(TelemetryProvider telemetryProvider)
{
    _telemetrySubscription.Add(
        telemetryProvider.CarTelemetryStream
            .ObserveOn(SynchronizationContext.Current)
            .Subscribe(OnCarTelemetryDataReceived));

    _telemetrySubscription.Add(
        telemetryProvider.CarStatusStream
            .ObserveOn(SynchronizationContext.Current)
            .Subscribe(OnCarStatusDataReceived));

    _telemetrySubscription.Add(
        telemetryProvider.CarDamageStream
            .ObserveOn(SynchronizationContext.Current)
            .Subscribe(OnCarDamageDataReceived));
}
```

Poniższe metody wywoływane są w momencie otrzymania pakietów i przypisują wartości do wybranych zmiennych:

```
private void OnCarTelemetryDataReceived(CarTelemetryPacket packet)
{
    var playerId = packet.header.playerCarIndex;
    var data = packet.carTelemetryData[playerId];

    TyresPressure = data.tyresPressure;
    TyresInnerTemperature = data.tyresInnerTemperature;
    TyresSurfaceTemperature = data.tyresSurfaceTemperature;
}

private void OnCarStatusDataReceived(CarStatusPacket packet)
{
    var playerId = packet.header.playerCarIndex;
    var data = packet.carStatusData[playerId];

    TyresAgeLaps = data.tyresAgeLaps;
    VisualTyreCompound = data.visualTyreCompound;
    ActualTyreCompound = data.actualTyreCompound;
}

private void OnCarDamageDataReceived(CarDamagePacket packet)
{
    var playerId = packet.header.playerCarIndex;
    var data = packet.carDamageData[playerId];

    TyresWear = data.tyresWear;
}
```

Dodatkowo stworzone zostały zmienne, które formatują zbierane dane w przyjazdy interfejsowi string lub przetrzymują kolory wykorzystywane później, na przykład tablice *TyresInnerTemperatureString* i *TyresInnerTemperatureColor*:

```
public string[] TyresInnerTemperatureString =>
[
    new string($"{TyresInnerTemperature[0]}°C"),
    new string($"{TyresInnerTemperature[1]}°C"),
    new string($"{TyresInnerTemperature[2]}°C"),
    new string($"{TyresInnerTemperature[3]}°C")
];

public Brush[] TyresInnerTemperatureColor =>
[
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[0]
    ),
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[1]
    ),
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[2]
    ),
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[3]
    )
];
```

Żeby można było z nich korzystać w *View* i ich zmiana była rejestrowana, należy wywołać na nich funkcję *OnPropertyChanged*:

```
partial void OnTyresInnerTemperatureChanged(byte[] oldValue, byte[]
newValue)
{
    OnPropertyChanged(nameof(TyresInnerTemperatureString));
    OnPropertyChanged(nameof(TyresInnerTemperatureColor));
}
```

Teraz w *TyreCardView* można wykorzystać wszystkie stworzone zmienne, na przykład do wyświetlania wskaźników temperatur opon:

```
<Border Grid.Row="1" Grid.Column="1"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[2]}" />
<Border Grid.Row="1" Grid.Column="4"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[3]}" />
<Border Grid.Row="3" Grid.Column="1"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[0]}" />
<Border Grid.Row="3" Grid.Column="4"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[1]}" />
```

Lub wyświetlania konkretnych temperatur:

```
<Border Grid.Row="1" Grid.Column="0"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[2]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
<Border Grid.Row="1" Grid.Column="5"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[3]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
<Border Grid.Row="3" Grid.Column="0"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[0]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
<Border Grid.Row="3" Grid.Column="5"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[1]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
```

W tym komponencie wykorzystane zostały także zasoby, takie jak czcionka wczytana z pliku:

```
<FontFamily x:Key="JetBrainsMonoExtraBold">
    pack://application,,,Assets/Fonts/#JetBrains Mono Bold
</FontFamily>
```

Czy style nałożone na dany element:

```
<Style x:Key="TyreTemperatureDot" TargetType="{x:Type Border}">
    <Setter Property="Margin" Value="0 6" />
    <Setter Property="Width" Value="28" />
    <Setter Property="Height" Value="16" />
    <Setter Property="VerticalAlignment" Value="Bottom" />
    <Setter Property="CornerRadius" Value="6" />
</Style>
```

Przykładowy *BatteryChartViewModel* korzystający z biblioteki LiveCharts2 [3] wymaga skonfigurowania obu osi:

```
public Axis[] XAxes =>
[
    new Axis
    {
        Name = "Battery usage",
        MinStep = 1,
        Labeler = value => $"{value:F0}",
        LabelsPaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        },
        NamePaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        }
    }
];
```

```
public Axis[] YAxes =>
[
    new Axis
    {
        Labeler = value => $"{value:F0}%",
        LabelsPaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        },
        NamePaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        },
        SeparatorsPaint = new SolidColorPaint
        {
            Color = new SKColor(48, 48, 48)
        }
    }
];
```


W jego konstruktorze poza podpięciem *strumieni* i zainicjowaniem wykresów, znajduje się także przypisanie czcionki z użyciem klasy pomocniczej:

```
public BatteryChartViewModel(TelemetryProvider telemetryProvider)
{
    _customTypeface = FontHelper.LoadCustomFont();

    Ers = new ObservableCollection<ObservablePoint>();

    Series = new ISeries[]
    {
        new LineSeries<ObservablePoint>
        {
            Values = Ers,
            Name = "Ers",
            GeometrySize = 6,
            Stroke = new SolidColorBrush(new SKColor(31, 119, 180), 2),
            GeometryStroke = new SolidColorBrush(
                new SKColor(31, 119, 180), 2
            ),
            Fill = null,
            YToolTipLabelFormatter = point =>
                $"Lap {point.Coordinate.SecondaryValue:F0}:" +
                $"{Math.Round(point.Coordinate.PrimaryValue, 2)}%"
        }
    };

    HookEvents(telemetryProvider);
}
```

Wykorzystanie tego *ViewModelu* w *BatteryChartView* wygląda następująco:

```
<lvc:CartesianChart
    Series="{Binding Series}"
    XAxes="{Binding XAxes}"
    YAxes="{Binding YAxes}" />
```

2.4.6 Services

W tym projekcie stworzone zostały cztery usługi:

```
EngineerDashboard.App/Services
├─ DatabaseService.cs
├─ PartialTelemetryRow.cs
├─ TelemetryLoggerService.cs
└─ TelemetryProvider.cs
```

DatabaseService.cs zawierający implementację zapisu danych telemetrycznych do bazy i będący pośrednikiem dla zapytań pochodzących z aplikacji.

PartialTelemetryRow.cs i *TelemetryLoggerService.cs* zawierające implementację zapisu danych telemetrycznych do pliku *.csv*.

TelemetryProvider.cs będący pośrednikiem między *TelemetryClient* a aplikacją.

2.4.6.1 TelemetryProvider

W pierwszej kolejności dostawca telemetry *TelemetryProvider* tworzy instancję klienta telemetry *TelemetryClient*, pobierając port z pliku *.env*:

```
private readonly TelemetryClient _telemetryClient =
    new(int.Parse(
        Environment.GetEnvironmentVariable("TELEMETRY_CLIENT_PORT")
    ));
```

Następnie tworzy *subject* dla każdego pakietu, czyli obiekt który jednocześnie jest obserwatorem i może być obserwowany:

```
private readonly Subject<CarDamagePacket> _carDamageSubject = new();
private readonly Subject<CarSetupPacket> _carSetupSubject = new();
private readonly Subject<CarStatusPacket> _carStatusSubject = new();
(pozostałe subjecty...)
```

Tworzy *strumienie* dla każdego pakietu, które będą powiadamiać ich obserwatorów:

```
public IObservable<CarDamagePacket> CarDamageStream =>
    _carDamageSubject;
public IObservable<CarSetupPacket> CarSetupStream =>
    _carSetupSubject;
public IObservable<CarStatusPacket> CarStatusStream =>
    _carStatusSubject;
(pozostałe pakiety...)
```

Podpina do nich *eventy* pochodzące z *TelemetryClient*:

```
public TelemetryProvider()
{
    _telemetryClient.OnCarDamageDataReceive += packet =>
        _carDamageSubject.OnNext(packet);
    _telemetryClient.OnCarSetupDataReceive += packet =>
        _carSetupSubject.OnNext(packet);
    _telemetryClient.OnCarStatusDataReceive += packet =>
        _carStatusSubject.OnNext(packet);
    (pozostałe pakiety...)
}
```

W momencie wyłączenia aplikacji czyści *TelemetryClient* i jego wszystkie zależności:

```
public void Dispose()
{
    _telemetryClient.Stop();

    _carDamageSubject.OnCompleted();
    _carDamageSubject.Dispose();

    _carSetupSubject.OnCompleted();
    _carSetupSubject.Dispose();

    _carStatusSubject.OnCompleted();
    _carStatusSubject.Dispose();

    (pozostałe subjecty...)
}
```

2.4.6.2 TelemetryLogger

Ta usługa korzysta ze stworzonego wcześniej *TelemetryProvider* i dzięki *dependency injection* dostawca telemetry może zostać „wstrzyknięty” bezpośrednio do konstruktora loggera:

```
private readonly CompositeDisposable _telemetrySubscription = new();

(...)
```

```
public TelemetryLoggerService(TelemetryProvider telemetryProvider)
{
    (...)
}
```

Wewnątrz konstruktora do grupy *telemetrySubscription* dodawane są *strumienie*, których *TelemetryLogger* będzie nasłuchiwał oraz uruchamiany jest timer, który umożliwia zapis danych co sekundę:

```
_telemetrySubscription.Add(
    telemetryProvider.CarDamageStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(packet => OnPacket(packet.header.sessionTime, row =>
            row.CarDamagePacket = packet)
        )
);
```

```
_telemetrySubscription.Add(
    telemetryProvider.CarStatusStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(packet => OnPacket(packet.header.sessionTime, row =>
            row.CarStatusPacket = packet)
        )
);
```

```
(pozostałe strumienie: CarTelemetryStream, LapDataStream,
SessionStream, EventStream, FinalClassificationStream)
```

```
_flushTimer = new Timer(_ => FlushCompletedRows(), null,
    TimeSpan.FromSeconds(1), TimeSpan.FromSeconds(1));
```

Pakiety otrzymane w ten sposób przekazywane są do *PartialTelemetryRow*:

```
private void OnPacket(
    float sessionTime,
    Action<PartialTelemetryRow> apply
)
{
    if (!_loggingActive) return;

    var row = _rows.GetOrAdd(
        sessionTime, _ => new PartialTelemetryRow
        { SessionTimestamp = sessionTime }
    );

    apply(row);
    row.LastUpdated = DateTime.UtcNow;
}
```

Który składa dane z wszystkich pakietów w jeden wiersz zapisywany do pliku:

```
private string parseDataToRow(int playerId)
{
    var carDamageData = CarDamagePacket?.carDamageData[playerId];
    var carStatusData = CarStatusPacket?.carStatusData[playerId];
    var carTelemetryData =
CarTelemetryPacket?.carTelemetryData[playerId];
    var lapData = LapDataPacket?.lapData[playerId];
    var sessionData = SessionPacket;

    var carDamageString = string.Join(',',
        Math.Round((double)carDamageData?.tyresWear[0], 3),
        Math.Round((double)carDamageData?.tyresWear[1], 3),
        Math.Round((double)carDamageData?.tyresWear[2], 3),
        Math.Round((double)carDamageData?.tyresWear[3], 3)
    );

    var carStatusString = string.Join(',',
        Math.Round((double)carStatusData?.fuelInTank, 3),
        (byte?)carStatusData?.actualTyreCompound,
        (byte?)carStatusData?.visualTyreCompound,
        carStatusData?.tyresAgeLaps,
        carStatusData?.ersStoreEnergy
    );

    (pozostałe stringi...)
}
```

Z odpowiednimi headerami:

```
public static string Header =>
    string.Join(
        ',',
        "Timestamp",
        _carDamageHeader,
        _carStatusHeader,
        _carTelemetryHeader,
        _lapHeader,
        _sessionHeader
    );
```

Loggowanie danych do pliku rozpoczyna się w momencie uzyskania pakietu *EventPacket* z kodem *SSTA* oznaczającym start sesji. Analogicznie loggowanie kończy się w momencie otrzymania kodu *SEND* oznaczającego koniec sesji. Dodatkowo wydarzenie uzyskania najszybszego czasu okrążenia również pochodzi z tego pakietu:

```
private void HandleEvent(EventPacket packet)
{
    var stringCode = new string(packet.eventStringCode);

    switch (stringCode)
    {
        case "SSTA":
            StartLogging();
            break;
        case "SEND":
            StopLogging();
            break;
        case "FTLP":
            _fastestLapDriverId =
                packet.eventDetails.fastestLap.vehicleIdx;
            break;
    }
}
```

Poniżej znajdują się implementacje metod rozpoczęcia i zatrzymania loggowania:

```
private void StartLogging()
{
    if (_loggingActive) return;

    string logDirectory =
        Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "logs");
    Directory.CreateDirectory(logDirectory);

    _logFilePath = Path.Combine(logDirectory, $"{DateTime.UtcNow:yyyy-MM-dd_HH-mm-ss}.csv");

    _headerWritten = false;
    _loggingActive = true;

    _flushTimer.Change(TimeSpan.Zero, TimeSpan.FromSeconds(1));
}

private void StopLogging()
{
    if (!_loggingActive) return;

    _flushTimer.Change(Timeout.Infinite, Timeout.Infinite);
    FlushCompletedRows();
}
```

Na koniec wyścigu przychodzi ostatni pakiet *FinalClassificationPacket*, który dodaje kilka kolumn do pliku *.csv*:

```
private void HandleFinalClassification(FinalClassificationPacket
packet)
{
    var playerId = packet.header.playerCarIndex;

    var data = packet.classificationData[(int)playerId];

    var lines = File.ReadAllLines(_logFilePath).ToList();

    lines[0] +=
        ",starting_position,finishing_position,dnf,penalties_time,fastest_lap";

    var startingPosition = data.gridPosition;
    var finishingPosition = data.position;
    var hasDnf = data.resultStatus != ResultStatus.FINISHED;
    var penaltiesTime = data.penaltiesTime;
    var hasFastestLap = _fastestLapDriverId == playerId;

    for (int i = 1; i < lines.Count; i++)
    {
        lines[i] += $",{startingPosition},{finishingPosition},{hasDnf},
{penaltiesTime},{hasFastestLap}";
    }

    File.WriteAllLines(_logFilePath, lines);

    _loggingActive = false;
    _logFilePath = null;
}
```


Plik *.csv* uzupełniany jest na bieżąco, więc w przypadku wystąpienia błędu aplikacji wciąż można odzyskać część danych:

```
private void WriteRowToFile(PartialTelemetryRow row)
{
    if (string.IsNullOrEmpty(_logFilePath)) return;

    var csvRow = row.ToCSV();
    if (string.IsNullOrWhiteSpace(csvRow)) return;

    lock (_fileLock)
    {
        using var writer = new StreamWriter(
            _logFilePath,
            append: true,
            Encoding.UTF8
        );

        if (!_headerWritten)
        {
            writer.WriteLine(PartialTelemetryRow.Header);
            _headerWritten = true;
        }

        writer.WriteLine(csvRow);
    }
}
```

2.4.6.3 DatabaseService

Identycznie jak w przypadku *TelemetryLogger* ta usługa korzysta ze stworzonego wcześniej *TelemetryProvider* i *AppDbContext*, więc dzięki *dependency injection* dostawca telemetry i kontekst bazy danych mogą zostać „wstrzyknięci” bezpośrednio do konstruktora usługi:

```
private readonly CompositeDisposable _telemetrySubscription = new();
private readonly AppDbContext _context;

(...)

public DatabaseService(
    AppDbContext context,
    TelemetryProvider telemetryProvider
)
{
    (...)
}
```

Wewnątrz konstruktora do zmiennej prywatnej przypisywany jest kontekst bazy danych, a do grupy *telemetrySubscription* dodawane są *strumienie*, których *DatabaseService* będzie nasłuchiwał. Te strumienie dostarczają dane, które są wysyłane w trakcie wyścigu w tle:

```
_context = context;

_telemetrySubscription.Add(
    telemetryProvider.ParticipantsStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(OnParticipantsDataReceived));

_telemetrySubscription.Add(
    telemetryProvider.SessionStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(OnSessionDataReceived));

(pozostałe strumienie: LapDataStream, CarDamageStream,
CarStatusStream, EvetnStream, FinalClassificationStream)
```

Przykładowa obsługa pakietu *SessionPacket*, który odbierany jest tylko raz, na początku wyścigu:

```
private void OnSessionDataReceived(SessionPacket packet)
{
    if (_hasRaceInfo) return;

    _race = new Race
    {
        track_id = (int)packet.trackId,
        date = DateTime.UtcNow,
        ai_difficulty = packet.aiDifficulty,
        length = packet.totalLaps,
    };

    _hasRaceInfo = true;
    TrySaveToDatabase();
}
```

Funkcja *TrySaveToDatabase* to jedna z wielu, która korzysta z wcześniej stworzonych usług w projekcie *Database*:

```
private void TrySaveToDatabase()
{
    if (_hasSavedToDatabase) return;
    if (!_hasDrivers || !_hasRaceInfo || !_hasStartPositions) return;
    _hasSavedToDatabase = true;
    Task.Run(async () =>
    {
        for (int i = 0; i < _numDrivers; i++)
            _drivers[i].id = (await DriverService
                .CreateDriverAsync(_context, _drivers[i])).id;

        _raceId = (await RaceService
            .CreateRaceAsync(_context, _race)).id;

        for (int i = 0; i < _drivers.Count; i++)
        {
            var raceEntry = new RaceEntry
            { driver_id = _drivers[i].id,
              race_id = _raceId.Value };

            _raceEntries.Add(raceEntry);
            _raceEntries[i].id = (await RaceEntryService
                .CreateRaceEntryAsync(_context, raceEntry)).id;
        }
    });
}
```

Przykładowa obsługa pakietu *LapDataPacket*, który odbierany jest cały czas, ale zapisywany do bazy co okrążenie:

```
private void OnLapDataReceived(LapDataPacket packet)
{
    if (!_hasStartPositions)
    {
        foreach (var entry in packet.lapData)
            _startPositions.Add(entry.carPosition);

        _hasStartPositions = true;
        TrySaveToDatabase();
    }

    for (int i = 0; i < _numDrivers; i++)
    {
        var lapData = packet.lapData[i];
        var currentLapNum = lapData.currentLapNum;

        if (lapData.pitStopTimerInMS != 0)
            _latestPitStopTime[i] = lapData.pitStopTimerInMS;

        if (currentLapNum < 2)
            continue;

        if (_lastLoggedLapNums.TryGetValue(i, out var lastLoggedLap) &&
            lastLoggedLap == currentLapNum)
            continue;

        var lap = new Lap
        {
            race_entry_id = _raceEntries[i].id,
            lap_number = lapData.currentLapNum - 1,
            tyre_wear = (int)_latestTyreWear[i],
            tyre_compound_id = _latestVisualTyreCompound[i],
            current_position = lapData.carPosition,
            delta_leader = lapData.deltaToRaceLeaderInMS,
            delta_front = lapData.deltaToCarInFrontInMS,
            last_lap_time = (int)lapData.lastLapTimeInMS,
        };

        Task.Run(async () =>
        {
            await LapService.CreateLapAsync(_context, lap);
        });

        _lastLoggedLapNums[i] = currentLapNum;
    }
}
```

Dodatkowo *DatabaseService* jest pośrednikiem między aplikacją a usługami bazy danych, dlatego implementuje własne metody zwracające wyniki zapytań, na przykład:

```
public async Task<Collection<Driver>> GetDrivers()
{
    var drivers = await DriverService
        .GetDrivers(_context);

    return drivers;
}

public async Task<int> GetDriverRacesCount(int driverId)
{
    var wins = await RaceEntryService
        .GetDriverRaceEntriesCount(_context, driverId);

    return wins;
}

public async Task<int> GetDriverWinsCount(int driverId)
{
    var wins = await RaceResultService
        .GetDriverWinsCount(_context, driverId);

    return wins;
}
```

2.4.7 Helpers

Folder *Helpers* zawiera klasy pomocnicze:

EngineerDashboard.App/Helpers

- |— ColorHelper.cs
- |— FontHelper.cs
- |— Formatter.cs

ColorHelper zwraca kolory *Brush* w zależności od podanych argumentów.

FontHelper ładuje czcionkę z pliku.

Formatter formatuje czasy do odpowiednich stringów.

2.4.7.1 FontHelper

W tej klasie istnieją dwie metody, pierwsza ładuje czcionkę z domyślnego pliku:

```
public static SKTypeface LoadCustomFont()
{
    try
    {
        var uri = new Uri("pack://application:,,,/Assets/Fonts/
JetBrainsMono-Regular.ttf");
        var info = System.Windows.Application.GetResourceStream(uri);

        if (info != null) return SKTypeface.FromStream(info.Stream);
    }
    catch (Exception ex)
    {
        Debug.WriteLine($"Failed to load custom font: {ex.Message}");
    }

    return SKTypeface.FromFamilyName("Consolas") ??
        SKTypeface.Default;
}
```

Druga, przeciążona, ładuje czcionkę z pliku podanego w argumencie:

```
public static SKTypeface LoadCustomFont(string fontFileName)
{
    (...)
}
```

2.4.7.2 ColorHelper

Przykładowa metoda zwracająca *Brush* w zależności od podanego zespołu kierowcy *GetTeamBrush*:

```
public static Brush GetTeamBrush(Team team) => team switch
{
    Team.MERCEDES => Brushes.Aqua,
    Team.FERRARI => Brushes.Red,
    Team.REDBULL => Brushes.RoyalBlue,
    Team.WILLIAMS => Brushes.DodgerBlue,
    Team.ASTONMARTIN => Brushes.ForestGreen,
    Team.ALPINE => Brushes.DeepSkyBlue,
    Team.ALPHATAURI => Brushes.SteelBlue,
    Team.HAAS => Brushes.LightGray,
    Team.MCLAREN => Brushes.DarkOrange,
    Team.ALFAROMEA => Brushes.DarkRed,
    Team.F1WORLD => Brushes.Goldenrod,
    _ => Brushes.Indigo
};
```

Przykładowy słownik mapujący temperaturę opon na kolor *Brush*:

```
[TyreCompound.C3] = new TyreGripProfile
{
    TemperatureToGrip = new Dictionary<byte, Brush>
    {
        [75] = Brushes.CornflowerBlue,
        [80] = Brushes.LightSkyBlue,
        [85] = Brushes.LightSkyBlue,
        [90] = Brushes.PaleGreen,
        [95] = Brushes.MediumSeaGreen,
        [100] = Brushes.MediumSeaGreen,
        [105] = Brushes.MediumSeaGreen,
        [110] = Brushes.Yellow,
        [115] = Brushes.Orange,
        [120] = Brushes.OrangeRed,
        [125] = Brushes.Red
    }
},
```

2.4.7.3 Formatter

Przykładowa metoda formatująca czas w milisekundach na minuty, sekundy i milisekundy:

```
public static string FormatMsToLapTimeString(uint ms)
{
    TimeSpan ts = TimeSpan.FromMilliseconds(ms);

    string formatted = string.Format("{0}:{1:D2}.{2:D3}",
        (int)ts.TotalMinutes,
        ts.Seconds,
        ts.Milliseconds);

    return formatted;
}
```

Przykładowa metoda formatująca czas w milisekundach na różnicę między dwoma samochodami:

```
public static string FormatMsToDeltaString(ushort ms)
{
    TimeSpan ts = TimeSpan.FromMilliseconds(ms);

    string formatted = string.Format("+{0}.{1:D3}",
        (int)ts.TotalSeconds,
        ts.Milliseconds);

    return formatted;
}
```

Przykładowa metoda formatująca czas w milisekundach na czas pit stopu:

```
public static string FormatMsToPitStopString(uint ms)
{
    TimeSpan ts = TimeSpan.FromMilliseconds(ms);

    return string.Format("{0}.{1:D3}",
        (int)ts.TotalSeconds,
        ts.Milliseconds);
}
```


2.4.8 Converters

Na frontendzie całej aplikacji wykorzystywane są różne *convertery*:

```
EngineerDashboard.App/Converters
├─ BoolToVisibilityConverter.cs
├─ PositionToBrushConverter.cs
├─ RankToBrushConverter.cs
├─ TeamToBrushConverter.cs
├─ TyreCompoundConverter.cs
├─ TyreToBrushConverter.cs
├─ TyreVisibilityConverter.cs
└─ WearToTextDecorationConverter.cs
```

BoolToVisibilityConverter konwertuje wartość zmiennej typu *bool* na widoczność elementu w interfejsie użytkownika.

PositionToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor pozycji kierowcy na mecie.

RankToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor rangi kierowcy w rankingu.

TeamToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor zespołu kierowcy.

TyreCompoundConverter konwertuje obiekt klasy *VisualTyreCompound* na jej skrótowe oznaczenie.

TyreToBrushConverter wykorzystuje *ColorHelper* aby zmienić kolor mieszanki opon.

TyreVisibilityConverter konwertuje wartości zmiennych typu *bool* na widoczność opon wśród dostępnych mieszanek.

WearToTextDecorationConverter konwertuje wartość zużycia opon na dekorator skrótowego oznaczenia opon wśród dostępnych mieszanek.

Przykładowy konwerter widoczności *BoolToVisibility*:

```
public object Convert(object value, Type targetType,
                    object parameter, CultureInfo culture)
{
    return (bool)value ?
        Visibility.Visible :
        Visibility.Collapsed;
}
```

Przykładowy konwerter koloru *RankToBrushConverter*:

```
public object Convert(object value, Type targetType,
                    object parameter, CultureInfo culture)
{
    if (value is string rankName) return ColorHelper
        .GetRankBrush(rankName);
    return Brushes.Gray;
}
```

Przykładowy konwerter oznaczenia *TyreCompoundConverter*:

```
public object Convert(object value, Type targetType,
                    object parameter, CultureInfo culture)
{
    if (value is VisualTyreCompound compound)
    {
        return compound switch
        {
            VisualTyreCompound.SOFT => "(S)",
            VisualTyreCompound.MEDIUM => "(M)",
            VisualTyreCompound.HARD => "(H)",
            VisualTyreCompound.INTER => "(I)",
            VisualTyreCompound.WET => "(W)",
            _ => string.Empty
        };
    }
    return string.Empty;
}
```

2.4.9 MainWindow

Plik *MainWindow.xaml* zawiera trzy główne komponenty - panel z informacjami o sesji, przyciski nawigacyjne i główną zawartość.

Każdy przycisk w menu nawigacyjnym ma przypisaną komendę, która zmienia zawartość głównej części ekranu:

```
<ui:Button Command="{Binding ShowDriversPageCommand}"
Padding="10,10" Margin="0 10">
  <StackPanel Orientation="Horizontal">
    <Image
      Source="pack://application:,,,
              /Assets/Icons/drivers-table-icon.png"
      Width="16"
      Height="16" />
  </StackPanel>
</ui:Button>

<Button Command="{Binding ShowCarSetupCommand}">
  (...)
</Button>

<Button Command="{Binding ShowChartsPageCommand}">
  (...)
</Button>

<Button Command="{Binding ShowInputsAndTelemetryPageCommand}">
  (...)
</Button>

<Button Command="{Binding ShowRankingPageCommand}">
  (...)
</Button>

Panel z informacjami o sesji, tak jak menu nawigacyjne, jest zawsze widoczny.
CurrentPageView jest kontenerem na zmienną zawartość:

<ContentControl Grid.Row="0" Grid.Column="0" Grid.ColumnSpan="4"
  Margin="0 10"
  Content="{Binding SessionInfoView}" />

<ContentControl Grid.Row="1" Grid.Column="1" Grid.ColumnSpan="3"
  Content="{Binding CurrentPageView}" />
```

MainWindowViewModel zarządza wszystkimi *View* i *ViewModelami*. Tak samo jak w przypadku usług, wystarczy umieścić wszystkie zależności w konstruktorze *MainWindowViewModelu* a *dependency injection* nam je poda. Używane są w nim również niektóre *ViewModele* i *DatabaseService*, a do *driversRankingViewModel* podpięty jest *event* wybrania kierowcy z rankingu. Na koniec uruchamiana jest główna strona z tabelą kierowców:

```
public MainWindowViewModel(
    SessionInfoView sessionInfoView,
    DriversTableView driversTableView,
    TyreCardView tyreCardView,
    LapTimeChartView lapTimeChartView,
    TyreWearChartView tyreWearChartView,
    (pozostałe view...)
    DriversRankingViewModel driversRankingViewModel,
    DriverInfoViewModel driverInfoViewModel,
    RaceLapsViewModel raceLapsViewModel,
    DatabaseService databaseService)
{
    SessionInfoView = sessionInfoView;

    _driversTableView = driversTableView;
    _tyreCardView = tyreCardView;
    _lapTimeChartView = lapTimeChartView;
    _tyreWearChartView = tyreWearChartView;
    (pozostałe view...)

    _driversRankingViewModel = driversRankingViewModel;
    _driverInfoViewModel = driverInfoViewModel;
    _raceLapsViewModel = raceLapsViewModel;

    _databaseService = databaseService;

    _driversRankingViewModel.DriverSelected +=
        ShowDriverInfoPage;

    ShowDriversPage();
}
```

Przykładowa komenda wybrania zawartości *ShowDriversPage* na głównej stronie interfejsu użytkownika:

```
[RelayCommand]
private void ShowDriversPage()
{
    if (_currentPage == Page.Drivers)
        return;

    _currentPage = Page.Drivers;
    CurrentPageView = _driversTableView;
}
```

Przykładowa komenda wybrania zawartości *ShowInputsAndTelemetryPage* wraz z utworzeniem dynamicznej siatki:

```
[RelayCommand]
private void ShowInputsAndTelemetryPage()
{
    if (_currentPage == Page.InputsAndTelemetry)
        return;

    _currentPage = Page.InputsAndTelemetry;

    RemoveFromParent(_inputsChartView);
    RemoveFromParent(_telemetryChartView);

    var telemetryGrid = new Grid();
    telemetryGrid.RowDefinitions.Add(new RowDefinition());
    telemetryGrid.RowDefinitions.Add(new RowDefinition());

    telemetryGrid.SetValue(
        Grid.MarginProperty, new Thickness(30, 0, 30, 0)
    );

    _inputsChartView.SetValue(Grid.RowProperty, 0);
    _telemetryChartView.SetValue(Grid.RowProperty, 1);

    telemetryGrid.Children.Add(_inputsChartView);
    telemetryGrid.Children.Add(_telemetryChartView);

    CurrentPageView = telemetryGrid;
}
```

Przykładowa komenda wybrania zawartości *ShowRankingPage* wraz z uruchomieniem asynchronicznej metody pobrania danych o kierowcach z bazy:

```
[RelayCommand]
private async Task ShowRankingPage()
{
    _currentPage = Page.Ranking;
    await _driversRankingViewModel.LoadDriversAsync();
    CurrentPageView = _driversRankingView;
}
```

Przykładowa komenda wybrania zawartości *ShowDriverInfoPage* z obsługą eventu *GoBackRequested* dla usprawnienia nawigacji między panelami:

```
[RelayCommand]
private void ShowDriverInfoPage(Driver driver)
{
    if (driver == null) return;

    _currentPage = Page.DriverInfo;
    var driverInfoViewModel = new(_databaseService, driver);

    if (_driverInfoView.DataContext is
        DriverInfoViewModel previousViewModel)
    {
        previousViewModel.RaceSelected -= ShowLapsPage;
        previousViewModel.GoBackRequested -= async () =>
        {
            _currentPage = Page.Ranking;
            _driversRankingView.DataContext = _driversRankingViewModel;
            await _driversRankingViewModel.LoadDriversAsync();
            CurrentPageView = _driversRankingView;
        };
    }

    driverInfoViewModel.RaceSelected += ShowLapsPage;
    driverInfoViewModel.GoBackRequested += async () =>
    {
        _currentPage = Page.Ranking;
        _driversRankingView.DataContext = _driversRankingViewModel;
        await _driversRankingViewModel.LoadDriversAsync();
        CurrentPageView = _driversRankingView;
    };

    _driverInfoView.DataContext = driverInfoViewModel;
    CurrentPageView = _driverInfoView;
}
```

2.4.10 Views & ViewModels

Projekt zawiera następujące *ViewModele*:

EngineerDashboard.App/ViewModels

- |— BatteryChartViewModel.cs
- |— CarInfoCardViewModel.cs
- |— DamageCardViewModel.cs
- |— DriverInfoViewModel.cs
- |— DriversRankingViewModel.cs
- |— DriversRowViewModel.cs
- |— DriversTableViewModel.cs
- |— FuelChartViewModel.cs
- |— InputsChartViewModel.cs
- |— LapTimeChartViewModel.cs
- |— MainWindowViewModel.cs
- |— RaceLapsViewModel.cs
- |— SessionInfoViewModel.cs
- |— TelemetryChartViewModel.cs
- |— TyreCardViewModel.cs
- |— TyreSetViewModel.cs
- |— TyreWearChartViewModel.cs

Oraz odpowiadające im *View*:

EngineerDashboard.App/Views

- |— BatteryChartView.xaml
- |— CarInfoCardView.xaml
- |— DamageCardView.xaml
- |— DriverInfoView.xaml
- |— DriversRankingView.xaml
- |— DriversRowView.xaml
- |— DriversTableView.xaml
- |— FuelChartView.xaml
- |— InputsChartView.xaml
- |— LapTimeChartView.xaml
- |— RaceLapsView.xaml
- |— SessionInfoView.xaml
- |— TelemetryChartView.xaml
- |— TyreCardView.xaml
- |— TyreWearChartView.xaml

BatteryChart - wykres zużycia baterii na przestrzeni całego wyścigu.

CarInfoCard - dane o samochodzie i dostępnych oponach w pit stopie.

DamageCard - uproszczony schemat uszkodzeń pojazdu.

DriverInfo - szczegółowe informacje o kierowcy i historia jego wyścigów.

DriversRanking - ranking kierowców w formie listy.

DriversRow - pojedynczy wiersz tabeli kierowców w czasie rzeczywistym.

DriversTable - tabela wszystkich kierowców w czasie rzeczywistym.

FuelChart - wykres zużycia paliwa na przestrzeni całego wyścigu.

InputsChart - wykres aplikacji gazu i hamulca na przestrzeni pojedynczego okrążenia.

LapTimeChart - wykres czasów okrążeń na przestrzeni całego wyścigu.

RaceLaps - lista wszystkich okrążeń wybranego kierowcy w wybranym wyścigu.

SessionInfo - pasek informacji o aktualnej sesji.

TelemetryChart - wykres prędkości, biegów i aktywacji systemu DRS na przestrzeni pojedynczego okrążenia.

TyreCard - uproszczony schemat danych o oponach.

TyreWearChart - wykres zużycia opon na przestrzeni wyścigu.

Przykładowy *TyreCardViewModel* korzystający z *TelemetryProvidera*, znów w taki sam sposób jak usługi korzysta z *dependency injection*:

```
private readonly CompositeDisposable _telemetrySubscription = new();

public TyreCardViewModel(TelemetryProvider telemetryProvider)
{
    (...)
}
```

Wewnątrz konstruktora uruchamiana jest usługa podpinająca wszystkie *strumienie* do grupy *telemetrySubscription* oraz zainicjowane zostają wszystkie niezbędne zmienne:

```
public TyreCardViewModel(TelemetryProvider telemetryProvider)
{
    HookEvents(telemetryProvider);

    TyresWear = [0, 0, 0, 0];
    TyresInnerTemperature = [0, 0, 0, 0];
    TyresSurfaceTemperature = [0, 0, 0, 0];
    TyresPressure = [0, 0, 0, 0];
    VisualTyreCompound = VisualTyreCompound.SOFT;
    ActualTyreCompound = TyreCompound.C5;
    TyresAgeLaps = 0;
}

private void HookEvents(TelemetryProvider telemetryProvider)
{
    _telemetrySubscription.Add(
        telemetryProvider.CarTelemetryStream
            .ObserveOn(SynchronizationContext.Current)
            .Subscribe(OnCarTelemetryDataReceived));

    _telemetrySubscription.Add(
        telemetryProvider.CarStatusStream
            .ObserveOn(SynchronizationContext.Current)
            .Subscribe(OnCarStatusDataReceived));

    _telemetrySubscription.Add(
        telemetryProvider.CarDamageStream
            .ObserveOn(SynchronizationContext.Current)
            .Subscribe(OnCarDamageDataReceived));
}
```

Poniższe metody wywoływane są w momencie otrzymania pakietów i przypisują wartości do wybranych zmiennych:

```
private void OnCarTelemetryDataReceived(CarTelemetryPacket packet)
{
    var playerId = packet.header.playerCarIndex;
    var data = packet.carTelemetryData[playerId];

    TyresPressure = data.tyresPressure;
    TyresInnerTemperature = data.tyresInnerTemperature;
    TyresSurfaceTemperature = data.tyresSurfaceTemperature;
}

private void OnCarStatusDataReceived(CarStatusPacket packet)
{
    var playerId = packet.header.playerCarIndex;
    var data = packet.carStatusData[playerId];

    TyresAgeLaps = data.tyresAgeLaps;
    VisualTyreCompound = data.visualTyreCompound;
    ActualTyreCompound = data.actualTyreCompound;
}

private void OnCarDamageDataReceived(CarDamagePacket packet)
{
    var playerId = packet.header.playerCarIndex;
    var data = packet.carDamageData[playerId];

    TyresWear = data.tyresWear;
}
```

Dodatkowo stworzone zostały zmienne, które formatują zbierane dane w przyjazdy interfejsowi string lub przetrzymują kolory wykorzystywane później, na przykład tablice *TyresInnerTemperatureString* i *TyresInnerTemperatureColor*:

```
public string[] TyresInnerTemperatureString =>
[
    new string($"{TyresInnerTemperature[0]}°C"),
    new string($"{TyresInnerTemperature[1]}°C"),
    new string($"{TyresInnerTemperature[2]}°C"),
    new string($"{TyresInnerTemperature[3]}°C")
];

public Brush[] TyresInnerTemperatureColor =>
[
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[0]
    ),
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[1]
    ),
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[2]
    ),
    ColorHelper.GetTyreTemperatureBrush(
        ActualTyreCompound, TyresInnerTemperature[3]
    )
];
```

Żeby można było z nich korzystać w *View* i ich zmiana była rejestrowana, należy wywołać na nich funkcję *OnPropertyChanged*:

```
partial void OnTyresInnerTemperatureChanged(byte[] oldValue, byte[]
newValue)
{
    OnPropertyChanged(nameof(TyresInnerTemperatureString));
    OnPropertyChanged(nameof(TyresInnerTemperatureColor));
}
```

Teraz w *TyreCardView* można wykorzystać wszystkie stworzone zmienne, na przykład do wyświetlania wskaźników temperatur opon:

```
<Border Grid.Row="1" Grid.Column="1"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[2]}" />
<Border Grid.Row="1" Grid.Column="4"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[3]}" />
<Border Grid.Row="3" Grid.Column="1"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[0]}" />
<Border Grid.Row="3" Grid.Column="4"
    Style="{StaticResource TyreTemperatureDot}"
    Background="{Binding TyresInnerTemperatureColor[1]}" />
```

Lub wyświetlania konkretnych temperatur:

```
<Border Grid.Row="1" Grid.Column="0"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[2]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
<Border Grid.Row="1" Grid.Column="5"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[3]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
<Border Grid.Row="3" Grid.Column="0"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[0]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
<Border Grid.Row="3" Grid.Column="5"
    Style="{StaticResource TyreInfoBlock}">
    <ui:TextBlock
        Text="{Binding TyresInnerTemperatureString[1]}"
        Style="{StaticResource TyreInfoContentTop}" />
</Border>
```

W tym komponencie wykorzystane zostały także zasoby, takie jak czcionka wczytana z pliku:

```
<FontFamily x:Key="JetBrainsMonoExtraBold">
    pack://application,,,Assets/Fonts/#JetBrains Mono Bold
</FontFamily>
```

Czy style nałożone na dany element:

```
<Style x:Key="TyreTemperatureDot" TargetType="{x:Type Border}">
    <Setter Property="Margin" Value="0 6" />
    <Setter Property="Width" Value="28" />
    <Setter Property="Height" Value="16" />
    <Setter Property="VerticalAlignment" Value="Bottom" />
    <Setter Property="CornerRadius" Value="6" />
</Style>
```

Przykładowy *BatteryChartViewModel* korzystający z biblioteki LiveCharts2 [3] wymaga skonfigurowania obu osi:

```
public Axis[] XAxes =>
[
    new Axis
    {
        Name = "Battery usage",
        MinStep = 1,
        Labeler = value => $"{value:F0}",
        LabelsPaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        },
        NamePaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        }
    }
];
```

```
public Axis[] YAxes =>
[
    new Axis
    {
        Labeler = value => $"{value:F0}%",
        LabelsPaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        },
        NamePaint = new SolidColorPaint
        {
            Color = new SKColor(230, 230, 230),
            SKTypeface = _customTypeface
        },
        SeparatorsPaint = new SolidColorPaint
        {
            Color = new SKColor(48, 48, 48)
        }
    }
];
```

W jego konstruktorze poza podpięciem *strumieni* i zainicjowaniem wykresów, znajduje się także przypisanie czcionki z użyciem klasy pomocniczej:

```
public BatteryChartViewModel(TelemetryProvider telemetryProvider)
{
    _customTypeface = FontHelper.LoadCustomFont();

    Ers = new ObservableCollection<ObservablePoint>();

    Series = new ISeries[]
    {
        new LineSeries<ObservablePoint>
        {
            Values = Ers,
            Name = "Ers",
            GeometrySize = 6,
            Stroke = new SolidColorBrush(new SKColor(31, 119, 180), 2),
            GeometryStroke = new SolidColorBrush(
                new SKColor(31, 119, 180), 2
            ),
            Fill = null,
            YToolTipLabelFormatter = point =>
                $"Lap {point.Coordinate.SecondaryValue:F0}:" +
                $"{Math.Round(point.Coordinate.PrimaryValue, 2)}%"
        }
    };

    HookEvents(telemetryProvider);
}
```

Wykorzystanie tego *ViewModelu* w *BatteryChartView* wygląda następująco:

```
<lvc:CartesianChart
    Series="{Binding Series}"
    XAxes="{Binding XAxes}"
    YAxes="{Binding YAxes}" />
```

3 Instrukcja obsługi aplikacji desktopowej

Aby zapewnić poprawne działanie aplikacji należy:

- za pomocą skryptu *docker-compose.yaml* uruchomić bazę danych w konternerze,
- skonfigurować ustawienia telemetry w grze *F1 23* (ustawić adres IP inżyniera i wybrać port - domyślnie 20777),
- uruchomić aplikację.

Film demonstracyjny dostępny jest na YouTube[15].

3.1 Pasek statusu sesji

Pasek statusu sesji znajduje się na samej górze każdego ekranu i zawiera następujące informacje:

- [**<rodzaj sesji>** @ **<nazwa toru>**]
- [**laps:** **<aktualne okrążenie>**/**<ilość okrążeń>**]
- [**weather:** **<pogoda>**]
- [**air:** **<temperatura powietrza>** | **track:** **<temperatura toru>**]

W przypadku wydarzenia na torze, temperatury zmieniają się na informacje o flagach lub samochodzie bezpieczeństwa, na przykład:

[FORMATION_LAP]

[FULL]

3.2 Tabela kierowców

Tabela kierowców zawiera najważniejsze informacje o kierowcach otrzymywane w czasie rzeczywistym:

- pozycja kierowcy
- nazwisko kierowcy w kolorze jego zespołu
- status systemu DRS sygnalizowany zmianą koloru
- różnica w czasie do kierowcy z przodu
- różnica w czasie do lidera wyścigu
- wybrana opona/status pit stopu
- zużycie opon
- wiek opon
- ilość pit stopów
- czas pierwszego sektora
- czas drugiego sektora
- czas ostatniego okrążenia

3.3 Informacje o samochodzie

Ta zakładka zawiera najważniejsze informacje o samochodzie otrzymywane w czasie rzeczywistym:

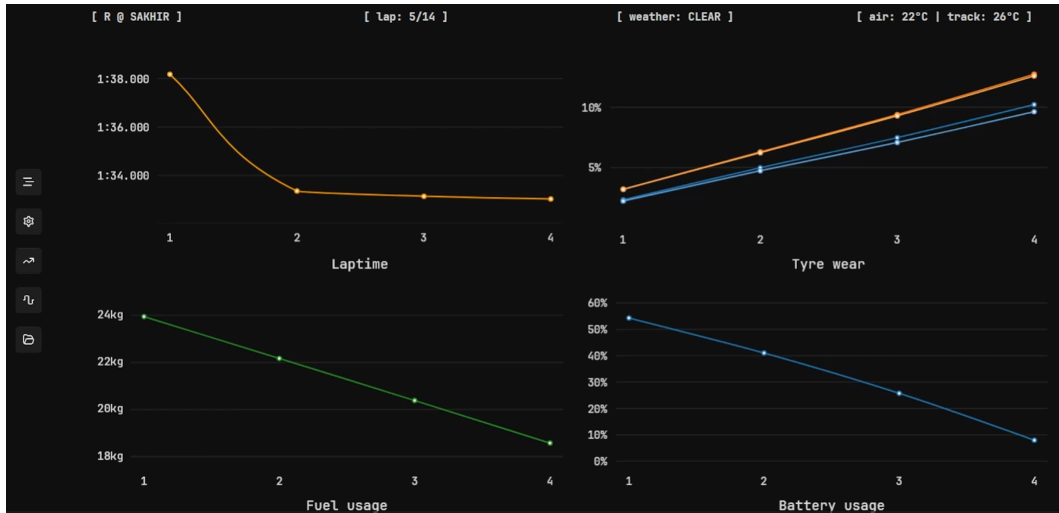
- aplikacja gazu,
- aplikacja hamulca,
- aktualna prędkość,
- stan naładowania baterii,
- status systemu DRS,
- status pit stopu,
- flaga wyświetlana kierowcy,
- tryb wykorzystania baterii,
- mieszanki opon dostępne w pitstopie (ich zużycie i przewidywana żywotność o nich po najechaniu kursorem),
- schematyczna wizualizacja zużycia i temperatur opon,
- schematyczna wizualizacja uszkodzeń pojazdu.



3.4 Wykresy

Pierwszy ekran wykresów zawiera:

- wykres czasów okrążeń,
- wykres zużycia każdej z opon,
- wykres zużycia paliwa,
- wykres zużycia baterii.



Dodatkowo, każdy z wykresów posiada dodatkowe informacje pojawiające się po najechaniu kursorem na punkt na wykresie.

Drugi ekran zawiera:

- wykres aplikacji gazu i hamulca,
- wykres prędkości, biegów i aktywacji systemu DRS.



Tak samo, każdy z wykresów posiada dodatkowe informacje po najechaniu kursorem. W tle obu nowych wykresów cały czas widnieje zapis z najszybszego okrążenia wyścigu, w ciemniejszym odcieniu.

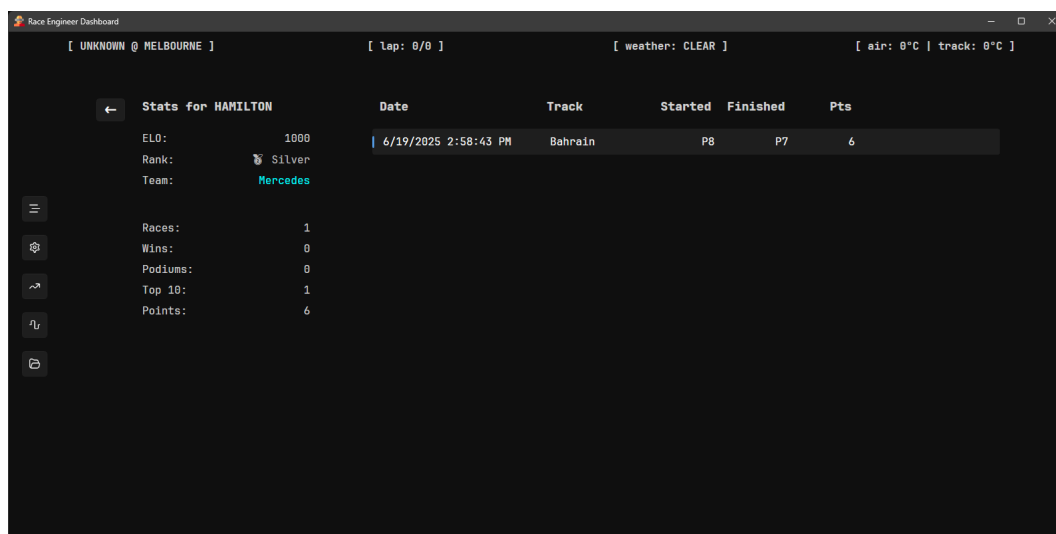
3.5 Ranking kierowców

Ostatni ekran zawiera ranking kierowców, których ELO obliczane jest w bazie danych na podstawie ich wyników w wyścigach, a ranga przypisywana zgodnie z osiągniętym poziomem ELO.

The figure shows a screenshot of the Race Engineer Dashboard displaying a driver ranking table. The table lists drivers with their IDs, names, teams, races, ELO scores, and ranks. The background shows race status: [UNKNOWN @ MELBOURNE], [Lap: 0/0], [weather: CLEAR], [air: 0°C | track: 0°C].

ID	Name	Team	Races	Elo	Rank
9	VERSTAPPEN	Red Bull	1	1007	Silver
50	RUSSELL	Mercedes	1	1004	Silver
59	GASLY	Alpine	1	1004	Silver
0	SAINZ	Ferrari	1	1003	Silver
54	NORRIS	McLaren	1	1003	Silver
7	HAMILTON	Mercedes	1	1000	Silver
2	RICCIARDO	AlphaTauri	1	999	Silver
10	HULKENBERG	Haas	1	999	Silver
11	MAGNUSSEN	Haas	1	999	Silver
58	LECLERC	Ferrari	1	999	Silver
62	ALBON	Williams	1	999	Silver
80	ZHOU	Alfa Romeo	1	999	Silver
14	PÉREZ	Red Bull	1	998	Silver
15	BOTTAS	Alfa Romeo	1	998	Silver
94	TSUNODA	AlphaTauri	1	998	Silver
132	SARGEANT	Williams	1	997	Silver
19	STROLL	Aston Martin	1	996	Silver
3	ALONSO	Aston Martin	1	995	Silver

Po dwukrotnym kliknięciu w kierowcę, wyświetla się ekran z jego osiągnięciami oraz historią wszystkich wyścigów i krótkim podsumowaniem.

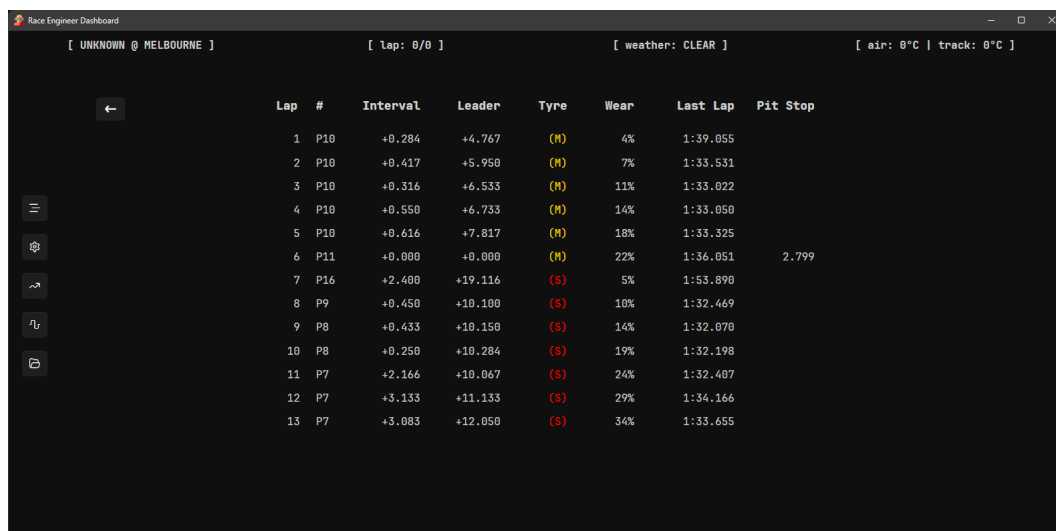


The screenshot shows the 'Stats for HAMILTON' screen. At the top, it displays '[UNKNOWN @ MELBOURNE]', '[Lap: 0/0]', '[weather: CLEAR]', and '[air: 0°C | track: 0°C]'. On the left, there are navigation icons. The main content area shows driver statistics: ELO: 1800, Rank: Silver, Team: Mercedes. Below this, a table lists career stats: Races: 1, Wins: 0, Podiums: 0, Top 10: 1, Points: 6. To the right, a table shows race history with columns: Date, Track, Started, Finished, and Pts. The first entry is for 6/19/2025 2:58:43 PM at Bahrain, with positions P8 and P7, and 6 points.

Stats for HAMILTON		Date	Track	Started	Finished	Pts
ELO:	1800	6/19/2025 2:58:43 PM	Bahrain	P8	P7	6
Rank:	Silver					
Team:	Mercedes					

Category	Value
Races:	1
Wins:	0
Podiums:	0
Top 10:	1
Points:	6

Po dwukrotnym kliknięciu w wyścig, wyświetla się ekran z okrążeniami kierowcy w danym wyścigu. Znajdują się tu informacje o pozycji na każdym okrążeniu, odległościach do samochodów z przodu, oponach, czasach okrążeń i czasach pit stopów.



The screenshot shows the lap data screen. At the top, it displays '[UNKNOWN @ MELBOURNE]', '[Lap: 0/0]', '[weather: CLEAR]', and '[air: 0°C | track: 0°C]'. On the left, there are navigation icons. The main content area is a table with columns: Lap #, Interval, Leader, Tyre, Wear, Last Lap, and Pit Stop. The table lists 13 laps. Laps 1-5 are P10, lap 6 is P11, lap 7 is P16, and laps 8-13 are P9, P8, P8, P7, P7, P7 respectively. The leader column shows increasing time gaps from +4.767 to +12.050. Tyre status is (M) for laps 1-5 and (S) for laps 6-13. Wear increases from 4% to 34%. A pit stop of 2.799 is recorded at the end of lap 6.

Lap #	Interval	Leader	Tyre	Wear	Last Lap	Pit Stop
1 P10	+0.284	+4.767	(M)	4%	1:39.055	
2 P10	+0.417	+5.950	(M)	7%	1:33.531	
3 P10	+0.316	+6.533	(M)	11%	1:33.022	
4 P10	+0.550	+6.733	(M)	14%	1:33.050	
5 P10	+0.616	+7.817	(M)	18%	1:33.325	
6 P11	+0.000	+0.000	(M)	22%	1:36.051	2.799
7 P16	+2.400	+19.116	(S)	5%	1:53.890	
8 P9	+0.450	+10.100	(S)	10%	1:32.469	
9 P8	+0.433	+10.150	(S)	14%	1:32.070	
10 P8	+0.250	+10.284	(S)	19%	1:32.198	
11 P7	+2.166	+10.067	(S)	24%	1:32.407	
12 P7	+3.133	+11.133	(S)	29%	1:34.166	
13 P7	+3.083	+12.050	(S)	34%	1:33.655	

4 Podsumowanie aplikacji desktopowej

4.1 Potencjał rozwoju

Przyszły rozwój aplikacji skupiłby się głównie na:

- obsłudze innych sesji niż wyścig (treningi, kwalifikacje, sprinty),
- usprawnieniach UI/UX,
- integracji modelu przewidywania pozycji na mecie/wyznaczania najlepszej strategii wewnątrz aplikacji,
- implementacji mapy toru,
- obsłudze wielu okien, w celu wyświetlania różnych paneli w tym samym czasie,
- rozwinięciu informacji o wyścigach i okrążeniach o dane z telemetry (linia wyścigowa, prędkość, porównanie okrążeń na mapie toru).

4.2 Wnioski

Udało się stworzyć aplikację desktopową, która spełnia wszystkie warunki postawione w fazie projektowej, umożliwiając efektywną analizę danych telemetrycznych w czasie rzeczywistym z gry *F1 23*. Projekt pozwolił nam wykorzystać potencjał WPF i architektury MVVM do obsługi dynamicznych danych.

5 Implementacja bazy danych

5.1 Struktura projektu

Implementacja została podzielona na trzy projekty znajdujące się w jednym rozwiązaniu:

```
.  
├─ EngineerDashboard.App  
├─ EngineerDashboard.Database  
├─ EngineerDashboard.Telemetry  
└─ EngineerDashboard.sln
```

EngineerDashboard.App zawiera pełną implementację aplikacji desktopowej.

EngineerDashboard.Database zawiera pliki konfiguracyjne bazy danych oraz implementację modeli, usług i kontekstu bazodanowych.

EngineerDashboard.Telemetry zawiera implementację klienta telemetrii.

5.1.1 EngineerDashboard.Database

```
EngineerDashboard.Database  
├─ AppDbContext.cs  
├─ Models  
├─ Services  
├─ docker-compose.yml  
└─ init
```

AppDbContext.cs - plik tworzący kontekst bazy danych.

Models - folder zawierający klasy modeli bazodanowych.

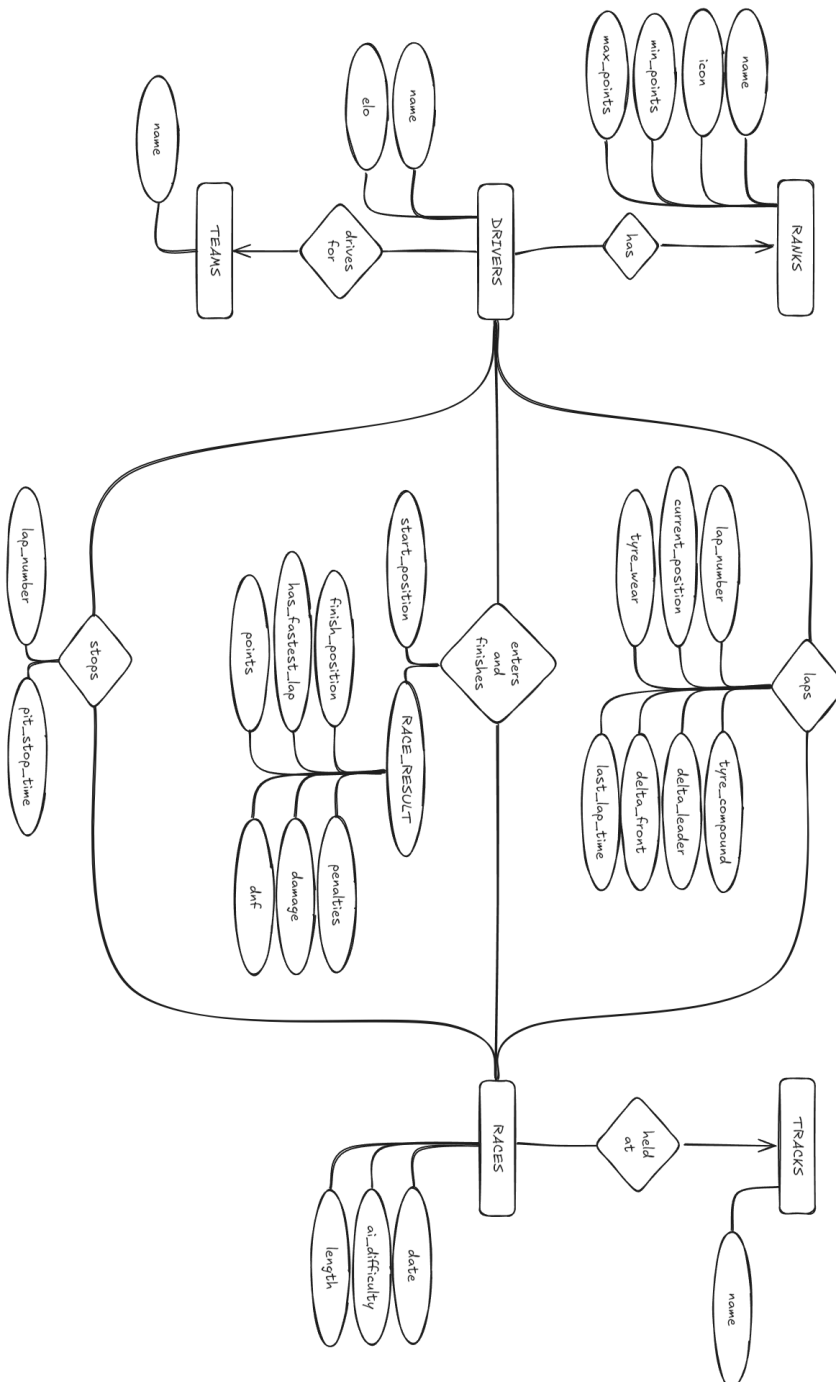
Services - folder zawierający usługi realizacji zapytań.

init - folder zawierający pliki konfiguracyjne bazy danych.

docker-compose.yml - plik tworzący kontener z wykorzystaniem plików konfiguracyjnych.

5.2 Schemat związków encji

Podstawą projektu bazy danych jest schemat związków encji, który jest logiczną reprezentacją wszystkich encji, ich atrybutów i związków:



Rysunek 3: Schemat związków encji bazy danych

Dwoma głównymi encjami w bazie są *Drivers* i *Races*, które reprezentują kierowców i wyścigi.

Encja *Divers* zawiera:

- id kierowcy [PK]
- nazwisko kierowcy,
- jego wynik ELO.

Encja *Drivers* wiąże się z encją *Teams*, aby kierowca mógł należeć do zespołu wyścigowego, oraz z encją *Ranks*, aby wynik ELO kierowcy był łatwiejszy w interpretacji.

Encja *Teams* zawiera:

- id zespołu [PK]
- nazwę zespołu.

Encja *Ranks* zawiera:

- id rangi [PK]
- nazwę rangi,
- ikonę rangi,
- minimalną ilość punktów przynależności do rangi,
- maksymalną ilość punktów przynależności do rangi.

Encja *Races* zawiera:

- id wyścigu [PK]
- datę odbycia wyścigu,
- poziom trudności przeciwników AI,
- długość wyścigu w okrążeniach.

Encja *Races* wiąże się z encją *Tracks*, ze względu na możliwość zmiany torów w kalendarzu.

Encja *Tracks* zawiera:

- id toru [PK]
- nazwę toru.

Główną część schematu stanowią trzy związki z własnymi atrybutami między encjami *Drivers* i *Races*:

- związek *laps* zawiera dane o okrążeniu dla pary kierowca-wyścig:
 - numer okrążenia,
 - aktualna pozycja kierowcy,
 - aktualne zużycie opon,
 - aktualna mieszanka opon,
 - różnica w czasie do lidera wyścigu,
 - różnica w czasie do kierowcy z przodu,
 - czas ostatniego okrążenia,
- związek *pit stops* zawiera dane o pit stopie dla pary kierowca-wyścig:
 - numer okrążenia,
 - czas pit stopu,
- związek *race entries* zawiera dane o wejściu i wyniku dla pary kierowca-wyścig:
 - pozycja na starcie,
 - wieloatrybutowy *race result*:
 - pozycja na mecie,
 - flaga najszybszego okrążenia w wyścigu,
 - ilość zdobytych punktów,
 - całkowity czas kar nałożonych na kierowcę,
 - znormalizowane uszkodzenia,
 - flaga DNF (did not finish).

Każdy kierowca może przynależeć do jednego zespołu, ale każdy zespół może mieć wielu kierowców [kierowca - zespół n:1].

Każdy kierowca może mieć jedną rangę, ale wielu kierowców może mieć tą samą rangę [kierowca - ranga n:1].

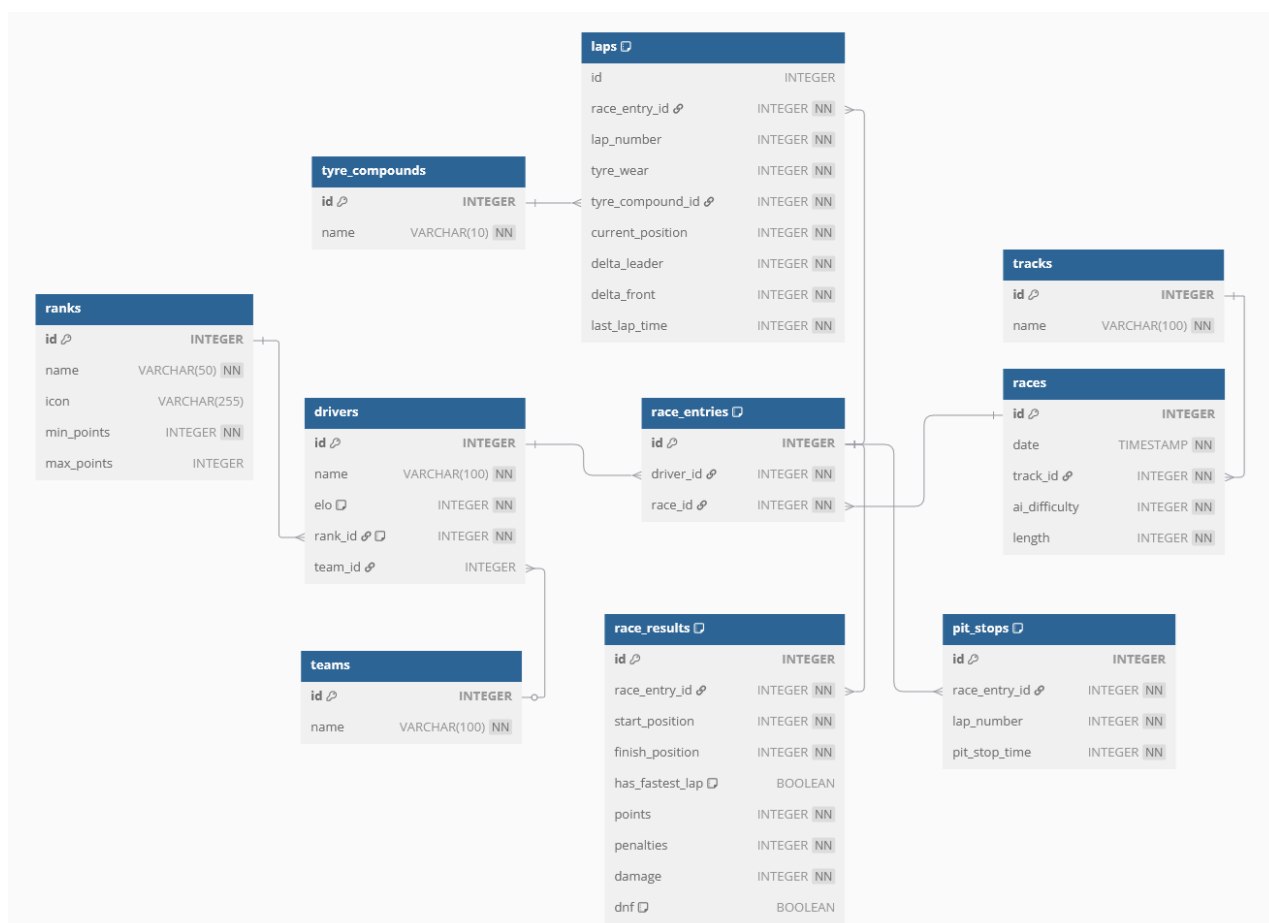
Każdy wyścig może odbywać się na jednym torze, ale wiele wyścigów może odbyć się w tym samym miejscu [wyścig - tor n:1].

Każdy kierowca może mieć wiele wyścigów na swoim koncie, a w każdym wyścigu bierze udział wielu kierowców [kierowca - wyścig n:m].

5.3 Normalizacja

Schemat związków encji jest wystarczający do logicznej reprezentacji danych, ale nie jest wystarczający do wykonania schematu relacyjnego i implementacji bazy w takim stanie. Każda baza musi spełniać przede wszystkim zasady normalizacji, które zapewnia 3 postaci normalne *NF* i postać normalną Boyce'a-Codda *BCNF*.

Poniżej znajduje się schemat relacyjny przygotowany po znormalizowaniu całej bazy danych. Głównymi różnicami względem schematu związków encji jest stworzenie tabeli *race_entries* łączącej *driver_id* i *race_id* pod wspólnym kluczem oraz wykorzystanie jej do uproszczenia relacji *laps*, *race_results* oraz *pit_stops* dla par kierowca-wyścig:



Rysunek 4: Schemat relacyjny bazy danych

5.3.1 Pierwsza postać normalna

Wymagania *1NF*:

- wszystkie wartości kolumn muszą być niepodzielne,
- każdy wiersz musi być unikalny,
- baza nie posiada powtarzających się grup kolumn.

Analiza:

- wszystkie kolumny w tabelach zawierają pojedyncze, niepodzielne wartości,
- każda tabela ma klucz główny,
- w tabelach nie ma kolumn, które się powtarzają.

Wniosek - pierwsza postać normalna *1NF* została spełniona.

5.3.2 Druga postać normalna

Wymagania *2NF*:

- musi spełniać *1NF*,
- wszystkie atrybuty niekluczowe muszą w pełni zależeć od całego klucza głównego.

Analiza:

- tabele z prostymi kluczami głównymi (*teams*, *ranks*, *drivers*, *tracks*, *races*, *race_results*, *tyre_compounds*, *pit_stops*) mają wszystkie atrybuty niekluczowe (np. *name*, *elo*, *date*) w pełni zależne od ich kluczy głównych,
- *race_entries* ma wygenerowany id jako klucz główny i unikalne ograniczenie na (*driver_id*, *race_id*). Ponieważ jedynymi innymi kolumnami są klucze obce (*driver_id*, *race_id*), nie ma atrybutów niekluczowych, które mogłyby wprowadzić częściowe zależności,
- *laps* ma wygenerowany id jako klucz główny i unikalne ograniczenie na (*race_entry_id*, *lap_number*). Wszystkie atrybuty (np. *tyre_wear*, *last_lap_time*) zależą od całego id, a złożone unikalne ograniczenie wspiera tę strukturę zależności.

Wniosek - druga postać normalna *2NF* została spełniona.

5.3.3 Trzecia postać normalna

Wymagania $3NF$:

- musi spełniać $2NF$
- atrybuty niekluczowe nie mogą zależeć od innych atrybutów niekluczowych; powinny zależeć tylko od klucza głównego.

Analiza:

- *teams*: *name* zależy od *id*,
- *ranks*: *name*, *icon*, *min_points*, *max_points* zależą od *id*,
- *drivers*: *name*, *elo*, *rank_id*, *team_id* zależą od *id*. Należy zauważyć, że *elo* wpływa na *rank_id* za pomocą wyzwalacza *update_driver_rank*, ale ponieważ *elo* jest bezpośrednim atrybutem tabeli, a nie innym atrybutem niekluczowym zależnym od trzeciego atrybutu, nie jest to zależność przechodnia,
- *tracks*: *name* zależy od *id*,
- *races*: *date*, *track_id*, *ai_difficulty*, *length* zależą od *id*,
- *race_entries*: Tylko klucze obce (*driver_id*, *race_id*), brak atrybutów niekluczowych,
- *race_results*: *start_position*, *finish_position*, *points* itd. zależą od *id*,
- *tyre_compounds*: *name* zależy od *id*,
- *laps*: *lap_number*, *tyre_wear* itd. zależą od *id*,
- *pit_stops*: *lap_number*, *pit_stop_time* zależą od *id*.

Wniosek - trzecia postać normalna $3NF$ została spełniona.

5.3.4 Postać normalna Boyce’a-Codda

Wymagania *BCNF*:

- musi spełniać *3NF*,
- dla każdej zależności funkcyjnej ($X \rightarrow Y$), X musi być kluczem kandydującym.

Analiza:

- większość tabel ma jednokolumnowy klucz główny (*id*), a wszystkie atrybuty od niego zależą, co czyni *id* jedynym kluczem kandydującym,
- *race_entries*: Klucz główny to *id*, ale (*driver_id*, *race_id*) jest kluczem kandydującym ze względu na unikalne ograniczenie. Ponieważ nie ma atrybutów niekluczowych poza kluczami obcymi, wszystkie zależności wskazują na *id* lub klucz kandydujący,
- *laps*: Klucz główny to *id*, z unikalnym ograniczeniem na (*race_entry_id*, *lap_number*). Wszystkie atrybuty zależą od *id*, a (*race_entry_id*, *lap_number*) jest kluczem kandydującym,
- *pit_stops*: Podobnie jak *laps*, z *id* jako kluczem głównym i (*race_entry_id*, *lap_number*) jako kluczem kandydującym.

Wniosek - postać normalna Boyce’a-Codda *BCNF* została spełniona.

5.3.5 Pozostałe postaci normalne

Dodatkowo, istnieją dwie kolejne postaci normalne *4NF* i *5NF*:

Czwarta postać normalna zapewnia brak nietrywialnych zależności wielowartościowych, chyba że są to zależności funkcyjne. Z racji struktury bazy, którą przygotowaliśmy, wystąpienie takiej zależności jest niemożliwe. Przykładem byłoby np. umieszczenie pod parą (*driver_id*, *race_id*) kolumn *tyre_compound_id* i *start_position*. Wtedy w takiej tabeli znajdą się dwie zależności:

- (*driver_id*, *race_id*) $\rightarrow$ *tyre_compound_id*,
- (*driver_id*, *race_id*) $\rightarrow$ *start_position*.

Piąta postać normalna zapewnia brak zależności złączeniowych, czyli żadna tabela nie może być rozłożona na mniejsze tabele bez utraty informacji. Schemat jest odpowiednio podzielony na encje i relacje, a każda z nich zawiera minimalną istotną ilość atrybutów, przy zachowaniu pełni kontekstu. W naszym przypadku spełnienie czwartej postaci normalnej jest jednoznaczne ze spełnieniem piątej postaci normalnej.

5.4 EngineerDashboard.Database

Pełny kod znajduje się w repozytorium projektu na GitHubie[12].

Baza danych tworzona jest lokalnie w kontenerze z wykorzystaniem skryptu *docker-compose.yml*:

```
version: '3.8'

services:
  postgres:
    image: postgres:15
    container_name: telemetry-postgres
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d
    ports:
      - "5432:5432"
    restart: always
    healthcheck:
      test: [ "CMD-SHELL", "pg_isready -U admin -d telemetry" ]
      interval: 10s
      retries: 5
      start_period: 10s
      timeout: 5s

  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "8080:80"
    depends_on:
      - postgres
    restart: always

volumes:
  postgres_data:
```

W folderze *init* projektu znajdują się 4 pliki:

```
EngineerDashboard.Database/init
├─ functions.sql
├─ init.sql
├─ procedures.sql
└─ views.sql
```

init.sql zawiera cały schemat bazy danych, wraz z użytkownikami.

functions.sql, *procedures.sql* i *views.sql* zawierają przykładowe funkcje, procedury i widoki, które można wykonać na bazie.

Po stworzeniu kontenera w bazie istnieją jedynie poniższe dane wprowadzone ręcznie:

- rangi

```
INSERT INTO ranks (name, icon, min_points, max_points) VALUES
  ('Bronze', '🥉', 0, 899),
  ('Silver', '🥈', 900, 1099),
  ('Gold', '🥇', 1100, 1399),
  ('Platinum', '💎', 1400, 1799),
  ('Master', '🏆', 1800, 2499),
  ('Champion', '🏆', 2500, NULL);
```

- tory

```
INSERT INTO tracks (id, name) VALUES
  (0, 'Melbourne'),
  (1, 'Paul Ricard'),
  (2, 'Shanghai'),
  (...),
  (32, 'Losail');
```

- zespoły

```
INSERT INTO teams (id, name) VALUES
  (0, 'Mercedes'),
  (1, 'Ferrari'),
  (2, 'Red Bull'),
  (...),
  (9, 'Alfa Romeo');
```

- mieszanki opon

```
INSERT INTO tyre_compounds(id, name) VALUES
  (16, 'SOFT'),
  (17, 'MEDIUM'),
  (18, 'HARD'),
  (7, 'INTER'),
  (8, 'WET');
```


Baza danych posiada 3 użytkowników:

- *fia* - organ zarządzający serią wyścigową, może zmieniać zestaw torów, zespoły, kierowców oraz rangi,
- *stewards* - sędziowie, którzy mogą jedynie nakładać kary i zmieniać pozycje końcowe kierowców w wyścigu,
- *telemetry_client* - użytkownik, z którego korzysta aplikacja, aby automatycznie wprowadzać dane do bazy.

W bazie istnieją dwa podstawowe triggery *trg_update_driver_elo* i *trg_update_rank_after_elo_update*, które mają za zadanie zaktualizować wynik ELO kierowcy po przyjęciu wyników wyścigu, a zaraz po tym, w razie potrzeby zaktualizować jego rangę.

Aktualny wzór obliczenia klasyfikacji ELO wygląda następująco:

$$E_n = \max(E_c + \lfloor K(A_s - E_s) \rfloor, 0)$$

gdzie:

- $E_s = \min\left(\max\left(\frac{0.5}{1 + \frac{D_m}{1000}}, 0.1\right), 0.9\right)$, with $D_m = \frac{A_d + L}{100}$.
- $A_s = \min(\max(0.4N_{pg} + 0.3N_p + 0.2N_d + 0.1N_{dnf} - N_{pen}, 0), 1)$.
- $N_{pg} = \min\left(\max\left(\frac{P_s - P_f}{20}, -1\right), 1\right) D_m$.
- $N_p = \frac{P + F_l}{25} D_m$, with $F_l = 1$ if fastest lap, else 0.
- $N_d = \frac{100 - D}{100} D_m 0.5$.
- $N_{dnf} = -0.5 D_m$ if DNF, else 0.
- $N_{pen} = \min\left(\frac{P_n}{10}, 0.2\right) D_m$.
- $K = 24$, E_c : current ELO, A_d : AI difficulty, L : race length, P_s : start position, P_f : finish position, P : points, D : damage, P_n : penalties.

Wzór został zaimplementowany w funkcji *update_driver_elo()* w pliku *EngineerDashboard.Database/init.sql*, która jest wykonywana za pomocą poniższego triggera:

```
CREATE OR REPLACE TRIGGER trg_update_driver_elo
AFTER INSERT OR UPDATE
OF start_position, finish_position, has_fastest_lap,
points, penalties, damage, dnf
ON race_results
FOR EACH ROW
EXECUTE FUNCTION update_driver_elo();
```

Po aktualizacji ELO uruchamiany jest poniższy trigger:

```
CREATE OR REPLACE TRIGGER trg_update_rank_after_elo_update
  AFTER UPDATE OF elo ON drivers
  FOR EACH ROW
  WHEN (OLD.elo IS DISTINCT FROM NEW.elo)
EXECUTE FUNCTION update_driver_rank();
```

Wykonujący poniższą funkcję:

```
CREATE OR REPLACE FUNCTION update_driver_rank()
  RETURNS TRIGGER AS $$
DECLARE
  new_rank_id INT;
BEGIN
  SELECT id INTO new_rank_id
  FROM ranks
  WHERE min_points <= NEW.elo
        AND (max_points >= NEW.elo OR max_points IS NULL)
  LIMIT 1;

  UPDATE drivers
  SET rank_id = new_rank_id
  WHERE id = NEW.id;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Dla usprawnienia wykonywania niektórych operacji na bazie, powstały dwie procedury, jedna funkcja i kilka widoków.

Procedury *set_penalty* i *change_position* mają za zadanie umożliwić sędziemu *steward* modyfikować wynik kierowcy znając *race_result_id*:

```
CREATE OR REPLACE PROCEDURE set_penalty(race_result_id INT,  
new_penalty INT)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF EXISTS (SELECT 1 FROM race_results WHERE id = race_result_id)  
THEN  
    UPDATE race_results  
    SET penalties = new_penalty  
    WHERE id = race_result_id;  
ELSE  
    RAISE NOTICE 'Nonexistent ID --> %', race_result_id;  
END IF;  
END;  
$;
```

```
CREATE OR REPLACE PROCEDURE change_position(race_result_id INT,  
new_position INT)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF EXISTS (SELECT 1 FROM race_results WHERE id = race_result_id)  
THEN  
    UPDATE race_results  
    SET finish_position = new_position  
    WHERE id = race_result_id;  
ELSE  
    RAISE NOTICE 'Nonexistent ID --> %', race_result_id;  
END IF;  
END;  
$;
```

Funkcja *get_driver_races* zwraca zestaw danych o wyścigach i ich wynikach danego kierowcy:

```
CREATE OR REPLACE FUNCTION get_driver_races(driver INT)
  RETURNS TABLE (
    race_date TIMESTAMP,
    track_name VARCHAR(100),
    finish_position INT,
    has_fastest_lap BOOLEAN,
    dnf BOOLEAN,
    penalties INT,
    points INT
  )
  LANGUAGE plpgsql
AS $$
BEGIN
  RETURN QUERY
  SELECT
    races.date,
    tracks.name,
    race_results.finish_position,
    race_results.has_fastest_lap,
    race_results.dnf,
    race_results.penalties,
    race_results.points
  FROM race_results
  JOIN race_entries
    ON race_entries.id = race_results.race_entry_id
  JOIN races
    ON races.id = race_entries.race_id
  JOIN tracks
    ON tracks.id = races.track_id
  WHERE race_entries.driver_id = driver;
END;
$$;
```

Przykładowe widoki obejmują:

- tabelę rankingu kierowców

```
CREATE VIEW drivers_table AS
SELECT drivers.id, drivers.name, teams.name, drivers.elo, ranks.name
FROM drivers
JOIN teams ON teams.id = drivers.team_id
JOIN ranks ON ranks.id = drivers.rank_id;
```

- wyścigi, które odbyły się w tym roku

```
CREATE VIEW races_this_year AS
SELECT races.id, races.date, tracks.name, races.ai_difficulty
FROM races
JOIN tracks ON tracks.id = races.track_id
WHERE EXTRACT(YEAR FROM races.date) = EXTRACT(YEAR FROM
CURRENT_DATE);
```

- kierowców, którzy należą do konkretnej rangi, np. brązowej, złotej i mistrzowskiej:

```
CREATE VIEW drivers_bronze AS
SELECT drivers.id, drivers.name, drivers.elo
FROM drivers
JOIN ranks ON ranks.id = drivers.rank_id
WHERE ranks.name = 'Bronze';
```

```
CREATE VIEW drivers_gold AS
SELECT drivers.id, drivers.name, drivers.elo
FROM drivers
JOIN ranks ON ranks.id = drivers.rank_id
WHERE ranks.name = 'Gold';
```

```
CREATE VIEW drivers_champion AS
SELECT drivers.id, drivers.name, drivers.elo
FROM drivers
JOIN ranks ON ranks.id = drivers.rank_id
WHERE ranks.name = 'Champion';
```

W folderze *Models* znajdują się klasy modeli dla każdej istniejącej tabeli:

EngineerDashboard.Database/Models

- |— Driver.cs
- |— Lap.cs
- |— PitStop.cs
- |— Race.cs
- |— RaceEntry.cs
- |— RaceResult.cs
- |— Rank.cs
- |— Team.cs
- |— Track.cs
- |— TyreCompound.cs

W folderze *Services* znajdują się usługi zapytań na konkretnych tabelach:

EngineerDashboard.Database/Services

- |— DriverService.cs
- |— LapService.cs
- |— PitStopService.cs
- |— RaceEntryService.cs
- |— RaceResultService.cs
- |— RaceService.cs

Przykładowa implementacja modelu *RaceEntry*, która wyznacza kolumny i relacje z innymi tabelami:

```
namespace EngineerDashboard.Database.Models;

public class RaceEntry
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int id { get; set; }

    [Column("driver_id")]
    public int driver_id { get; set; }

    [Column("race_id")]
    public int race_id { get; set; }

    [ForeignKey("driver_id")]
    public virtual Driver driver { get; set; } = null!;

    [ForeignKey("race_id")]
    public virtual Race race { get; set; } = null!;

    public virtual ICollection<Lap> laps { get; set; } =
        new List<Lap>();
    public virtual ICollection<PitStop> pit_stops { get; set; } =
        new List<PitStop>();
    public virtual RaceResult? race_result { get; set; }
}
```

Powyższa klasa dokładnie odwzorowuje wszystkie atrybuty tabeli *race_entries* i jej relacje z innymi tabelami, zgodnie z tym, co znajduje się w pliku *init.sql*.

Przykładowa implementacja usługi zapytań na tabeli *race_results*:

```
public class RaceResultService
{
    public static async Task<RaceResult> CreateRaceResultAsync(
        AppDbContext context,
        RaceResult raceResult
    )
    {
        if (!await context
            .RaceEntries
            .AnyAsync(re => re.id == raceResult.race_entry_id))
            throw new InvalidOperationException("Entry not found");

        context.RaceResults.Add(raceResult);
        await context.SaveChangesAsync();

        return raceResult;
    }

    public static async Task CreateRaceResultsAsync(
        AppDbContext context,
        IEnumerable<RaceResult> raceResults
    )
    {
        foreach (var raceResult in raceResults)
        {
            if (!await context
                .RaceEntries
                .AnyAsync(re => re.id == raceResult.race_entry_id))
                throw new InvalidOperationException($"Entry not found");
        }

        context.RaceResults.AddRange(raceResults);

        await context.SaveChangesAsync();
    }
}
```



```

public static async Task<int> GetDriverWinsCount(
    AppDbContext context,
    int driverId
)
{
    if (!await context
        .Drivers
        .AnyAsync(d => d.id == driverId))
        throw new InvalidOperationException("Driver not found");

    var wins = await context.RaceResults
        .Join(context.RaceEntries,
            rr => rr.race_entry_id,
            re => re.id,
            (rr, re) => new {RaceResult = rr, RaceEntry = re})
        .Where(joined =>
            joined.RaceEntry.driver_id == driverId &&
            joined.RaceResult.finish_position == 1)
        .CountAsync();

    return wins;
}

(pozostałe metody:
    GetDriverTopFinishesCount,
    GetDriverPointsCount)
}

```

Metoda *CreateRaceResultAsync* tworzy nowe *Race Result* w bazie danych i zwraca je jako obiekt. Metoda *CreateRaceResultsAsync* tworzy ich wiele jednocześnie.

Metoda *GetDriverWinsCount* korzysta z polecenia *JOIN* na tabelach *race_entries* i *race_results* oraz zwraca ilość wygranych danego kierowcy.

Metoda *GetDriverTopFinishesCount* zwraca ilość wyścigów ukończonych w top *n* pozycji.

Metoda *GetDriverPointsCount* zwraca ilość punktów zdobytych przez kierowcę w całej jego historii.

Alternatywnie, dzięki wykorzystaniu *Entity Framework Core*, możemy pominąć ręczne tworzenie *JOIN* na tabelach za pomocą metody *Include*:

```
public class DriverService
{
    (pozostałe metody...)

    public static async Task<Collection<Driver>> GetDrivers(
        ApplicationDbContext context
    )
    {
        var drivers = await context.Drivers
            .Include(d => d.rank)
            .Include(d => d.team)
            .Include(d => d.race_entries)
            .AsNoTracking()
            .ToListAsync();

        return new Collection<Driver>(drivers);
    }
}
```

Metoda *GetDrivers* w usłudze zapytań na tabeli *drivers* zwraca listę wszystkich kierowców wraz z ich rangami, zespołami do których należą i startami w wyścigach, zachowując atrybuty dołączonych tabel.

Tworzenie zapytań bez użycia czystego SQLa umożliwia implementacja kontekstu bazy danych *AppDbContext*, która zawiera wszystkie mapowania każdej tabeli, wszystkich ich atrybutów i relacji:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Team>(entity =>
    {
        entity.ToTable("teams");
        entity.HasKey(e => e.id);
        entity.Property(e => e.id)
            .ValueGeneratedNever();
        entity.Property(e => e.name)
            .IsRequired().HasMaxLength(100);
    });
}
```

(pozostałe tabele...)

```
modelBuilder.Entity<Driver>(entity =>
{
    entity.ToTable("drivers");
    entity.HasKey(e => e.id);
    entity.Property(e => e.id)
        .ValueGeneratedNever();
    entity.Property(e => e.name)
        .IsRequired().HasMaxLength(100);
    entity.Property(e => e.elo)
        .IsRequired().HasDefaultValue(1000);
    entity.Property(e => e.rank_id)
        .IsRequired().HasDefaultValue(1);
    entity.HasOne(e => e.rank)
        .WithMany(r => r.drivers)
        .HasForeignKey(e => e.rank_id)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasOne(e => e.team)
        .WithMany(t => t.drivers)
        .HasForeignKey(e => e.team_id)
        .OnDelete(DeleteBehavior.SetNull);
    entity.HasMany(e => e.race_entries)
        .WithOne(re => re.driver)
        .HasForeignKey(re => re.driver_id)
        .OnDelete(DeleteBehavior.Cascade);
});
```

(pozostałe tabele...)

```
}
```

5.5 EngineerDashboard.App

Punktem startowym aplikacji desktopowej jest plik *App.xaml.cs*, w którym zaimplementowany został wzorzec projektowy *dependency injection*. Na jego początku zarejestrowany zostaje kontekst bazy danych i *singleton* usługi *DatabaseService*:

```
DotNetEnv.Env.TraversePath().Load();

ServiceCollection services = new ServiceCollection();

services.AddSingleton<TelemetryProvider>();
services.AddSingleton<TelemetryLoggerService>();

services.AddDbContextFactory<AppDbContext>(options =>
{
    var connectionString =
        $"Host={Environment.GetEnvironmentVariable("DATABASE_ADDRESS")};" +
        $"Port={Environment.GetEnvironmentVariable("DATABASE_PORT")};" +
        $"Database={Environment.GetEnvironmentVariable("DATABASE_NAME")};" +
        $"Username={Environment.GetEnvironmentVariable("DATABASE_USER")};" +
        $"Password={Environment.GetEnvironmentVariable("DATABASE_PASSWORD")};";

    options.UseNpgsql(connectionString);
});
services.AddSingleton<DatabaseService>();
```

Z uwagi na naturę aplikacji, w której żadne zapytanie nie jest wykonywane równoległe do innego i nie korzysta z tego samego kontekstu, możemy zarejestrować *DatabaseService* jako *singleton*.

Następnie rejestrowane są wszystkie stworzone *ViewModele* oraz *Modele*, z których później korzystają *View*, a na sam koniec główne okno *MainWindow*.

Przy uruchomieniu aplikacji, uruchamiane są także dwie usługi działające w tle - *TelemetryLoggerService* oraz *DatabaseService* i pokazywane jest główne okno aplikacji. Przed zamknięciem aplikacji czyszczony jest *serviceProvider*:

5.5.1 Services

W tym projekcie stworzone zostały cztery usługi:

```
EngineerDashboard.App/Services
├─ DatabaseService.cs
├─ PartialTelemetryRow.cs
├─ TelemetryLoggerService.cs
└─ TelemetryProvider.cs
```

DatabaseService.cs zawierający implementację zapisu danych telemetrycznych do bazy i będący pośrednikiem dla zapytań pochodzących z aplikacji.

PartialTelemetryRow.cs i *TelemetryLoggerService.cs* zawierające implementację zapisu danych telemetrycznych do pliku *.csv*.

TelemetryProvider.cs będący pośrednikiem między *TelemetryClient* a aplikacją.

5.5.1.1 TelemetryProvider

W pierwszej kolejności dostawca telemetry *TelemetryProvider* tworzy instancję klienta telemetry *TelemetryClient*, pobierając port z pliku *.env*:

```
private readonly TelemetryClient _telemetryClient =
    new(int.Parse(
        Environment.GetEnvironmentVariable("TELEMETRY_CLIENT_PORT")
    ));
```

Następnie tworzy *subject* dla każdego pakietu, czyli obiekt który jednocześnie jest obserwatorem i może być obserwowany:

```
private readonly Subject<CarDamagePacket> _carDamageSubject = new();
private readonly Subject<CarSetupPacket> _carSetupSubject = new();
private readonly Subject<CarStatusPacket> _carStatusSubject = new();
(pozostałe subjecty...)
```

Tworzy *strumienie* dla każdego pakietu, które będą powiadamiać ich obserwatorów:

```
public IObservable<CarDamagePacket> CarDamageStream =>
    _carDamageSubject;
public IObservable<CarSetupPacket> CarSetupStream =>
    _carSetupSubject;
public IObservable<CarStatusPacket> CarStatusStream =>
    _carStatusSubject;
(pozostałe pakiety...)
```

Podpina do nich *eventy* pochodzące z *TelemetryClient*:

```
public TelemetryProvider()
{
    _telemetryClient.OnCarDamageDataReceive += packet =>
        _carDamageSubject.OnNext(packet);
    _telemetryClient.OnCarSetupDataReceive += packet =>
        _carSetupSubject.OnNext(packet);
    _telemetryClient.OnCarStatusDataReceive += packet =>
        _carStatusSubject.OnNext(packet);
    (pozostałe pakiety...)
}
```

Dzięki temu, usługa bazy danych może nasłuchiwać na wszystkie pakiety przychodzące na port i wykonywać z nimi własne operacje.

5.5.1.2 DatabaseService

Ta usługa korzysta ze stworzonego wcześniej *TelemetryProvider* i *AppDbContext*, więc dzięki *dependency injection* dostawca telemetrii i kontekst bazy danych mogą zostać „wstrzyknięci” bezpośrednio do konstruktora usługi:

```
private readonly CompositeDisposable _telemetrySubscription = new();
private readonly AppDbContext _context;

(...)

public DatabaseService(
    AppDbContext context,
    TelemetryProvider telemetryProvider
)
{
    (...)
}
```

Wewnątrz konstruktora do zmiennej prywatnej przypisywany jest kontekst bazy danych, a do grupy *telemetrySubscription* dodawane są *strumienie*, których *DatabaseService* będzie nasłuchiwał. Te strumienie dostarczają dane, które są wysyłane w trakcie wyścigu w tle:

```
_context = context;

 telemetrySubscription.Add(
    telemetryProvider.ParticipantsStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(OnParticipantsDataReceived));

 telemetrySubscription.Add(
    telemetryProvider.SessionStream
        .ObserveOn(SynchronizationContext.Current)
        .Subscribe(OnSessionDataReceived));
```

(pozostałe strumienie: *LapDataStream*, *CarDamageStream*, *CarStatusStream*, *EventStream*, *FinalClassificationStream*)

Przykładowa obsługa pakietu *SessionPacket*, który odbierany jest tylko raz, na początku wyścigu:

```
private void OnSessionDataReceived(SessionPacket packet)
{
    if (!_hasRaceInfo) return;

    _race = new Race
    {
        track_id = (int)packet.trackId,
        date = DateTime.UtcNow,
        ai_difficulty = packet.aiDifficulty,
        length = packet.totalLaps,
    };

    _hasRaceInfo = true;
    TrySaveToDatabase();
}
```

Funkcja *TrySaveToDatabase* to jedna z wielu, która korzysta z usług zapytań stworzonych wcześniej w projekcie *Database*:

```
private void TrySaveToDatabase()
{
    if (_hasSavedToDatabase) return;
    if (!_hasDrivers || !_hasRaceInfo || !_hasStartPositions) return;
    _hasSavedToDatabase = true;
    Task.Run(async () =>
    {
        for (int i = 0; i < _numDrivers; i++)
            _drivers[i].id = (await DriverService
                .CreateDriverAsync(_context, _drivers[i])).id;

        _raceId = (await RaceService
            .CreateRaceAsync(_context, _race)).id;

        for (int i = 0; i < _drivers.Count; i++)
        {
            var raceEntry = new RaceEntry
            { driver_id = _drivers[i].id,
              race_id = _raceId.Value };

            _raceEntries.Add(raceEntry);
            _raceEntries[i].id = (await RaceEntryService
                .CreateRaceEntryAsync(_context, raceEntry)).id;
        }
    });
}
```

Ta metoda ma za zadanie zebrać dane o wyścigu, listę kierowców i informacje o ich pozycjach startowych, a następnie zapisać je do bazy danych korzystając z metod *DriverService.CreateDriverAsync()*, *RaceService.CreateRaceAsync()* i *RaceEntryService.CreateRaceEntryAsync()*.

Wykonanie tych zapytań spowoduje stworzenie podstaw do umieszczenia w bazie kolejnych danych (okrążenia, pit stopy, wyniki).

Przykładowa obsługa pakietu *LapDataPacket*, który odbierany jest cały czas, ale zapisywany do bazy co okrążenie:

```
private void OnLapDataReceived(LapDataPacket packet)
{
    if (!_hasStartPositions)
    {
        foreach (var entry in packet.lapData)
            _startPositions.Add(entry.carPosition);

        _hasStartPositions = true;
        TrySaveToDatabase();
    }

    for (int i = 0; i < _numDrivers; i++)
    {
        var lapData = packet.lapData[i];
        var currentLapNum = lapData.currentLapNum;

        if (lapData.pitStopTimerInMS != 0)
            _latestPitStopTime[i] = lapData.pitStopTimerInMS;

        if (currentLapNum < 2)
            continue;

        if (_lastLoggedLapNums.TryGetValue(i, out var lastLoggedLap) &&
            lastLoggedLap == currentLapNum)
            continue;

        var lap = new Lap
        {
            race_entry_id = _raceEntries[i].id,
            lap_number = lapData.currentLapNum - 1,
            tyre_wear = (int)_latestTyreWear[i],
            tyre_compound_id = _latestVisualTyreCompound[i],
            current_position = lapData.carPosition,
            delta_leader = lapData.deltaToRaceLeaderInMS,
            delta_front = lapData.deltaToCarInFrontInMS,
            last_lap_time = (int)lapData.lastLapTimeInMS,
        };

        Task.Run(async () =>
        {
            await LapService.CreateLapAsync(_context, lap);
        });

        _lastLoggedLapNums[i] = currentLapNum;
    }
}
```

Dodatkowo *DatabaseService* jest pośrednikiem między aplikacją a usługami bazy danych (wrapperem), dlatego implementuje własne metody zwracające wyniki zapytań, na przykład:

```
public async Task<Collection<Driver>> GetDrivers()
{
    var drivers = await DriverService
        .GetDrivers(_context);

    return drivers;
}

public async Task<int> GetDriverRacesCount(int driverId)
{
    var wins = await RaceEntryService
        .GetDriverRaceEntriesCount(_context, driverId);

    return wins;
}

public async Task<int> GetDriverWinsCount(int driverId)
{
    var wins = await RaceResultService
        .GetDriverWinsCount(_context, driverId);

    return wins;
}
```

6 Podsumowanie bazy danych

6.1 Potencjał rozwoju

Przyszły rozwój bazy danych skupiłby się głównie na:

- rozwiązaniu problemu kontraktów kierowców i zmiany zespołów (zwykła tabela z momentem rozpoczęcia i zakończenia kontraktu nie zadziałałaby - kierowcy mogą być wypożyczani, zastępowani na pojedyncze sesje, mogą zmieniać zespoły w trakcie trwania kontraktu),
- optymalizacji umieszczania danych w bazie pod kątem ich odczytu (w PostgreSQL partycjonowanie, czyli stworzenie nowej tabeli na wzór na przykład *laps* ale identyfikowalnej względem wyścigu *laps_<id wyścigu>*),
- stworzenie funkcjonalności sezonów, obejmującej klasyfikację kierowców, konstruktorów i wyniki mistrzostw,
- zcentralizowanie bazy w postaci publicznego serwera, aby móc obsługiwać wielu użytkowników aplikacji jednocześnie, tworząc ranking na przykład dla lig esportowych,
- stworzenie aplikacji dla sędziów *stewards* i organu zarządzającego *fia*, aby posiadali interfejs do wykonywania swojej pracy,
- usprawnienie obliczania wyniku ELO,
- zapis bardziej szczegółowych danych do bazy, umożliwiających poprawę obliczania wyników ELO (uszkodzenia konkretnych elementów bolidu, zużycie komponentów), rekonstrukcji całych wyścigów i okrążeń na wykresach (zużycie paliwa, baterii, aplikacja gazu, hamulca, prędkość, bieg).

6.2 Wnioski

Udało się stworzyć bazę danych dla aplikacji desktopowej, która spełnia wszystkie warunki postawione w fazie projektowej, umożliwiając stworzenie rankingu kierowców, dzięki danym telemetrycznym pobieranym w czasie rzeczywistym z gry *F1 23*. Projekt nie tylko pozwolił nam wykorzystać potencjał PostgreSQL do obsługi dynamicznych danych, ale nauczył nas wiele w przestrzeni integracji systemów, odpowiedniej architektury i procesu tworzenia baz danych.

7 Implementacja modelu sztucznej inteligencji

7.1 Zbieranie danych

Dane mają postać wycinka z telemetry pojazdu gracza. Każdy plik csv to pojedynczy wyścig z danymi zbieranymi co 1 sekundę.

Dane zostały zebrane poprzez faktyczny udział w wyścigach w grze komputerowej *F1 23*. Pełny zestaw na czas realizacji projektu stanowi 25 wyścigów (25 plików *.csv*), które zostały zarejestrowane klientem telemetry stworzonym jako element aplikacji desktopowej w ramach przedmiotu *Programowanie Obiektowe i Graficzne*.

Dane składają się z identyfikatora czasu sesji *timestamp* i 40 kolumn wartości.

Wśród nich znajdują się między innymi informacje o stanie opon (ich rodzaj, zużycie, temperatury zewnętrzne i wewnętrzne, ciśnienia), czas ostatniego okrążenia i pojedynczych sektorów, odległości do lidera i samochodu z przodu (delty), a także pozycje startowe i końcowe.

Do szkolenia i testowania modelu użyte zostały wszystkie zebrane dane.

Pozycja końcowa *finishing_position* jest naszą szukaną.

0	Timestamp	float64
1	tyre_wear_rear_left	float64
2	tyre_wear_rear_right	float64
3	tyre_wear_front_left	float64
4	tyre_wear_front_right	float64
5	fuel_in_tank	float64
6	actual_tyre_compound	int64
7	visual_tyre_compound	int64
8	tyres_age_laps	int64
9	ers_store_energy	float64
10	tyre_surface_temp_rear_left	int64
11	tyre_surface_temp_rear_right	int64
12	tyre_surface_temp_front_left	int64
13	tyre_surface_temp_front_right	int64
14	tyre_inner_temp_rear_left	int64
15	tyre_inner_temp_rear_right	int64
16	tyre_inner_temp_front_left	int64
17	tyre_inner_temp_front_right	int64
18	tyre_pressure_rear_left	float64
19	tyre_pressure_rear_right	float64
20	tyre_pressure_front_left	float64
21	tyre_pressure_front_right	float64
22	lap_distance	float64
23	last_lap_ms	int64
24	sector_1_ms	int64
25	sector_2_ms	int64
26	delta_in_front_ms	int64
27	delta_leader_ms	int64
28	car_position	int64
29	current_lap	int64
30	num_pit_stops	int64
31	pit_timer_active	int64
32	pit_timer_ms	int64
33	track_temp	int64
34	air_temp	int64
35	total_laps	int64
36	pit_rejoin_position	int64
37	starting_position	int64
38	finishing_position	int64
39	dnf	bool
40	penalties_time	int64
41	fastest_lap	bool

7.2 Przygotowanie danych

Aby móc wprowadzić zebrane dane do modelu, należy najpierw wylosować zbiór wyścigów treningowych i testowych. Aby uniknąć przeplotu danych z jednego wyścigu między zbiorem treningowym a testowym, dane są rozdzielane na poziomie całych wyścigów.

Następnie dane testowe dzielimy na 3 części: początek, środek i koniec wyścigu w stosunku 30-40-30. W ten sposób jesteśmy w stanie sprawdzić dokładność modelu w różnych momentach wyścigu.

Mieszamy dane treningowe i testowe, a na koniec w zależności od preferencji je normalizujemy. Normalizacja dotyczy wszystkich danych numerycznych, a nie dotyczy danych typu bool, pozycji startowej i końcowej oraz numerów okrążeń.

7.3 Wybór modelu

Na początku planowaliśmy użyć klasyfikatora Naiwnego Bayesa. Problemem okazała się ilość danych, które możemy zebrać, a w szczególności reprezentacja każdej klasy wynikowej.

Aby Naiwny Bayes był w stanie sprawnie przewidywać pozycję na mecie, każda pozycja powinna być odpowiednio reprezentowana w zbiorze treningowym. Pozycji na mecie jest 20, a więc minimum 20 wyścigów z każdą z nich musiałoby się tam znaleźć. Taki scenariusz był poza naszymi możliwościami, gdy już spędziliśmy okolice 15h na zebranie wszystkich danych.

Dodatkowo, Naiwny Bayes dobiera jedynie pojedynczą klasę wynikową. Z tego powodu wybór padł na Bayesowską Regresję Liniową, która okazała się naturalnym zastępstwem i bardzo dobrze wpisuje się w typ naszych danych.

Bayesowska Regresja Liniowa, w odróżnieniu od zwykłej Regresji Liniowej, odróżnia się tym, że nie przewiduje pojedynczego wyniku, lecz cały rozkład wyników z ich prawdopodobieństwami (traktuje parametry modelu jako zmienne losowe z rozkładem prawdopodobieństwa, a nie jako wartości punktowe). Dla takiego scenariusza zdecydowanie bardziej stosowne jest przewidywanie „kierowca dojedzie na 7 miejscu na 73%” lub „kierowca na 95% skończy między 4 a 6 pozycją”.

Model oczywiście zwraca wyniki zmiennoprzecinkowe, w tym wychodzące poza zakres $[1;20]$, wprowadzając przymus interpretacji. Chociażby, czy ujemny wynik oznacza dominujące zwycięstwo? Z tego powodu wprowadziliśmy kilka uproszczeń - faktyczny wynik jest zwracany, ale dokładność sprawdzaliśmy dla wyniku ograniczonego przedziałem $[1;20]$ i zaokrąglonego do najbliższej liczby całkowitej.

7.4 Model matematyczny

Bayesowska Regresja Liniowa rozszerza standardową regresję liniową, traktując współczynniki regresji jako zmienne losowe z rozkładem a priori. Aktualizujemy go za pomocą zaobserwowanych danych, aby uzyskać rozkład a posteriori, którego następnie używamy do predykcji.

W tym modelu korzystamy z podstawowego wzoru:

$$y = X\beta + \varepsilon$$

gdzie:

- y to wektor obserwacji,
- X to macierz projektowa (*design matrix*)
- β to wektor parametrów,
- $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$ to szum Gaussowski.

Zakładamy, że parametry β mają rozkład a priori:

$$\beta \sim \mathcal{N}\left(0, \frac{1}{\alpha} I\right)$$

gdzie α jest precyzją priorową (odwrotnością wariancji). Precyzja szumu oznaczona jest jako β , więc $\sigma^2 = \frac{1}{\beta}$.

Po obserwacji danych (X, y) , rozkład a posteriori dla β również przyjmuje rozkład normalny:

$$\beta|X, y \sim \mathcal{N}(m_N, S_N)$$

gdzie macierz kowariancji a posteriori S_N wynika ze wzoru:

$$S_N = (\alpha I + \beta X^T X)^{-1}$$

a średnia a posteriori m_N wynika ze wzoru:

$$m_N = \beta S_N X^T y$$

Wtedy dla predykcji w nowym punkcie x_* , rozkład ma postać:

$$y_*|x_*, X, y \sim \mathcal{N}(\mu_*, \sigma_*^2)$$

gdzie średnia i wariancja predykcji wyrażone są wzorami:

$$\mu_* = x_* m_N$$

$$\sigma_*^2 = \frac{1}{\beta} + x_* S_N x_*^T$$

7.5 Klasa modelu

Kod klasy zaimplementowanego modelu Bayesowskiej Regresji Liniowej ma postać:

```
class BayesianLinearRegression:
    def __init__(self, alpha=1.0, beta=1.0):
        self.alpha = alpha # Prior precision
        self.beta = beta # Noise precision
        self.m_N = None # Posterior mean
        self.S_N = None # Posterior covariance

    def fit(self, X, y):
        X = np.asarray(X)
        y = np.asarray(y)
        n_features = X.shape[1]
        I = np.eye(n_features)
        A = self.alpha * I + self.beta * X.T @ X
        self.S_N = np.linalg.solve(A, I)
        self.m_N = self.beta * self.S_N @ X.T @ y

    def predict(self, X, return_std=False):
        X = np.asarray(X)
        y_mean = X @ self.m_N
        if return_std:
            y_var = 1 / self.beta + np.sum(X @ self.S_N * X, axis=1)
            y_std = np.sqrt(y_var)
            return y_mean, y_std
        return y_mean
```

Metoda *fit* oblicza macierz A , odpowiadającą precyzji a posteriori; macierz kowariancji a posteriori S_N ; oraz średnią a posteriori m_N .

Metoda *predict* oblicza średnią predykcji y_{mean} oraz wariancję predykcji y_{var} .

8 Testy modelu sztucznej inteligencji

8.1 Opis testów

Aby otrzymać średni wynik przeprowadzamy n przejść przez proces trenowania i testowania modelu. Dla każdego przejścia zmieniane jest ziarno losowania początkowego podziału plików.

Dodatkowo, testujemy wpływ normalizacji i optymalizacji parametrów na wyniki danych, dlatego przygotowaliśmy następującą funkcję:

```
run_multiple_experiments(  
    n_runs=100,  
    train_size_ratio=0.8,  
    normalize=False,  
    optimize_hyperparameters=False  
)
```

która jako argumenty przyjmuje ilość przejść, podział treningowo-testowy oraz dwie flagi - normalizacji i optymalizacji parametrów.

Funkcja zaczyna się od zdefiniowania pustych tablic, kolumn poddawanych normalizacji i usunięcia wybranych kolumn:

```
metrics = {  
    'start_accuracy': [], 'mid_accuracy': [], 'end_accuracy': [],  
    'start_mae': [], 'mid_mae': [], 'end_mae': []  
}  
  
numerical_cols = [  
    'tyre_wear_rear_left', 'tyre_wear_rear_right',  
    'tyre_wear_front_left', 'tyre_wear_front_right',  
    'fuel_in_tank', 'ers_store_energy',  
    'tyre_surface_temp_rear_left', 'tyre_surface_temp_rear_right',  
    'tyre_surface_temp_front_left',  
    'tyre_surface_temp_front_right', 'tyre_inner_temp_rear_left',  
    'tyre_inner_temp_rear_right', 'tyre_inner_temp_front_left',  
    'tyre_inner_temp_front_right',  
    'tyre_pressure_rear_left', 'tyre_pressure_rear_right',  
    'tyre_pressure_front_left',  
    'tyre_pressure_front_right', 'lap_distance', 'last_lap_ms',  
    'sector_1_ms', 'sector_2_ms',  
    'delta_in_front_ms', 'delta_leader_ms', 'track_temp',  
    'air_temp', 'penalties_time'  
]  
  
drop_cols = ['finishing_position', 'actual_tyre_compound',  
    'visual_tyre_compound']
```

Cechy *actual_tyre_compound* oraz *visual_tyre_compound* zostały zmodyfikowane z uwagi na fakt, że reprezentują identyfikator opon w teledzienniku gry, zamiast faktycznego znaczenia (np. twardości mieszanki):

```
def process_tyre_compound(df):
    df['is_soft_compound'] = df['visual_tyre_compound'] == 16
    df['is_medium_compound'] = df['visual_tyre_compound'] == 17
    df['is_hard_compound'] = df['visual_tyre_compound'] == 18

    df = df.drop('visual_tyre_compound', axis=1)
    return df
```

Następnie dzielimy wyścigi na 3 części: początek, środek i koniec (wszystkie dane zostały zebrane na torze w Bahrajnie, gdzie długość wyścigów wyniosła 14 okrążeń):

```
total_laps = 14
start_laps = int(0.3 * total_laps)          # ~4 laps
end_laps = int(0.3 * total_laps)            # ~4 laps
mid_laps = total_laps - start_laps - end_laps # ~6 laps
```

Rozpoczynamy pętlę *n* przejść, ustawiamy ziarno oraz tworzymy zestawy plików testowych i treningowych:

```
for run in range(n_runs):
    random.seed(run)

    files = glob.glob('*.csv')
    random.shuffle(files)

    train_size = int(train_size_ratio * len(files))
    train_files = files[:train_size]
    test_files = files[train_size:]
```

Tworzymy zbiór treningowy:

```
df_train = pd.DataFrame()
for filename in train_files:
    df = pd.read_csv(filename)
    df[df.select_dtypes(include='bool').columns] =
        df.select_dtypes(include='bool').astype(int)
    df_train = pd.concat([df_train, df])
```

Tworzymy zbiory testowe:

```
df_test_start = pd.DataFrame()
df_test_mid = pd.DataFrame()
df_test_end = pd.DataFrame()

for filename in test_files:
    df = pd.read_csv(filename)
    df[df.select_dtypes(include='bool').columns] =
        df.select_dtypes(include='bool').astype(int)
    df_start = df[df['current_lap'] <= start_laps]
    df_mid = df[(df['current_lap'] > start_laps) &
        (df['current_lap'] <= start_laps + mid_laps)]
    df_end = df[df['current_lap'] > start_laps + mid_laps]
    df_test_start = pd.concat([df_test_start, df_start])
    df_test_mid = pd.concat([df_test_mid, df_mid])
    df_test_end = pd.concat([df_test_end, df_end])
```

Dzielimy na zbiory X i y:

```
X_train = df_train.drop(columns=drop_cols)
y_train = df_train['finishing_position']
X_test_start = df_test_start.drop(columns=drop_cols)
y_test_start = df_test_start['finishing_position']
X_test_mid = df_test_mid.drop(columns=drop_cols)
y_test_mid = df_test_mid['finishing_position']
X_test_end = df_test_end.drop(columns=drop_cols)
y_test_end = df_test_end['finishing_position']
```

Normalizujemy dane, jeśli tak zdecydowano:

```
if normalize:
    scaler = StandardScaler()
    X_train[numerical_cols] =
        scaler.fit_transform(X_train[numerical_cols])
    X_test_start[numerical_cols] =
        scaler.transform(X_test_start[numerical_cols])
    X_test_mid[numerical_cols] =
        scaler.transform(X_test_mid[numerical_cols])
    X_test_end[numerical_cols] =
        scaler.transform(X_test_end[numerical_cols])
```

Optymalizujemy parametry alpha i beta, jeśli tak zdecydowano, korzystając z utworzonego modelu Bayesowskiej Regresji Liniowej:

```
if optimize_hyperparameters:
    optuna.logging.set_verbosity(optuna.logging.ERROR)
    def objective(trial, X_train, y_train, X_val, y_val):
        alpha = trial.suggest_float('alpha', 1e-3, 1e3, log=True)
        beta = trial.suggest_float('beta', 1e-3, 1e3, log=True)
        model = BayesianLinearRegression(alpha=alpha, beta=beta)
        model.fit(X_train, y_train)
        preds, std = model.predict(X_val, return_std=True)
        mae = mean_absolute_error(y_val, preds)
        nlpd = -np.mean(norm.logpdf(y_val, loc=preds, scale=std))
        return mae + 0.1 * nlpd

    X_train_sub, X_val, y_train_sub, y_val =
        train_test_split(X_train,
                        y_train,
                        test_size=0.2,
                        random_state=42)
    study = optuna.create_study(direction='minimize')
    study.optimize(lambda trial:
        objective(trial, X_train_sub, y_train_sub, X_val, y_val),
        n_trials=50)
    best_alpha, best_beta = study.best_params['alpha'],
        study.best_params['beta']
else:
    best_alpha, best_beta = 1.0, 1.0
```

W końcu trenujemy model:

```
model = BayesianLinearRegression(alpha=best_alpha, beta=best_beta)
model.fit(X_train, y_train)
```

Sprawdzamy dokładności dla każdego z wydzielonych momentów wyścigu:

```
def compute_accuracy(model, X_test, y_test):
    y_pred, _ = model.predict(X_test, return_std=True)
    y_pred_discrete = np.clip(np.round(y_pred), 1, 20).astype(int)
    y_test = y_test.astype(int)
    accuracy = np.mean(y_pred_discrete == y_test) * 100
    return accuracy

def evaluate_regression_model(model, X_test, y_test, return_std):
    y_pred, y_std = model.predict(X_test, return_std=True)
    mae = mean_absolute_error(y_test, y_pred)
    accuracy = compute_accuracy(model, X_test, y_test)
    return mae, accuracy

mae_start, acc_start =
    evaluate_regression_model(model, X_test_start, y_test_start)
mae_mid, acc_mid =
    evaluate_regression_model(model, X_test_mid, y_test_mid)
mae_end, acc_end =
    evaluate_regression_model(model, X_test_end, y_test_end)
```

Zapisujemy wyniki do tablicy:

```
metrics['start_accuracy'].append(acc_start)
metrics['mid_accuracy'].append(acc_mid)
metrics['end_accuracy'].append(acc_end)
metrics['start_mae'].append(mae_start)
metrics['mid_mae'].append(mae_mid)
metrics['end_mae'].append(mae_end)
```

Teraz na potrzeby reprezentacyjne wykonujemy obliczenia dla jednego konkretnego wyścigu:

```
test_race_file = test_files[0]
df_race = pd.read_csv(test_race_file)
df_race[df_race.select_dtypes(include='bool').columns] =
    df_race.select_dtypes(include='bool').astype(int)
X_race = df_race.drop(columns=drop_cols)
y_race = df_race['finishing_position']

if normalize:
    X_race[numerical_cols] =
        scaler.transform(X_race[numerical_cols])

df_race_start = df_race[df_race['current_lap'] <= start_laps]
df_race_mid = df_race[(df_race['current_lap'] > start_laps) &
    (df_race['current_lap'] <= start_laps + mid_laps)]
df_race_end = df_race[df_race['current_lap'] > start_laps +
    mid_laps]

X_race_start = df_race_start.drop(columns=drop_cols)
X_race_mid = df_race_mid.drop(columns=drop_cols)
X_race_end = df_race_end.drop(columns=drop_cols)

if normalize:
    X_race_start[numerical_cols] =
        scaler.transform(X_race_start[numerical_cols])
    X_race_mid[numerical_cols] =
        scaler.transform(X_race_mid[numerical_cols])
    X_race_end[numerical_cols] =
        scaler.transform(X_race_end[numerical_cols])

y_pred, y_std = model.predict(X_race, return_std=True)

actual_position = y_race.iloc[0]
pred_continuous = np.average(y_pred, weights=1/(y_std**2))
pred_discrete = int(np.clip(np.round(pred_continuous), 1, 20))
uncertainty_std = np.mean(y_std)
```

Zapisujemy wyniki do tablicy:

```
if run == n_runs - 1:
    final_model = model
    final_X_train = X_train
    if not df_race_start.empty:
        y_pred_start, y_std_start =
            model.predict(X_race_start, return_std=True)
        pred_continuous_start =
            np.average(y_pred_start, weights=1/(y_std_start**2))
        pred_discrete_start =
            int(np.clip(np.round(pred_continuous_start), 1, 20))
        uncertainty_std_start =
            np.mean(y_std_start)
    else:
        y_pred_start, y_std_start = np.array([]), np.array([])
        pred_continuous_start,
        pred_discrete_start,
        uncertainty_std_start = np.nan, np.nan, np.nan

    if not df_race_mid.empty:
        y_pred_mid, y_std_mid =
            model.predict(X_race_mid, return_std=True)
        pred_continuous_mid =
            np.average(y_pred_mid, weights=1/(y_std_mid**2))
        pred_discrete_mid =
            int(np.clip(np.round(pred_continuous_mid), 1, 20))
        uncertainty_std_mid = np.mean(y_std_mid)
    else:
        y_pred_mid, y_std_mid = np.array([]), np.array([])
        pred_continuous_mid,
        pred_discrete_mid,
        uncertainty_std_mid = np.nan, np.nan, np.nan

    if not df_race_end.empty:
        y_pred_end, y_std_end =
            model.predict(X_race_end, return_std=True)
        pred_continuous_end =
            np.average(y_pred_end, weights=1/(y_std_end**2))
        pred_discrete_end =
            int(np.clip(np.round(pred_continuous_end), 1, 20))
        uncertainty_std_end = np.mean(y_std_end)
    else:
        y_pred_end, y_std_end = np.array([]), np.array([])
        pred_continuous_end,
        pred_discrete_end,
        uncertainty_std_end = np.nan, np.nan, np.nan
```



```

final_test_race_results = {
    'file': test_race_file,
    'actual': actual_position,
    'continuous': pred_continuous,
    'discrete': pred_discrete,
    'uncertainty': uncertainty_std,
    'y_pred_race': y_pred,
    'y_std_race': y_std,
    'start_continuous': pred_continuous_start,
    'start_discrete': pred_discrete_start,
    'start_uncertainty': uncertainty_std_start,
    'y_pred_start': y_pred_start,
    'y_std_start': y_std_start,
    'mid_continuous': pred_continuous_mid,
    'mid_discrete': pred_discrete_mid,
    'mid_uncertainty': uncertainty_std_mid,
    'y_pred_mid': y_pred_mid,
    'y_std_mid': y_std_mid,
    'end_continuous': pred_continuous_end,
    'end_discrete': pred_discrete_end,
    'end_uncertainty': uncertainty_std_end,
    'y_pred_end': y_pred_end,
    'y_std_end': y_std_end
}

```

Obliczamy średnią i odchylenie standardowe:

```

summary = {}
for key, values in metrics.items():
    summary[f'{key}_mean'] = np.mean(values)
    summary[f'{key}_std'] = np.std(values)

```

Wypisujemy wyniki:

```
print("\nExperiment Summary (Mean  $\pm$  Std):")
print(f"Start Laps Accuracy:
      {summary['start_accuracy_mean']:.2f}%  $\pm$ 
      {summary['start_accuracy_std']:.2f}%")
print(f"Mid Laps Accuracy:
      {summary['mid_accuracy_mean']:.2f}%  $\pm$ 
      {summary['mid_accuracy_std']:.2f}%")
print(f"End Laps Accuracy:
      {summary['end_accuracy_mean']:.2f}%  $\pm$ 
      {summary['end_accuracy_std']:.2f}%")
print(f"Start Laps MAE:
      {summary['start_mae_mean']:.2f}  $\pm$ 
      {summary['start_mae_std']:.2f}")
print(f"Mid Laps MAE:
      {summary['mid_mae_mean']:.2f}  $\pm$ 
      {summary['mid_mae_std']:.2f}")
print(f"End Laps MAE:
      {summary['end_mae_mean']:.2f}  $\pm$ 
      {summary['end_mae_std']:.2f}")
```

Przykładowy wynik:

```
Experiment Summary (Mean  $\pm$  Std):
Start Laps Accuracy: 19.93%  $\pm$  14.78%
Mid Laps Accuracy: 23.06%  $\pm$  12.23%
End Laps Accuracy: 26.14%  $\pm$  15.12%
Start Laps MAE: 1.90  $\pm$  0.55
Mid Laps MAE: 1.73  $\pm$  0.49
End Laps MAE: 1.40  $\pm$  0.49
```

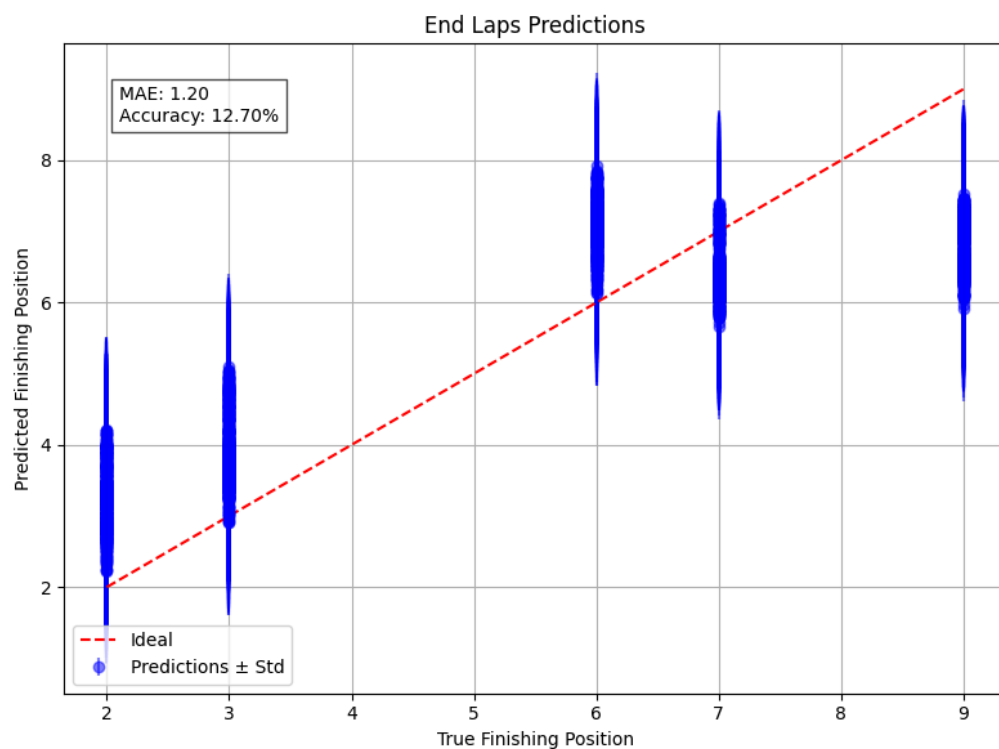
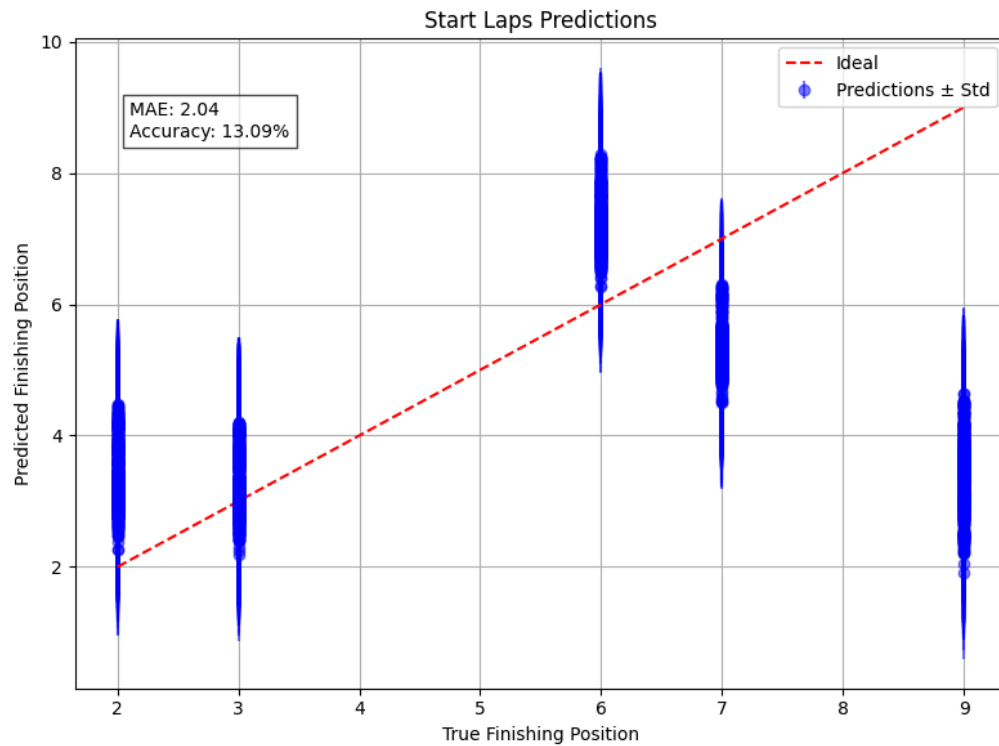
Tworzymy wykresy zależności pozycji przewidywanych od startowych dla całego zbioru testowego:

```
def evaluate_regression_model_with_plot(model, X_test, y_test,
                                       title, return_std=True):
    y_pred, y_std = model.predict(X_test, return_std=True)
    mae = mean_absolute_error(y_test, y_pred)
    accuracy = compute_accuracy(model, X_test, y_test)

    plt.figure(figsize=(8, 6))
    plt.errorbar(y_test, y_pred, yerr=y_std, fmt='o',
                color='blue', alpha=0.5, label='Predictions ± Std')
    plt.plot([min(y_test), max(y_test)],
            [min(y_test), max(y_test)], 'r--', label='Ideal')
    plt.xlabel('True Finishing Position')
    plt.ylabel('Predicted Finishing Position')
    plt.title(f'{title} Predictions')
    plt.legend()
    plt.grid(True)
    plt.text(min(y_test)*1.05, max(y_test)*0.95,
            f'MAE: {mae:.2f}\nAccuracy: {accuracy:.2f}%',
            fontsize=10, bbox=dict(facecolor='white'))
    plt.tight_layout()
    plt.show()
    return mae, accuracy

mae_start, acc_start = evaluate_regression_model_with_plot(
    final_model, X_test_start, y_test_start, 'Start Laps')
mae_mid, acc_mid = evaluate_regression_model_with_plot(
    final_model, X_test_mid, y_test_mid, 'Mid Laps')
mae_end, acc_end = evaluate_regression_model_with_plot(
    final_model, X_test_end, y_test_end, 'End Laps')
```

Poniżej przykładowe wykresy tej zależności dla początkowych i końcowych okrążeń - niebieskie punkty oznaczają przewidywane pozycje z ich odchyleniem dla każdego rekordu, a przerywana linia oznacza faktyczny wynik:



Następnie wypisujemy podsumowanie jednego z wyścigów z konkretnymi wynikami:

```
print("\nFinal Run Test Race Results:")
print(f"Test Race:
      {final_test_race_results['file']}")
print(f"Actual Finishing Position:
      {final_test_race_results['actual']}")
print()
print(f"Overall Continuous:
      {final_test_race_results['continuous']:.3f}")
print(f"Overall Discrete:
      {final_test_race_results['discrete']}")
print(f"Overall Uncertainty:
      {final_test_race_results['uncertainty']:.3f}")
print()
print(f"Start Continuous:
      {final_test_race_results['start_continuous']:.3f} ±
      {final_test_race_results['start_uncertainty']:.3f}")
print(f"Start Discrete:
      {final_test_race_results['start_discrete']}")
print()
print(f"Mid Continuous:
      {final_test_race_results['mid_continuous']:.3f} ±
      {final_test_race_results['mid_uncertainty']:.3f}")
print(f"Mid Discrete:
      {final_test_race_results['mid_discrete']}")
print()
print(f"End Continuous:
      {final_test_race_results['end_continuous']:.3f} ±
      {final_test_race_results['end_uncertainty']:.3f}")
print(f"End Discrete:
      {final_test_race_results['end_discrete']}")
```

Przykładowy wynik wygląda następująco i zawiera informacje o przewidywanej pozycji w formie zmiennoprzecinkowej, całkowitej i jej niepewności dla każdego z wyznaczonych momentów wyścigu:

Test Race: 2025-06-04_17-03-39.csv

Actual Finishing Position: 6

Overall Continuous: 6.915

Overall Discrete: 7

Overall Uncertainty: 1.306

Start Continuous: 7.211 $\pm$ 1.306

Start Discrete: 7

Mid Continuous: 6.625 $\pm$ 1.306

Mid Discrete: 7

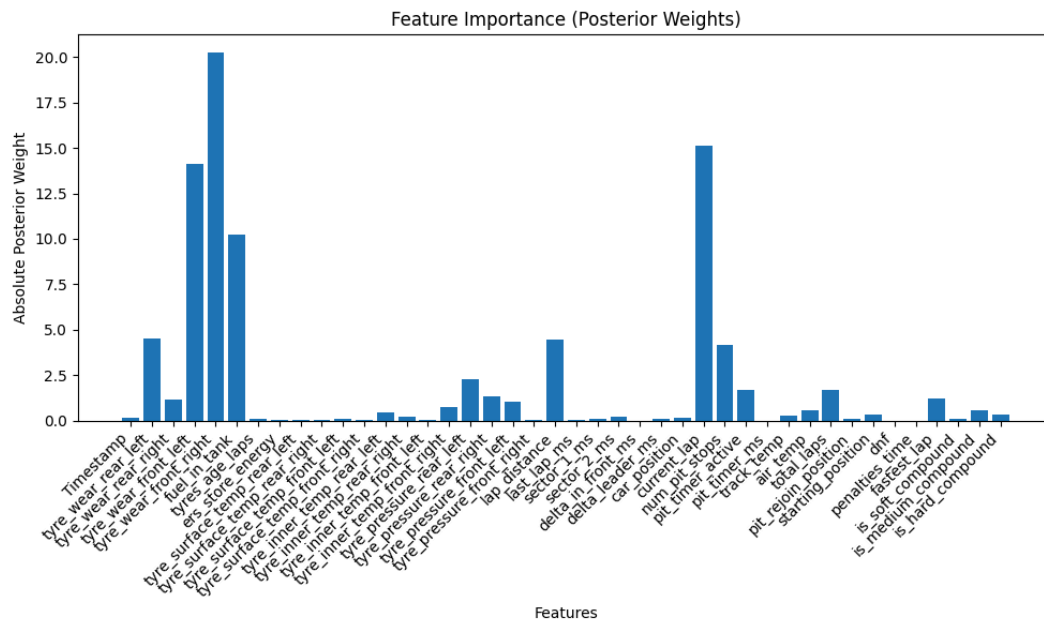
End Continuous: 7.073 $\pm$ 1.306

End Discrete: 7

Tworzymy wykres wag cech:

```
plt.figure(figsize=(10, 6))
weights = final_model.m_N
features = final_X_train.columns
plt.bar(features, np.abs(weights))
plt.xticks(rotation=45, ha='right')
plt.xlabel('Features')
plt.ylabel('Absolute Posterior Weight')
plt.title('Feature Importance (Posterior Weights)')
plt.tight_layout()
plt.show()
```

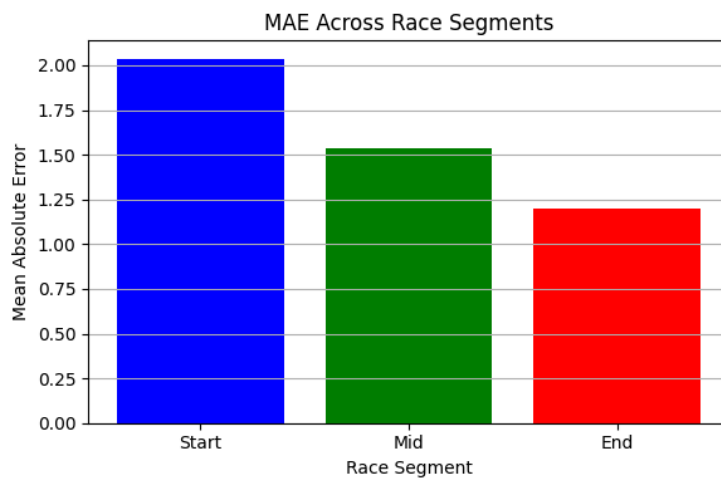
Przykładowy wykres wag cech wygląda następująco:



Tworzymy wykres średniego błędu w zależności od momentu wyścigu:

```
plt.figure(figsize=(6, 4))
splits = ['Start', 'Mid', 'End']
maes = [mae_start, mae_mid, mae_end]
plt.bar(splits, maes, color=['blue', 'green', 'red'])
plt.xlabel('Race Segment')
plt.ylabel('Mean Absolute Error')
plt.title('MAE Across Race Segments')
plt.grid(True, axis='y')
plt.tight_layout()
plt.show()
```

Przykładowy wykres błędu średniego wygląda następująco:



Tworzymy rozkład przewidywanych pozycji z ich prawdopodobieństwami:

```
def plot_combined_position_probabilities(results, race_file):
    positions = np.arange(1, 21)

    segments = [
        ('Overall Race', results['y_pred_race'],
         results['y_std_race'], 'purple'),
        ('Start Segment', results['y_pred_start'],
         results['y_std_start'], 'blue'),
        ('Mid Segment', results['y_pred_mid'],
         results['y_std_mid'], 'green'),
        ('End Segment', results['y_pred_end'],
         results['y_std_end'], 'orange'),
    ]
    bar_width = 0.2
    offset = [-1.5, -0.5, 0.5, 1.5]
    plt.figure(figsize=(10, 6))

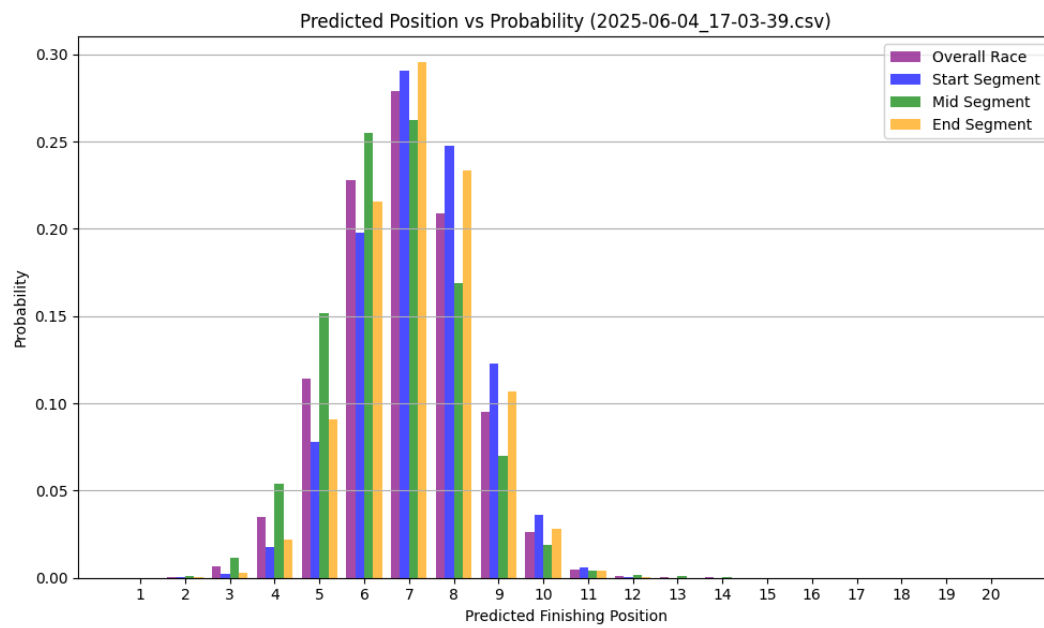
    for i, (label, y_pred, y_std, color) in enumerate(segments):
        probabilities = np.zeros(len(positions))
        for j, pos in enumerate(positions):
            probabilities[j] = np.mean(
                norm.pdf(pos, loc=y_pred, scale=y_std))
        probabilities = probabilities / np.sum(probabilities)

        pos_shifted = positions + bar_width * offset[i]
        plt.bar(pos_shifted, probabilities, width=bar_width,
                color=color, alpha=0.7, label=label)

    plt.xlabel('Predicted Finishing Position')
    plt.ylabel('Probability')
    plt.title(f'Predicted Position vs Probability ({race_file})')
    plt.grid(True, axis='y')
    plt.xticks(positions)
    plt.legend()
    plt.tight_layout()
    plt.show()

plot_combined_position_probabilities(final_test_race_results,
final_test_race_results['file'])
```

Przykładowy rozkład przewidywanych pozycji:



8.2 Wyniki testów

W ramach przeprowadzonych testów sprawdzane były 4 konfiguracje, wszystkie dla 100 przebiegów i podziału treningowo-testowego w stosunku 80-20:

8.2.1 Nienormalizowane dane, nieoptymalizowane parametry

Wywołanie funkcji dla danych założeń:

```
run_multiple_experiments(n_runs=100,  
                        train_size_ratio=0.8,  
                        normalize=False,  
                        optimize_hyperparameters=False)
```

Wyniki eksperymentu prezentują się następująco:

Experiment Summary (Mean $\pm$ Std):
Start Laps Accuracy: 19.91% $\pm$ 15.02%
Mid Laps Accuracy: 23.18% $\pm$ 12.03%
End Laps Accuracy: 27.24% $\pm$ 15.25%
Start Laps MAE: 1.94 $\pm$ 0.58
Mid Laps MAE: 1.76 $\pm$ 0.51
End Laps MAE: 1.44 $\pm$ 0.52

Widać w nich jasną zależność - w późniejszych częściach wyścigu model jest w stanie celniej przewidzieć pozycję na mecie, ale odchylenie wciąż jest stosunkowo duże.

8.2.2 Nienormalizowane dane, zoptymalizowane parametry

Wywołanie funkcji dla danych założeń:

```
run_multiple_experiments(n_runs=100,  
                        train_size_ratio=0.8,  
                        normalize=False,  
                        optimize_hyperparameters=True)
```

Wyniki eksperymentu prezentują się następująco:

Experiment Summary (Mean $\pm$ Std):
Start Laps Accuracy: 19.79% $\pm$ 14.89%
Mid Laps Accuracy: 23.28% $\pm$ 12.00%
End Laps Accuracy: 27.41% $\pm$ 15.28%
Start Laps MAE: 1.95 $\pm$ 0.58
Mid Laps MAE: 1.76 $\pm$ 0.51
End Laps MAE: 1.45 $\pm$ 0.52

W porównaniu do bazowego przykładu, dokładność minimalnie się zwiększyła w środku i na koniec wyścigu. Odchylenia i średnie błędy pozostały praktycznie bez zmian. Znacząco jednak wydłużył się czas przetwarzania danych, związany z optymalizacją parametrów z pomocą biblioteki *optuna*.

8.2.3 Znormalizowane dane, nieoptymalizowane parametry

Wywołanie funkcji dla danych założeń:

```
run_multiple_experiments(n_runs=100,  
                        train_size_ratio=0.8,  
                        normalize=True,  
                        optimize_hyperparameters=False)
```

Wyniki eksperymentu prezentują się następująco:

```
Experiment Summary (Mean ± Std):  
Start Laps Accuracy: 19.98% ± 15.12%  
Mid Laps Accuracy: 23.63% ± 12.18%  
End Laps Accuracy: 27.68% ± 15.29%  
Start Laps MAE: 1.92 ± 0.58  
Mid Laps MAE: 1.73 ± 0.50  
End Laps MAE: 1.42 ± 0.48
```

W porównaniu do bazowego przykładu, dokładność również minimalnie się zwiększyła. Średnie błędy minimalnie się zmniejszyły.

8.2.4 Znormalizowane dane, zoptymalizowane parametry

Wywołanie funkcji dla danych założeń:

```
run_multiple_experiments(n_runs=100,  
                        train_size_ratio=0.8,  
                        normalize=True,  
                        optimize_hyperparameters=True)
```

Wyniki eksperymentu prezentują się następująco:

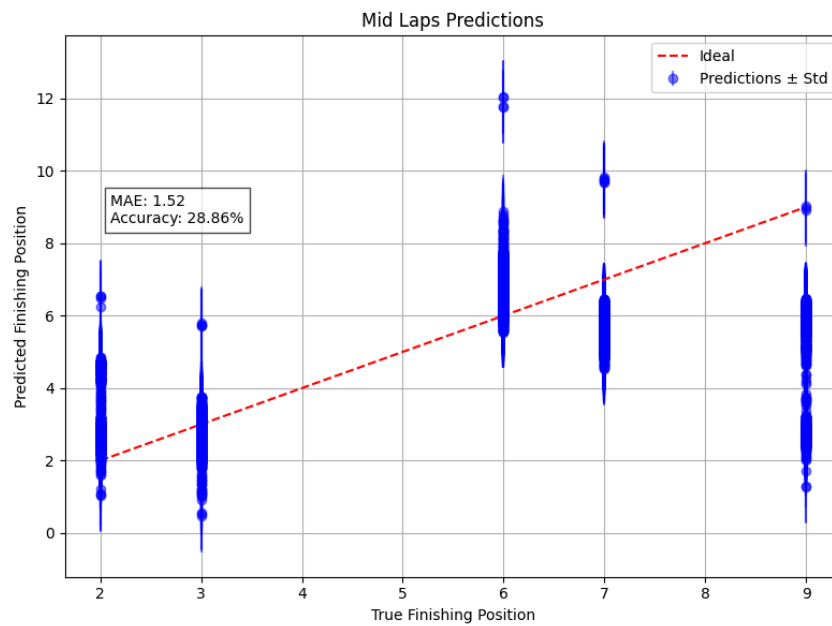
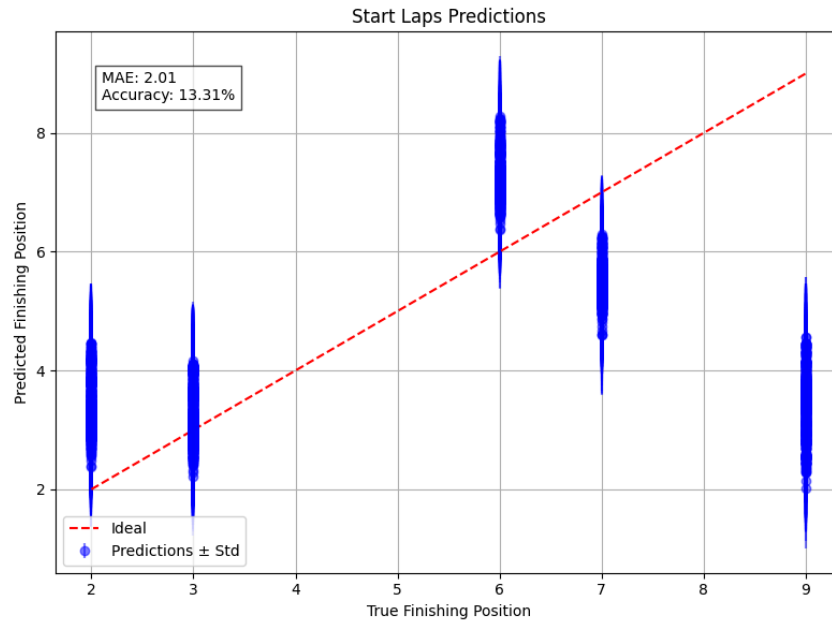
```
Experiment Summary (Mean ± Std):  
Start Laps Accuracy: 19.83% ± 14.91%  
Mid Laps Accuracy: 23.47% ± 11.93%  
End Laps Accuracy: 27.34% ± 15.33%  
Start Laps MAE: 1.94 ± 0.58  
Mid Laps MAE: 1.75 ± 0.50  
End Laps MAE: 1.44 ± 0.51
```

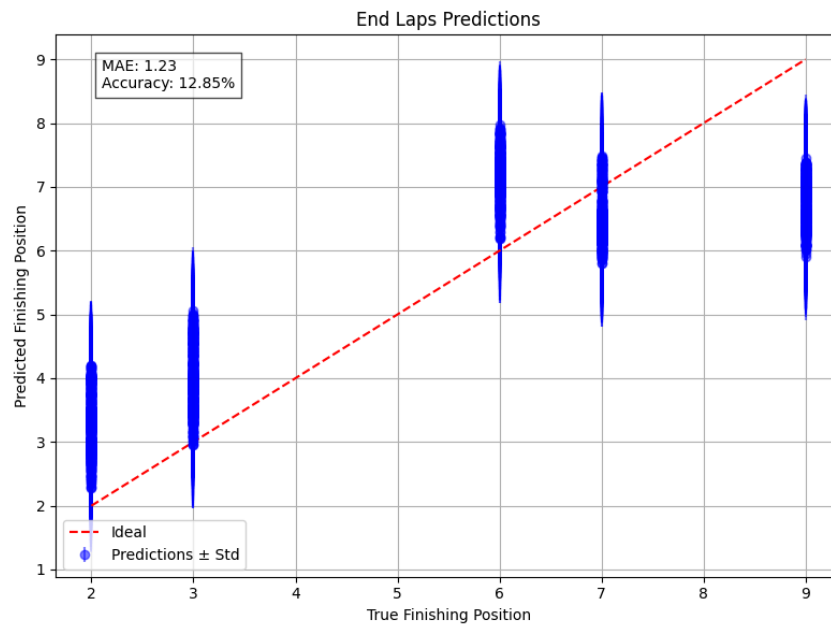
Połączenie normalizacji danych i optymalizacji parametrów dały lepsze wyniki niż w przykładzie bazowym, lecz mimo to lepsza okazała się sama normalizacja. Znow minimalnie wzrosły dokładności w środku i na koniec wyścigu, ale kosztem spadku dokładności na początku i braku zmian średnich błędach.

8.3 Podsumowanie

Z racji, że najbardziej interesującym nas wynikiem poza dokładną pozycją jest średni błąd, weźmy pod uwagę jedynie tę konfigurację z najlepszym wynikiem - Sekcja 8.2.3 - jedynie znormalizowane dane.

Na początku porównajmy wykresy zależności przewidywanej pozycji od prawdziwej pozycji na starcie, w środku i na koniec wyścigu:





Na wykresach jednoznacznie widać, że na przestrzeni trwania wyścigu przewidywane pozycje zbliżają się coraz bardziej do faktycznego wyniku, co potwierdza nasze liczby.

Weźmy teraz pod uwagę konkretny wyścig i wyniki osiągnięte przez model dla pojedynczego przypadku. Tak wygląda podsumowanie przewidywań:

Final Run Test Race Results:

Test Race: 2025-06-04_17-03-39.csv

Actual Finishing Position: 6

Overall Continuous: 6.778

Overall Discrete: 7

Overall Uncertainty: 1.001

Start Continuous: 7.072 $\pm$ 1.000

Start Discrete: 7

Mid Continuous: 6.526 $\pm$ 1.001

Mid Discrete: 7

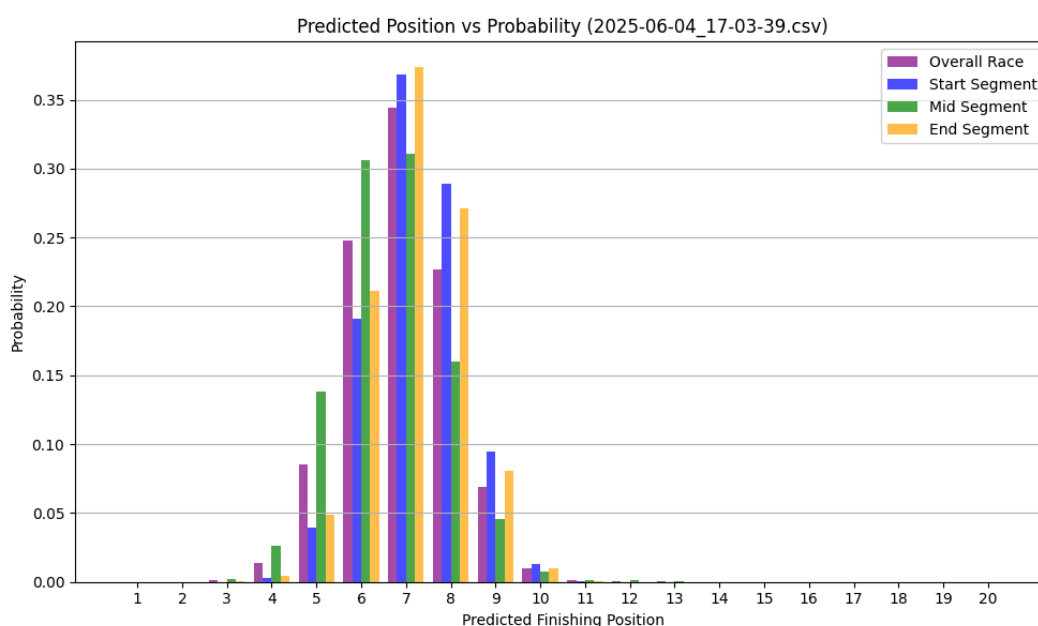
End Continuous: 6.875 $\pm$ 1.001

End Discrete: 7

Faktyczna pozycja na mecie to P6, natomiast model w każdym segmencie wyścigu przewidział pozycję całkowitą P7. Warte uwagi jest natomiast zmiennoprzecinkowa wartość przewidywania, która wahała się między 7.072 na początku, przez 6.526 w środku, do 6.875 na koniec wyścigu. Model brał więc pod uwagę okoliczności, które wynikały z zebranych danych.

Istotna jest też niepewność, która oscyluje w granicach jednej pozycji. Można by więc stwierdzić, że trafnie ocenił pozycję na mecie w środku i na końcu wyścigu, bo ta mieści się w przedziale ± 1 .

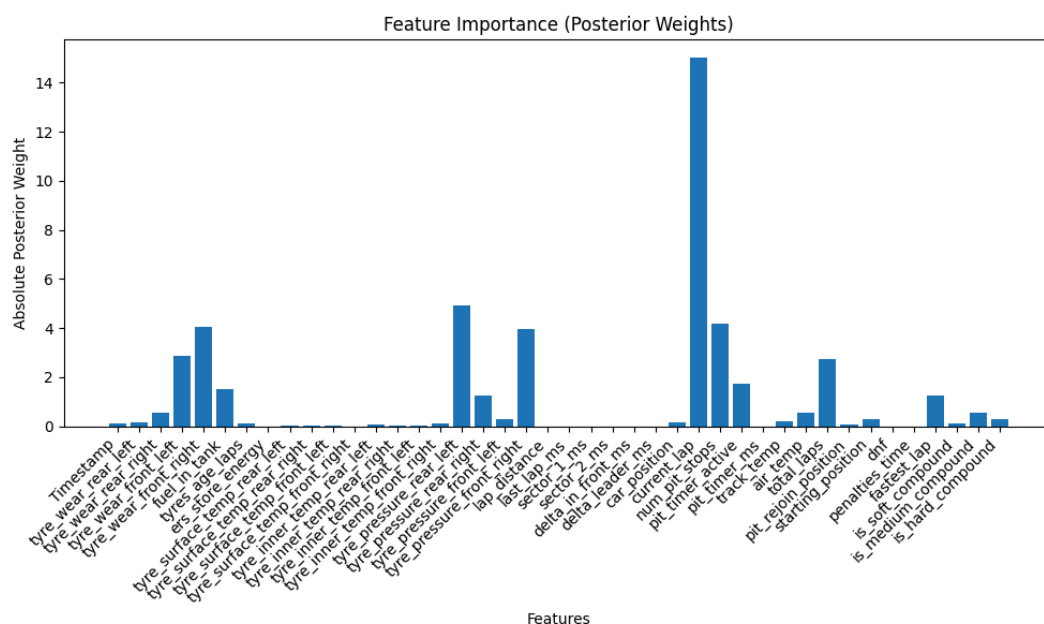
Jednak znacznie ciekawszym wynikiem jest rozkład prawdopodobieństw ukończenia na danej pozycji. Przyjrzyjmy mu się tutaj:



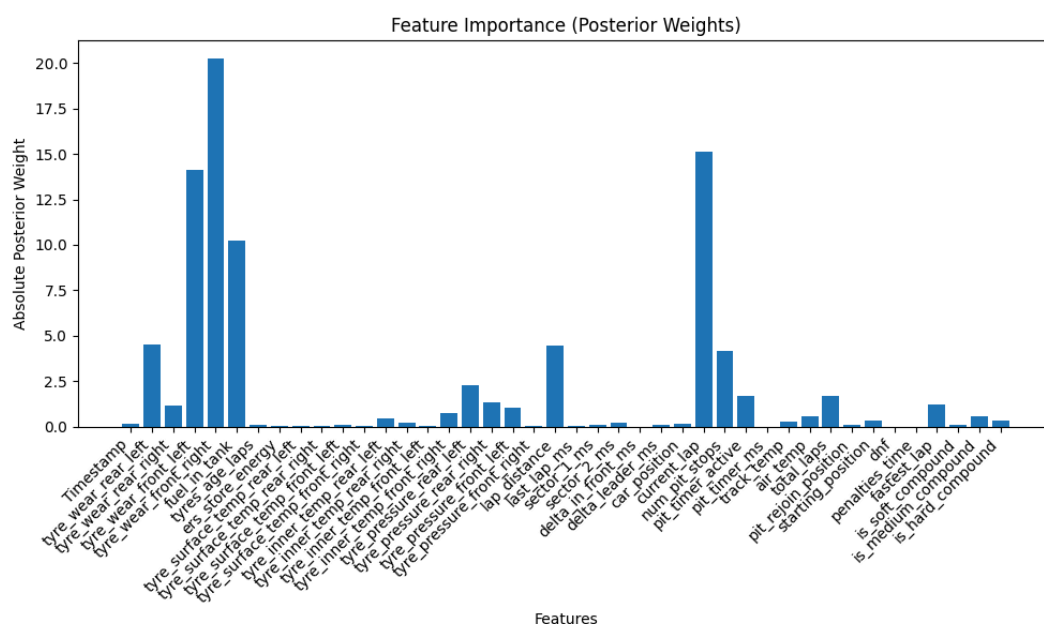
Jasno widać, że z największym prawdopodobieństwem kierowca ukończy na pozycji P7. W środkowym segmencie wyścigu szansa na P6 i P7 była niemal identyczna, natomiast w końcowym kierowcy zdecydowanie bliżej było spaść o jedną pozycję niż ją zyskać.

Wykres rozciąga się od pozycji trzeciej do trzynastej, gdzie wciąż widać jakieś prawdopodobieństwo, a wszystkie pozostałe wartości są pomijalnie mało prawdopodobne.

Jako ostatni omówiony zostanie wykres wag cech, który dla lepszej wizualizacji porównany będzie z konfiguracją modelu bez normalizacji:



Rysunek 5: Wykres wag cech dla modelu z danymi nienormalizowanymi



Rysunek 6: Wykres wag cech dla modelu z danymi znormalizowanymi

Na wykresie pochodzącym z modelu bez normalizacji zdecydowanie góruje cecha aktualnego okrążenia *current_lap*. Wynika to z tego, że dla modelu jest to bardzo ważny wskaźnik tego, jak daleko kierowca znajduje się w wyścigu (im dalej w wyścig tym lepsze przewidywanie). Dodatkowo, jest to wartość całkowita, która pod koniec wyścigu osiąga wartości rzędu kilkudziesięciu jednostek.

W porównaniu, po normalizacji ważniejsze okazują się cechy zużycia opon *tyre_wear*, które w zestawie danych występują jako wartości z przedziału [0;1]. Różnica wag między konkretnymi oponami wynika z charakterystyki toru i wpływu ich zużycia na osiągi pojazdu.

Zmalały także wagi ciśnień opon, co było jedną z trzech niespodziewanych zmian (lub ich braku), ze względu na ich powiązanie z temperaturami opon, które w prawdziwym motorsporcie są bardzo ważnym wskaźnikiem osiągnięć bolidu. Drugim zaskoczeniem była bardzo niska waga temperatur opon właśnie, a trzecim - równie niska waga rodzaju mieszanki opon.

Takie charakterystyki mogą być spowodowane głównie środowiskiem zbierania danych - gra komputerowa nie jest idealnym odzwierciedleniem rzeczywistości; źle dobrany poziom AI botów prowadzących inne bolidy mógł zniwelować wagę cech, nad którymi panuje kierowca (zbyt niski poziom sprawi, że kierowca zawsze będzie wyprzedzał przeciwników, a wtedy dobór opon i utrzymanie ich w dobrym stanie nie ma znaczenia); zbyt krótki dystans wyścigu i zbyt mało danych (strategię zarządzania oponami, która w rzeczywistości gra główną rolę, widać dopiero na długim dystansie).

9 Podsumowanie modelu sztucznej inteligencji

9.1 Potencjał rozwoju

Przyszły rozwój projektu skupiłby się głównie na:

- zebraniu większej ilości danych,
- przewidywaniu innych cech,
- rozszerzeniu o możliwość integracji modelu z aplikacją i wykorzystaniu go w czasie rzeczywistym,
- przetestowaniu innych modeli lub pracach nad siecią neuronową (optymalizacja strategii).

9.2 Wnioski

Najważniejszym wnioskiem jest fakt, że Bayesowska Regresja Liniowa sprawdziła się w naszym projekcie i spełniła nasze oczekiwania.

Byliśmy w stanie otrzymywać całkiem realistyczne wyniki i interpretować je w taki sposób, aby otrzymać dokładność na średnim poziomie około 25% w trakcie 100 przebiegów uczenia i testowania algorytmu, przy tak małym zbiorze testowym jakim jest 25 wyścigów, które nie pokrywają nawet wszystkich pozycji końcowych. Co ważniejsze, średni błąd wyniósł niecałe dwie pozycje na mecie, co dla tak losowego środowiska jakim jest motorsport, jest bardzo dobrym wynikiem.

Dalsze wnioski które można wyciągnąć już z samych wyników:

- dokładność rośnie, a średni błąd maleje wraz z postępem wyścigu, co zgadza się z intuicją - im dalej w wyścig tym bardziej stawka jest ułożona i łatwiej przewidzieć wynik
- duże odchylenia w wynikach świadczą o dużej zmienności w zestawie danych i jego małym rozmiarze (słaba reprezentacja różnych wyników)
- błąd na poziomie 1-2 pozycji jest dosyć trafny, mimo niewielkiego zbioru treningowego
- cechy pokroju aktualnego okrążenia, paliwa w zbiorniku dominują wagami czy zużycia opon, dwie pierwsze szczególnie przed normalizacją danych, co znów dowodzi pierwszego punktu

Oczywistym usprawnieniem wyników byłoby uwzględnienie w danych telemetrii innych kierowców w wyścigu - łatwiej przewidywać w odniesieniu do czegoś, niż tylko względem siebie.

Dokładność modelu zwiększyłoby również powiększenie zbioru treningowego oraz rozważniejszy dobór cech i parametrów.

10 Zarządzanie projektem

Celem tego projektu jest spięcie wszystkich składowych w jeden, spójny ekosystem, z którego będzie można korzystać niezawodnie, a także umieszczenie modelu sztucznej inteligencji w chmurze, aby oddelegować jego szkolenie z lokalnego środowiska.

Cele składowych projektów zostały opisane wyżej.

Projekt został wykonany na przestrzeni sześciu sprintów metodologii SCRUM. Zgodnie z pierwotnym podziałem obowiązków przydzielane były zadania, a terminy ich wykonania ustalane były według potrzeb członka zespołu.

10.1 Analiza projektu

Ukończenie projektu stawia przed nami wiele wyzwań. Niektórymi z nich są:

- spełnienie warunków zaliczenia każdego przedmiotu,
- różny termin poddawania projektów ocenie,
- styczność z nowymi technologiami,
- ograniczony czas, biorąc pod uwagę złożoność projektu,
- co za tym idzie - ryzyko nieukończenia którejś ze składowych.

Aby stawić czoła tym wyzwaniom, musimy odpowiednio rozplanować każdy z projektów, nałożyć sobie ścisłe terminy ukończenia zadań i doskonale komunikować sobie stan prac.

10.2 Decyzje

Wszystkie najważniejsze decyzje dotyczące wykorzystania narzędzi, architektur i rozwiązań zostały podjęte na poziomie tworzenia projektów składowych.

Pozostał nam jedynie wybór dostawcy usług chmurowych. Ten wybór padł na Azure, z kilku powodów:

- jest to narzędzie Microsoft, więc jako studenci mamy do niego łatwy dostęp,
- Azure udostępnia Machine Learning Studio, w którym w prosty sposób można umieścić Jupyter Notebook i dane potrzebne modelowi,
- mieliśmy wcześniej styczność z tym środowiskiem, więc było dla nas przyjazne i sprawdzone.

Dodatkowo, poniżej znajduje się uzasadnienie wyboru pozostałych narzędzi:

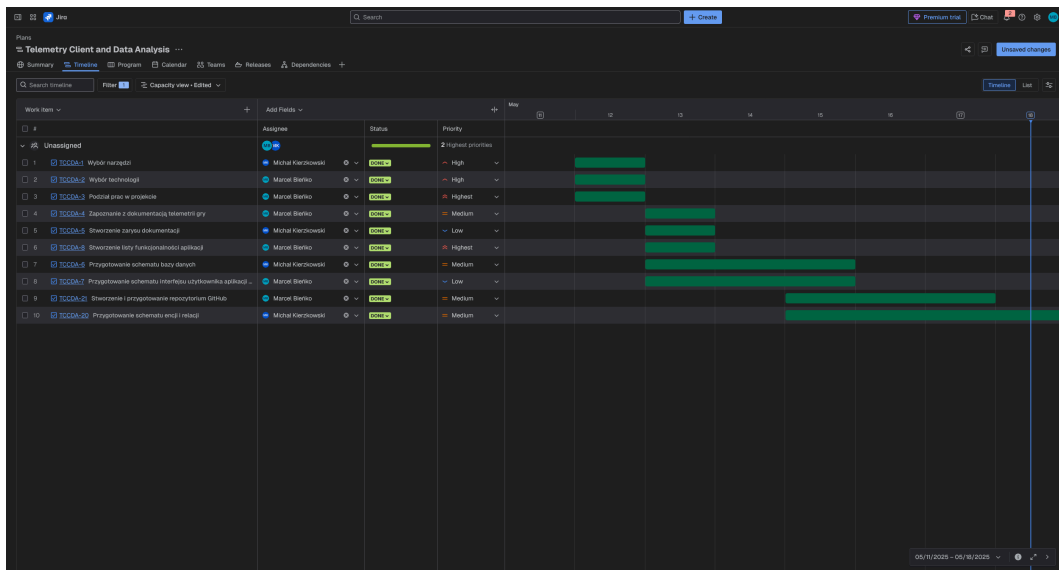
- GitHub[12] - od wielu lat korzystamy z tego narzędzia do zarządzania kodem; ma prostą integrację z Jirą, która usprawnia zarządzanie taskami,
- Jira[13] - jest to pierwsza nasza styczność z tym narzędziem do zarządzania projektami, ale w ramach darmowej subskrypcji studenckiej chcieliśmy poznać jego pełne możliwości (wcześniej korzystaliśmy z ClickUp, Notion, Trello, Linear); szczególnie zachęciły nowe dla nas burndown charty i możliwość integracji z GitHubem.

10.3 Model pracy

Na początku każdego tygodnia rozpisywać będziemy zadania na nadchodzący sprint. Każdy sprint obejmuje 7 dni, od poniedziałku do niedzieli. Pod koniec każdego sprintu, będziemy go analizować i wyciągać wnioski na następny.

Dzięki małej grupie projektowej jesteśmy w stałym, bezpośrednim kontakcie, więc nie potrzebujemy żadnej formy spotkań cotygodniowych/codziennych.

10.4 Sprint 1 - przygotowania



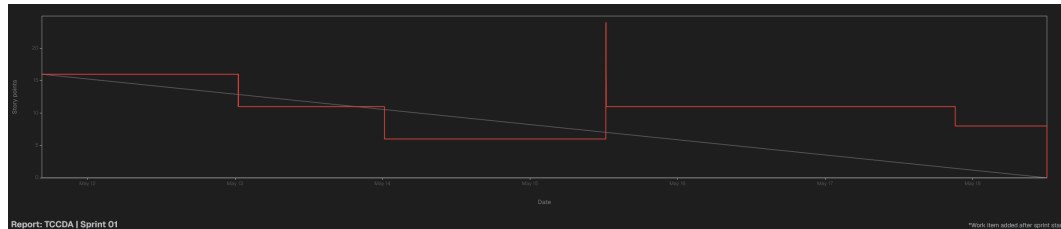
Rysunek 7: Wykres Gantt pierwszego sprintu

Pierwszy sprint okazał się być typowo przygotowawczy - wśród zadań do wykonania były m.in. wybór narzędzi czy technologii, podział prac, przygotowanie narzędzi do zarządzania kodem i projektem oraz pierwsze szkice bazy danych i interfejsu użytkownika.

Zadania w tym i każdym następnym sprincie podlegają następującemu schematowi:

- tag zadania składa się ze znaków „TCCDA” (Telemetry Client and Data Analysis od nazwy projektu) oraz numeru zadania,
- jest przydzielone do konkretnego członka zespołu,
- posiada estymowaną trudność (zgodnie z estymacją Fibonacciego),
- posiada status (TO-DO | IN PROGRESS | DONE),
- posiada priorytet (Lowest - Low - Medium - High - Highest),
- posiada przedział czasowy przeznaczony na jego wykonanie.

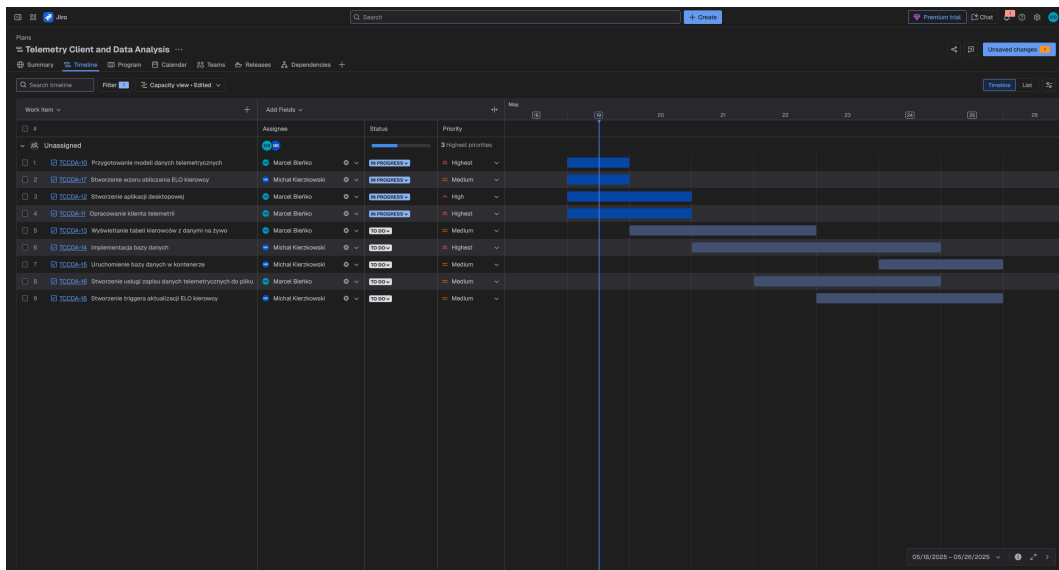
Po każdym sprincie dokonywaliśmy analizy przeprowadzonych prac i wyciągaliśmy wnioski. W centrum analizy zawsze stał *burndown chart*, który dla *sprintu 1* wyglądał następująco:



Największym problemem tego sprintu okazało się rozpisanie zadań, których ilość początkowo była zbyt mała, a następnie zbyt duża. Odzwierciedla się to mniej więcej w połowie wykresu, gdy dodane zostają zadania, oraz na jego końcu, gdy zadania z dużą wagą zamykane są w ostatni dzień.

Każde zadanie z tego sprintu zostało wykonane.

10.5 Sprint 2 - telemetria i baza danych



Rysunek 8: Wykres Gantt na początku drugiego sprintu

Drugi sprint to moment, w którym zaczynamy pisać kod. Pojawiają się pierwsze użycia GitHuba[12] oraz faktyczne zastosowanie nazewnictwa zadań.

Każde nowo rozpoczęte zadanie musiało mieć stworzonego dla siebie *brancha* o identycznej nazwie. Każdy *commit* w tym branchu musiał rozpoczynać się również od tej nazwy. Na koniec, *branch* był *mergeowany* po zatwierdzeniu *pull requesta*. Dodatkowo, każdy *commit* musiał spełniać metodologię *conventional commits*.

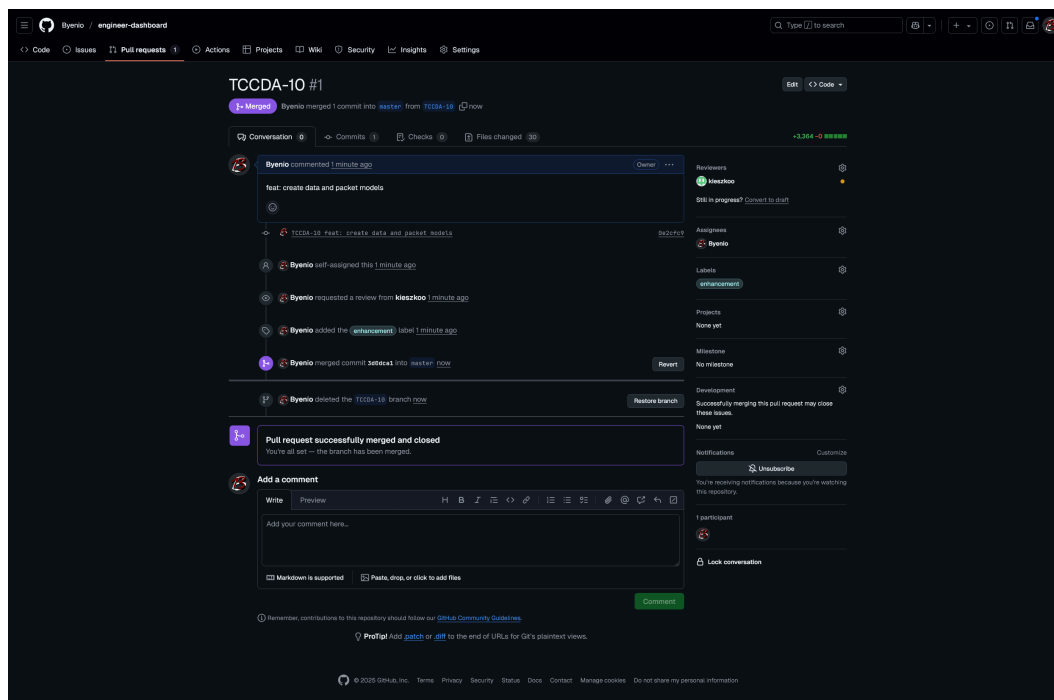
Commit miał postać:

```
TCCDA-<numer_zadania> <typ_commitu>: <krótki_opis>
```

na przykład:

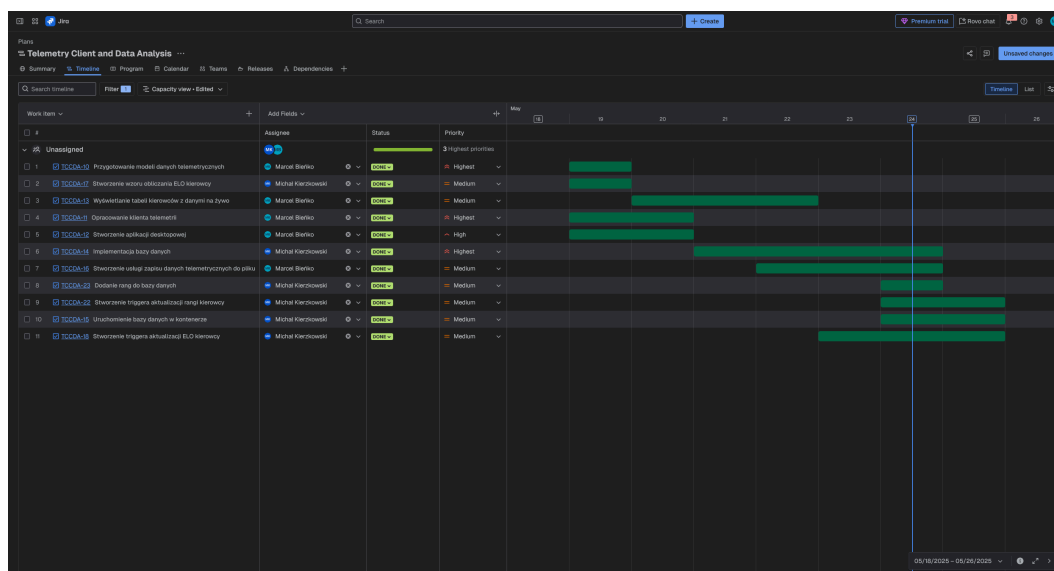
```
TCCDA-10 feat: create data and packet models
```

Dzięki temu podejściu, Jira automatycznie zmieniała status naszych zadań i zamykała je po udanym *pull request*ie.

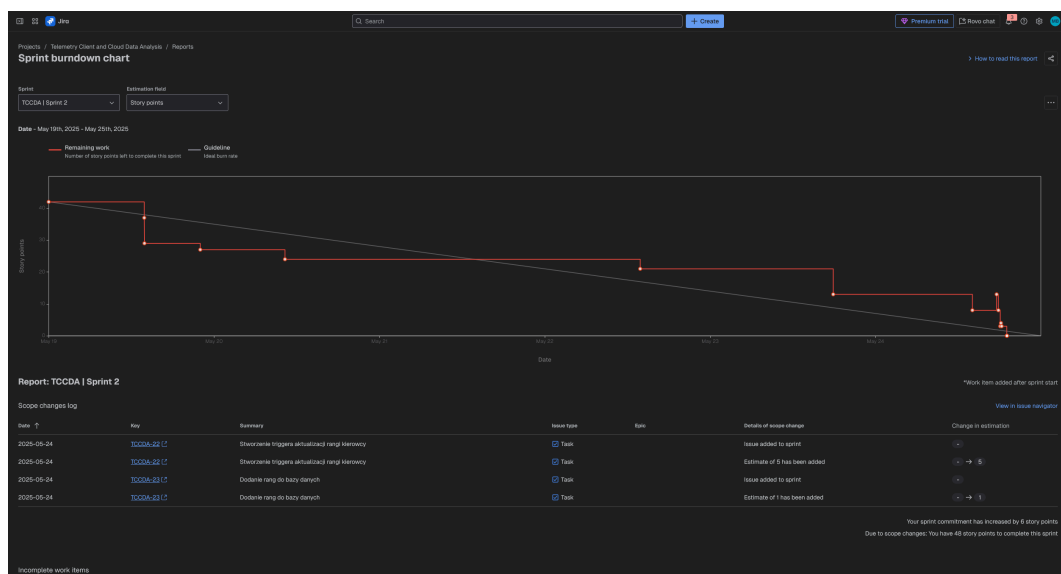


Rysunek 9: Pull request zadania TCCDA-10

Niestety w tym sprincie, podobnie jak w poprzednim, niewystarczająco dobrze określiliśmy przeznaczone na niego zadania. Okazały się jednak na tyle małe, że nie miały negatywnego wpływu na skuteczność zespołu.



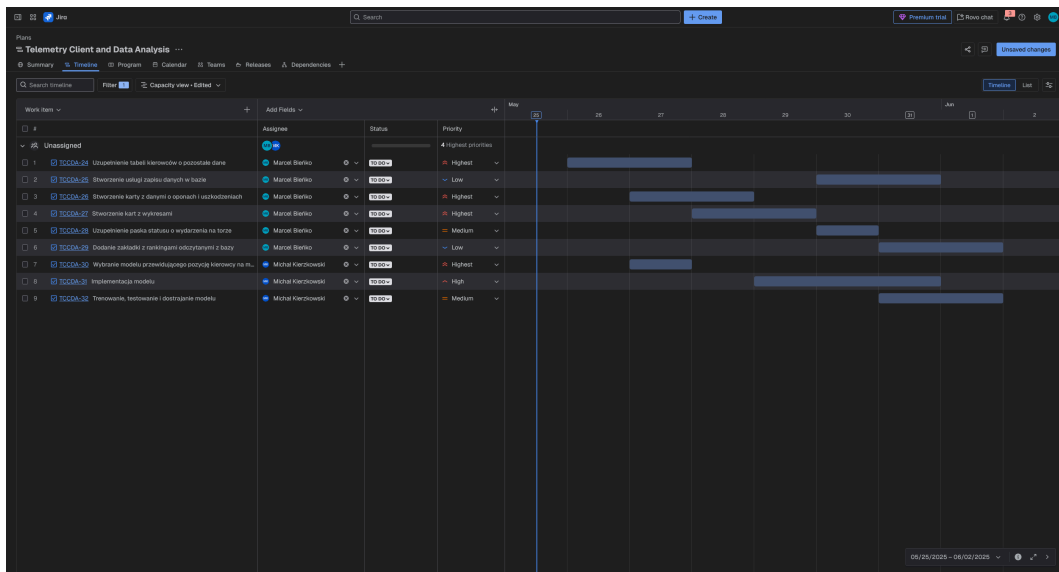
Rysunek 10: Wykres Gantt na koniec drugiego sprintu



Rysunek 11: Burndown chart na koniec drugiego sprintu

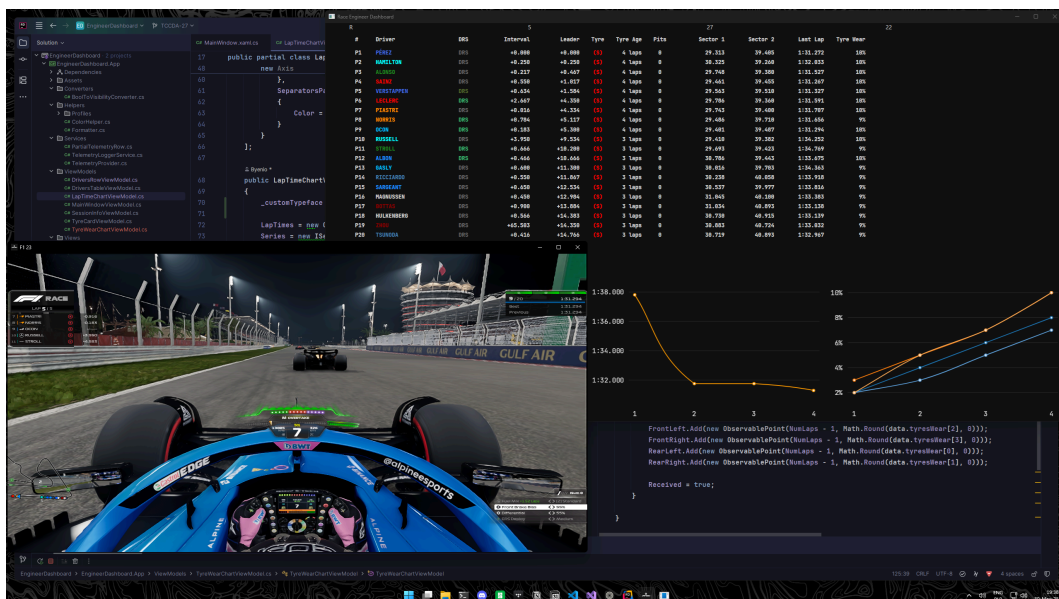
Tym razem rozkład zadań był zdecydowanie lepszy i bardziej równomierny, a taski zostały zakończone przed czasem.

10.6 Sprint 3 - interfejs i model

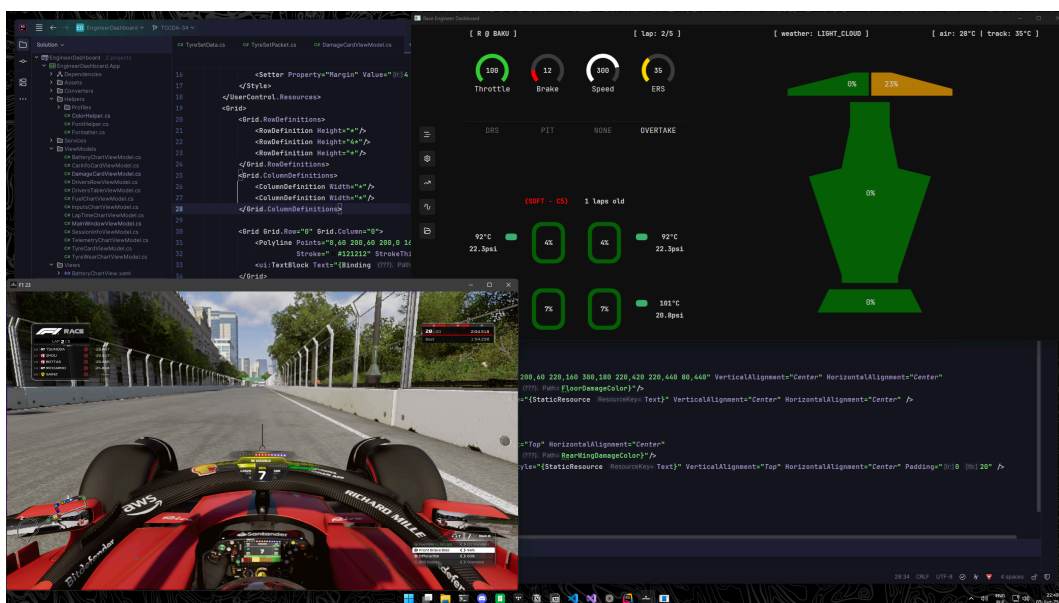


Rysunek 12: Wykres Gantt na początku trzeciego sprintu

Trzeci sprint w projekcie objął największe postępy w interfejsie użytkownika, a także wybranie i pierwsze implementacje modelu.



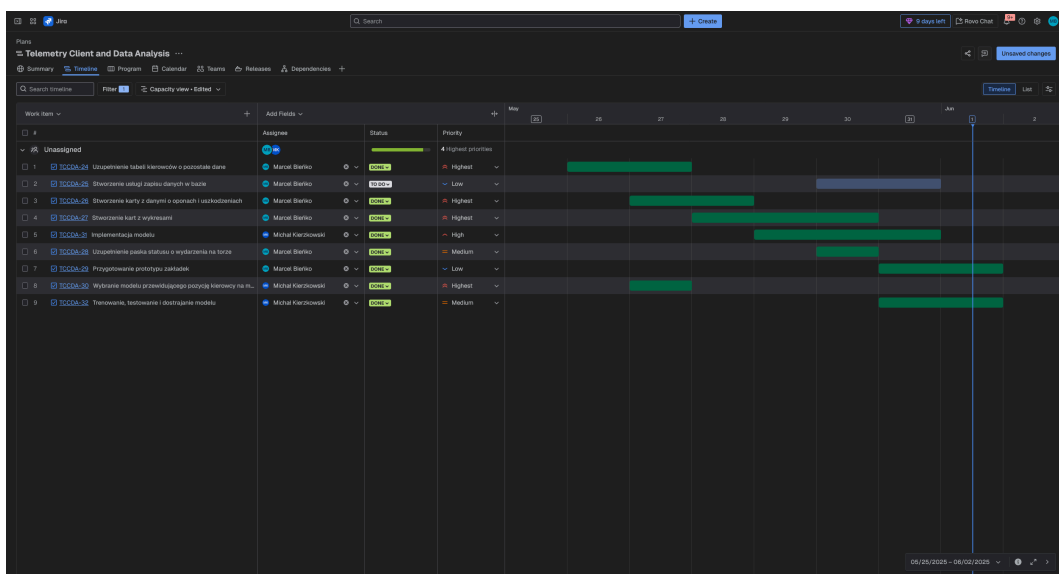
Rysunek 13: Wynik TCCDA-24 i TCCDA-27



Rysunek 14: Wynik TCCDA-26

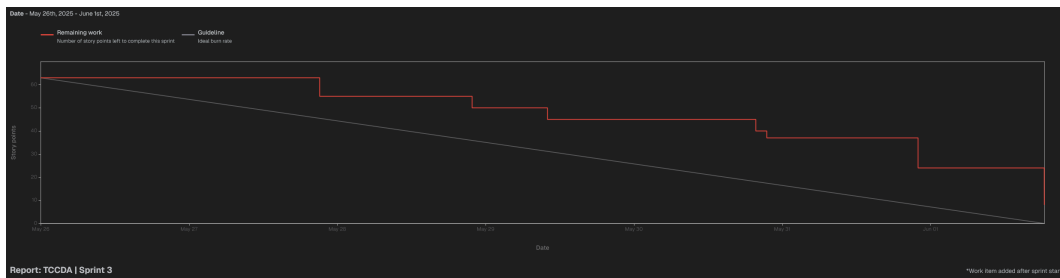
W tym sprincie podjęta została decyzja o wykorzystaniu Bayesowskiej Regresji Liniowej do przewidywania pozycji kierowcy na mecie wyścigu. Dzięki stworzonej w poprzednim sprincie usłudze zapisu danych telemetrycznych do pliku *.csv* mogliśmy od następnego tygodnia zacząć zbierać dane na żywo.

Z powodu pierwszej potrzeby wykorzystania nieznanej nam wcześniej biblioteki do tworzenia wykresów i integracji jej z danymi w czasie rzeczywistym, nie starczyło w tym tygodniu czasu na wykonanie zadania *TCCDA-25*, co przeniosło się do następnego sprintu.



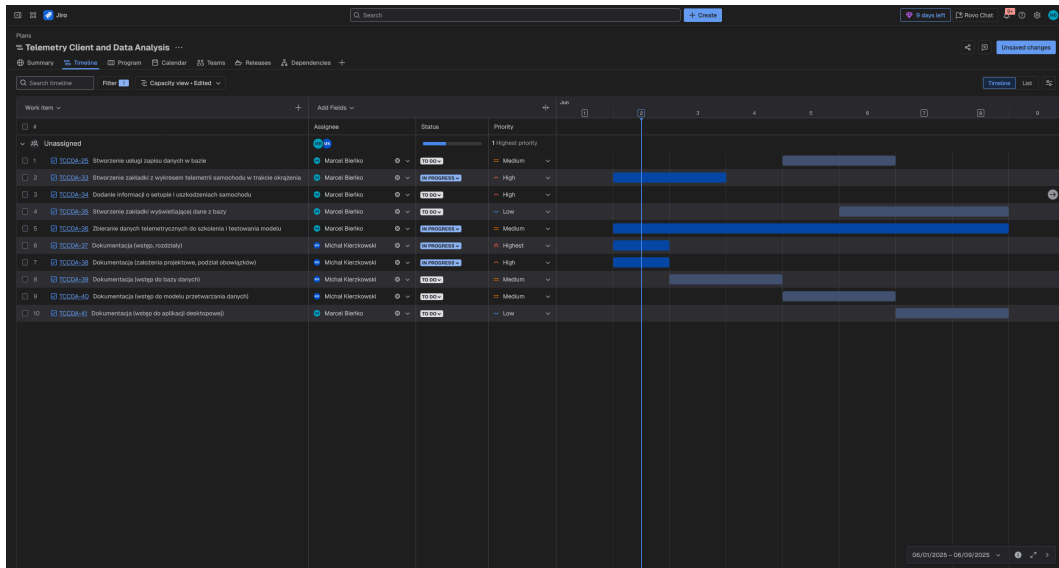
Rysunek 15: Wykres Gantt na koniec trzeciego sprintu

Na ten sprint zaplanowaliśmy zdecydowanie zbyt duże zadania, więc jako jedyny przerósł nasze możliwości produkcyjne.



Rysunek 16: Burndown chart na koniec trzeciego sprintu

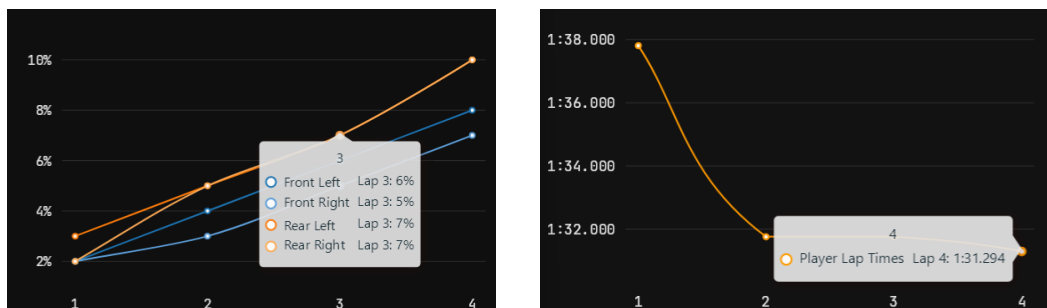
10.7 Sprint 4 - integracja bazy, dokumentacja i zbieranie danych



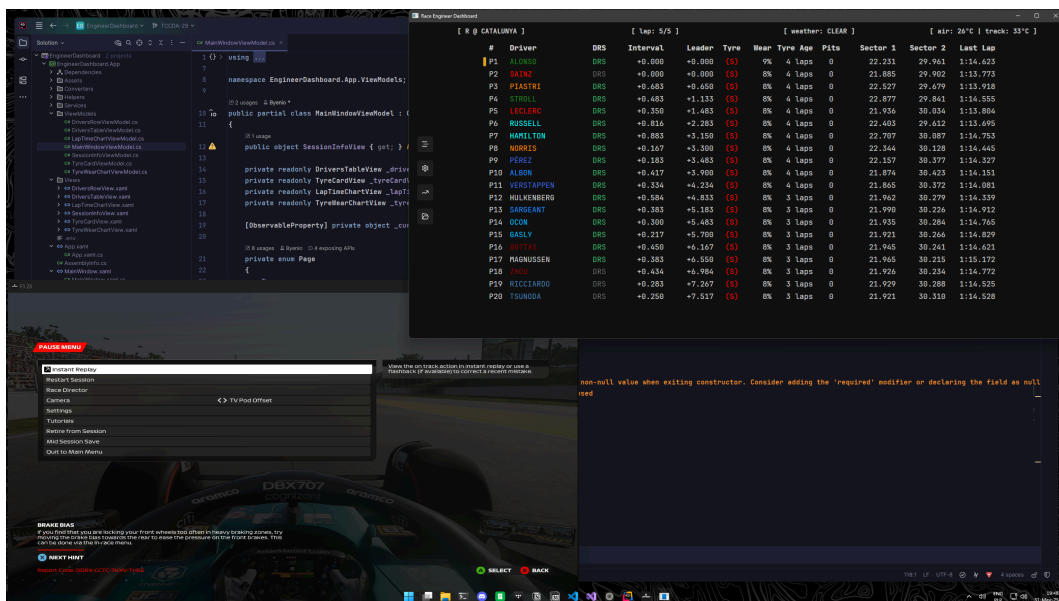
Rysunek 17: Wykres Gantt na początku czwartego sprintu

Czwarty sprint objął przede wszystkim nadrobienie nieukończonego zadania, zbieranie danych na potrzeby modelu oraz rozpoczęcie prac nad dokumentacjami. Nie zabrakło jednak nowych funkcjonalności aplikacji.

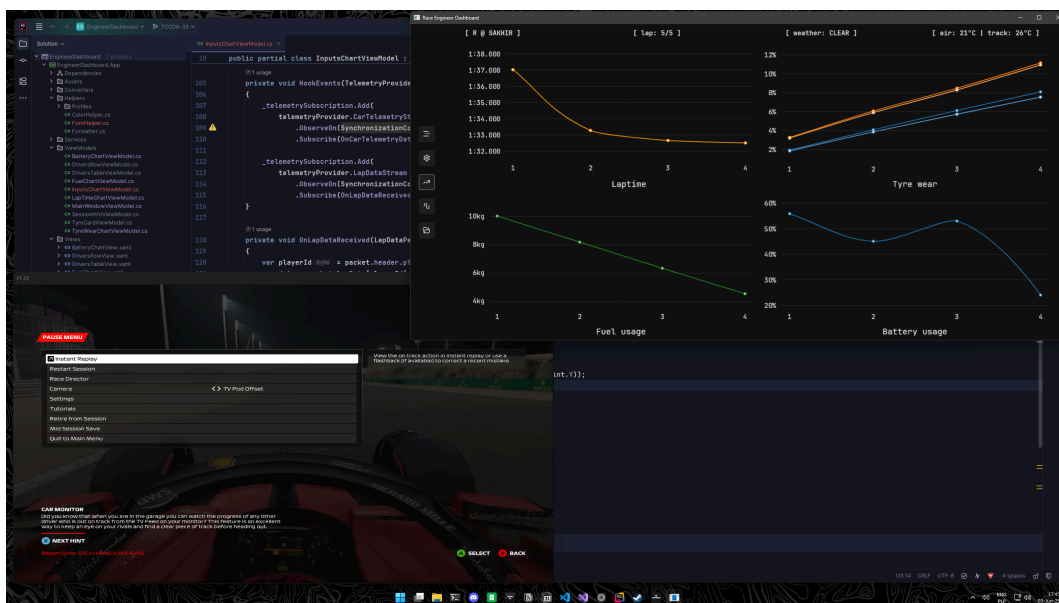
W wolnym czasie aplikacja otrzymała przebudowę UX/UI.



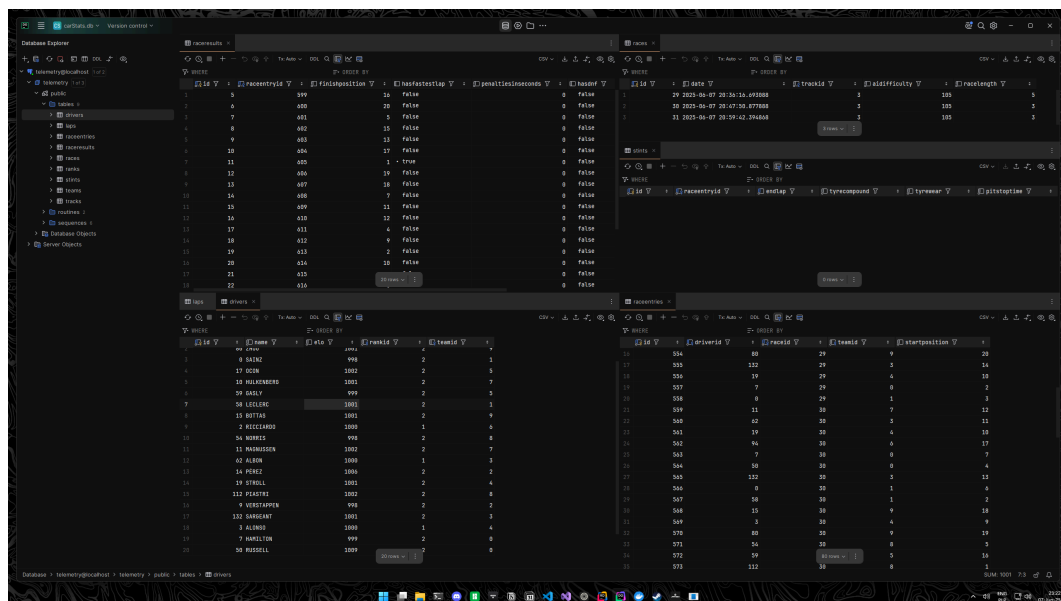
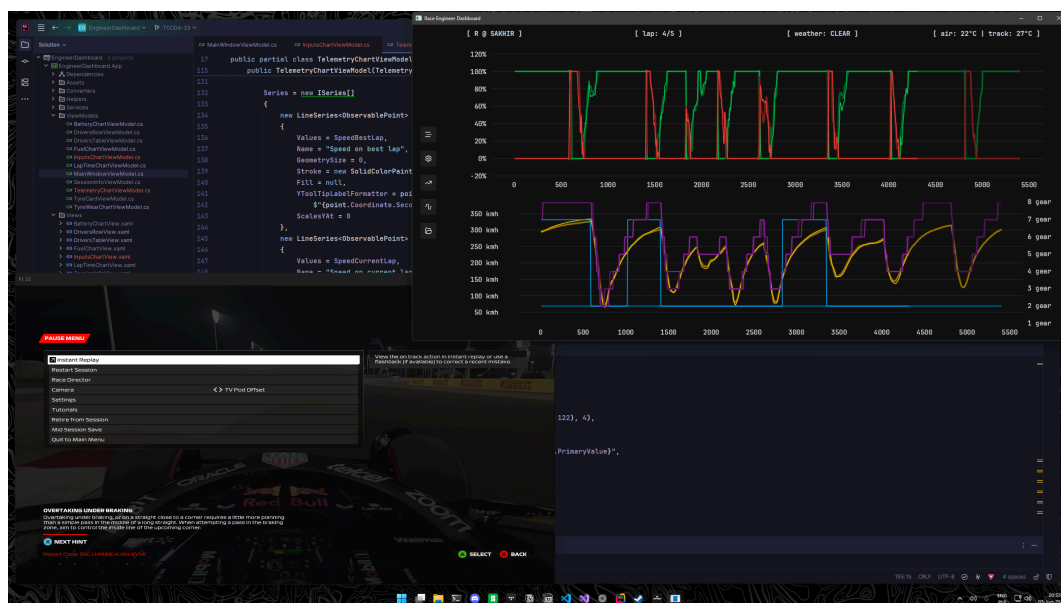
Rysunek 18: Dodatkowe informacje na wykresach

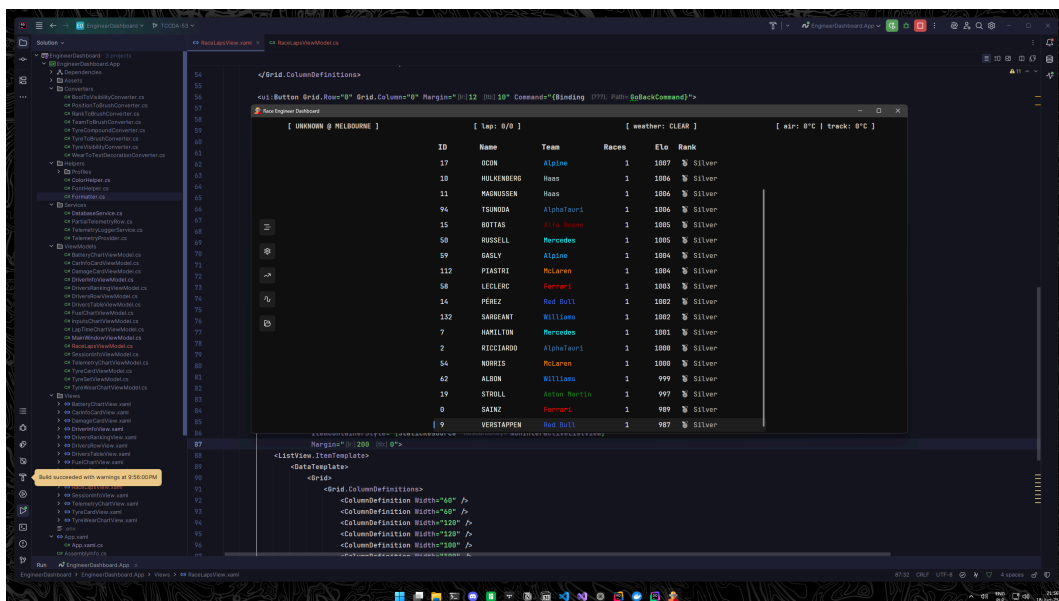


Rysunek 19: Nowy panel z tabelą kierowców



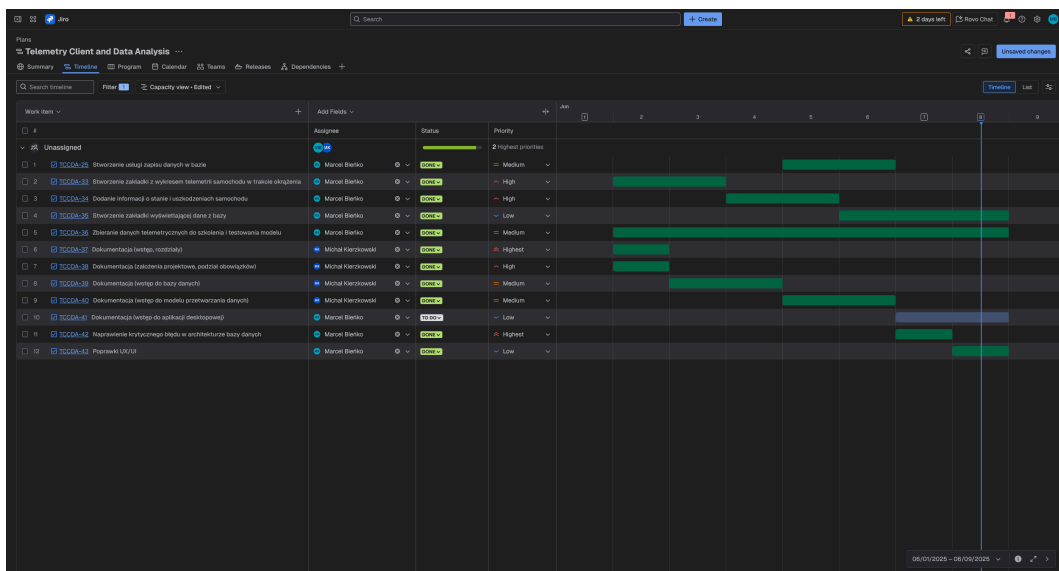
Rysunek 20: Nowe wykresy czasów okrążeń, zużycia opon, paliwa i baterii





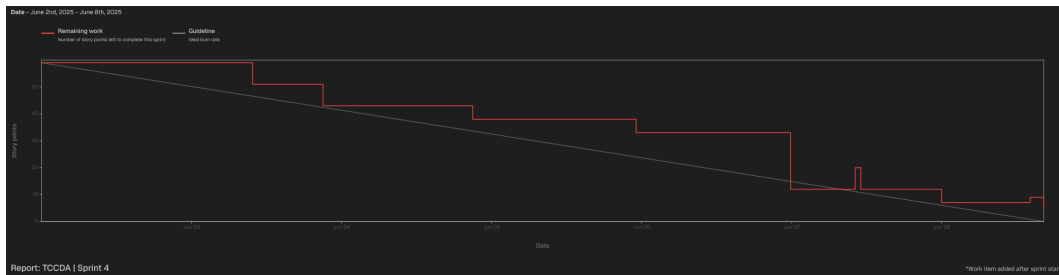
Rysunek 23: Nowy ranking kierowców po połączeniu z bazą danych

Ten sprint doprowadził aplikację ze stanu deweloperskiego do produkcyjnego, a przede wszystkim pozwolił nam odetchnąć od kodu oraz ruszyć z trenowaniem modelu i tworzeniem dokumentacji. Mimo to, znów pojawiło się jedno nieukończone zadanie, głównie z powodu dopracowywania aplikacji - jej działanie było bardziej priorytetowe od przygotowania wstępu do jej dokumentacji.



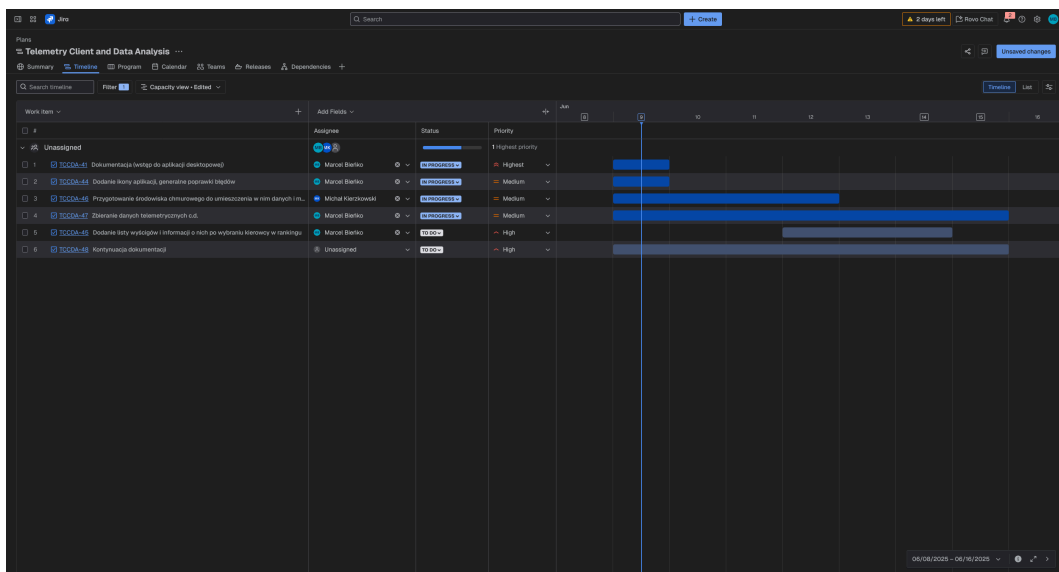
Rysunek 24: Wykres Gantt na koniec czwartego sprintu

Mimo to, sprint okazał się jednym z najbardziej udanych, w którym wszystko szło po naszej myśli. Waga i priorytet zadań zostały odpowiednio dobrane i rozłożone w czasie, a nieukończenie jednego z nich nie przekreśla sukcesu tego tygodnia.



Rysunek 25: Burndown chart na koniec czwartego sprintu

10.8 Sprint 5 - doskonalenie modelu, dokumentacja i ostatnie szlify

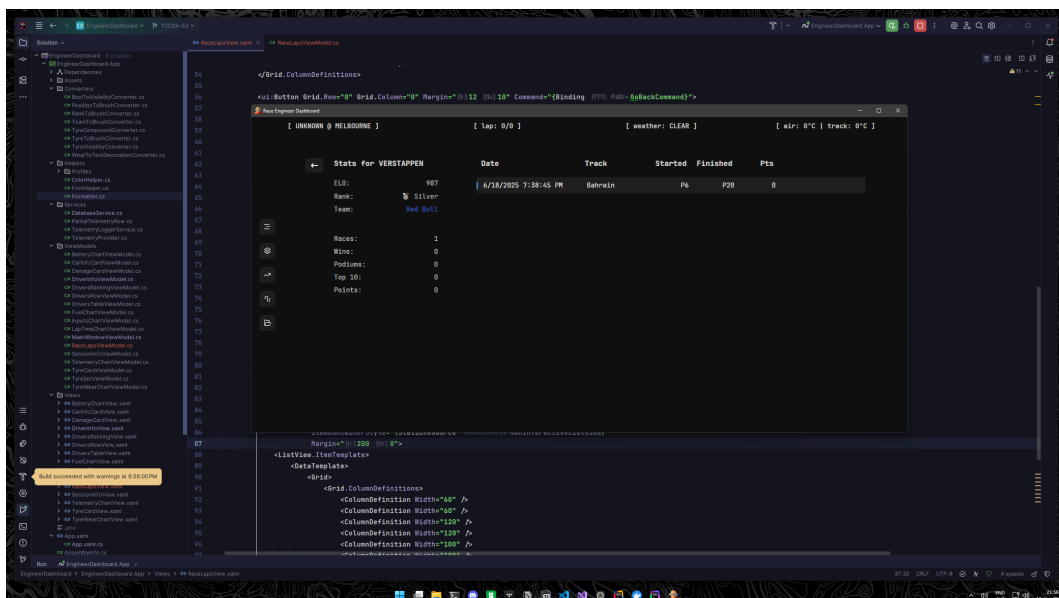


Rysunek 26: Wykres Gantt na początku piątego sprintu

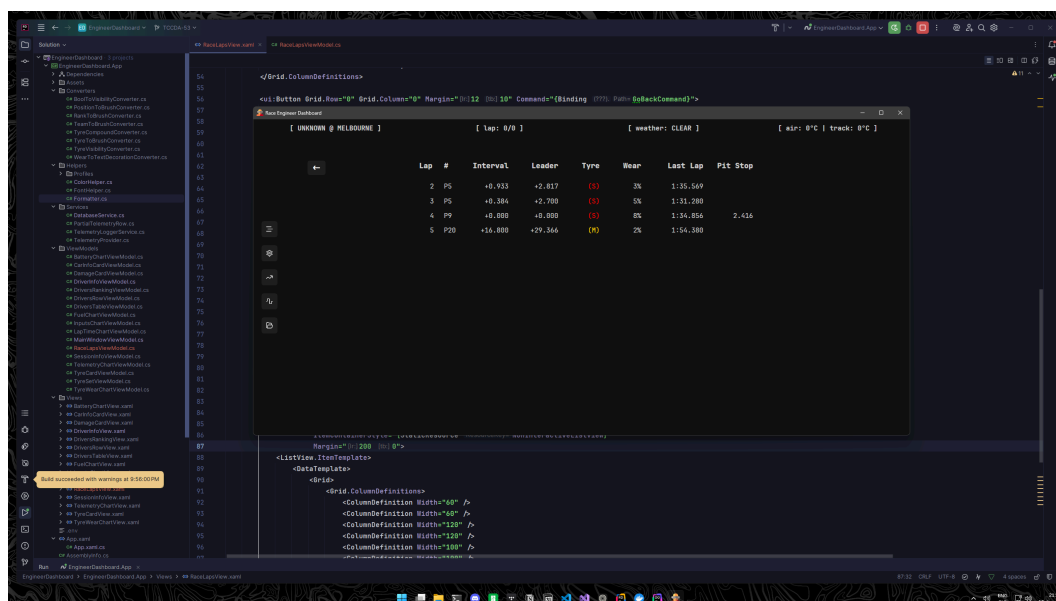
Ten sprint objął formalności związane z dokumentacją, dalsze zbieranie danych telemetrycznych w celu zwiększenia precyzjności modelu, przygotowanie chmury oraz poprawki aplikacji i znanych błędów.

Task dotyczący dokumentacji pozostał nieprzypisany, z uwagi na równocześnie współpracę członków zespołu.

Efektami tasków *TCCDA-44* i *TCCDA-45* były:



Rysunek 27: Panel z osiągnięciami i historią wyścigów kierowcy

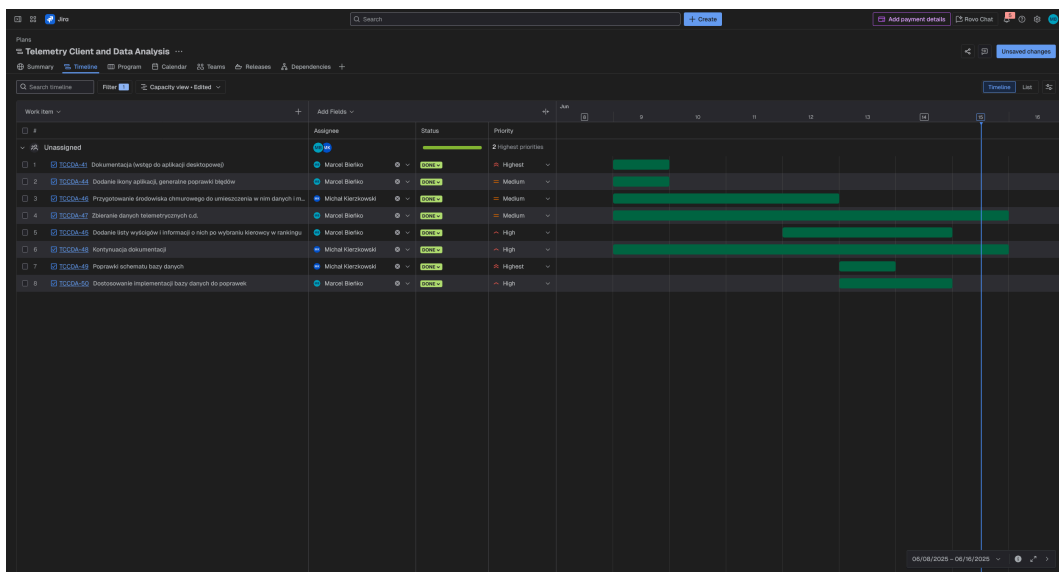


Rysunek 28: Lista okrążeń z wybranymi danymi z konkretnego wyścigu

W trakcie pracy nad nowymi funkcjami ukazanymi wyżej wyniknęły problemy w architekturze bazy danych. Objawiały się one głównie błędami w zapisie okrążeń, uruchamiania triggerów oraz nieprawidłową postacią schematu związków encji.

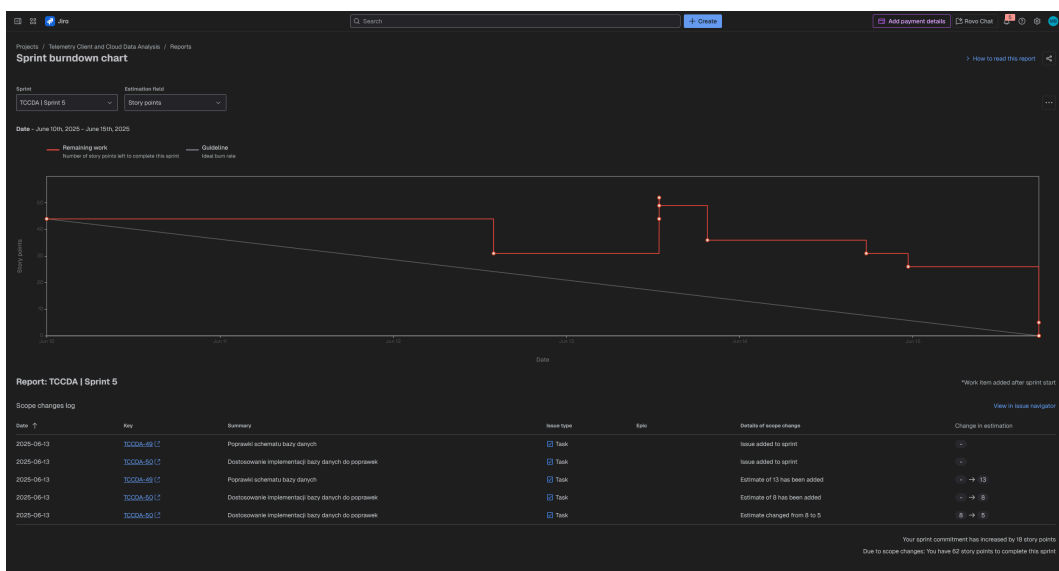
Mimo powagi tych niedopatrzeń, usunięcie ich potrzebowało na tyle niewielkich zmian, że zdecydowaliśmy nie umieszczać tego w dokumentacji. W razie potrzeby wglądu, wszystkie zmiany znaleźć można w repozytorium projektu[12].

Ten sprint również zakończył się powodzeniem i ukończeniem wszystkich przypisanych zadań.



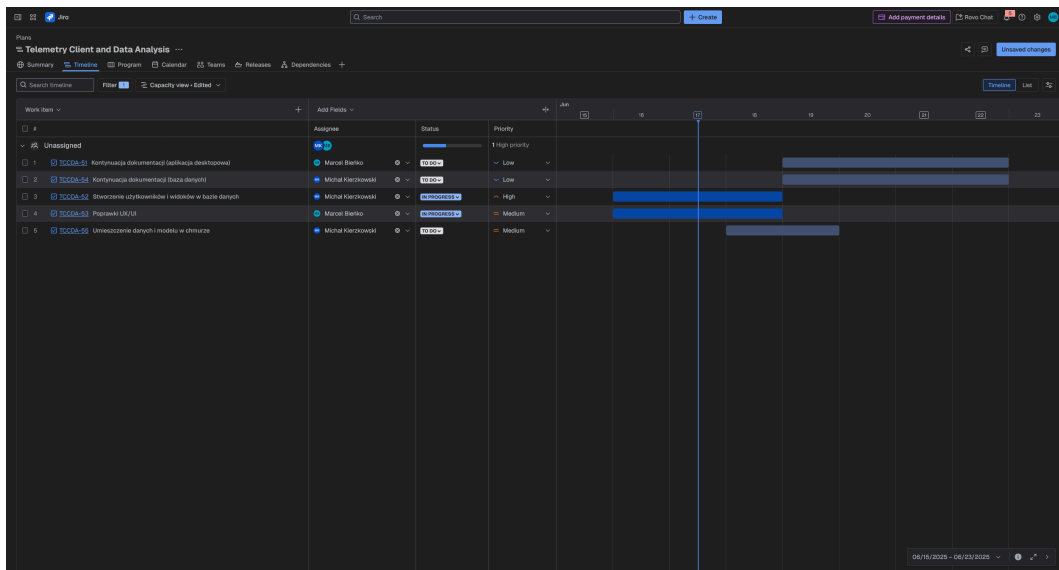
Rysunek 29: Wykres Gantt na koniec piątego sprintu

Do momentu wykrycia błędów w bazie, sprint zostałby wykonany przed czasem. Zakończenie dwóch ważnych zadań (zbieranie danych telemetrycznych i kontynuacja dokumentacji) dawały złudne wrażenie zamknięcia sprintu „na styk”, z racji, że cały tydzień został przydzielony na ich wykonanie. Dzięki pozostawieniu tego sprintu na zadania tego pokroju, mieliśmy spokojną głowę, mimo potrzeby hotfixów. Lekcją na lepsze zobrazowanie postępów przy takich taskach jest rozbięcie ich na mniejsze części (na przykład dokładna część dokumentacji lub zebranie danych z n wyścigów).



Rysunek 30: Burndown chart na koniec piątego sprintu

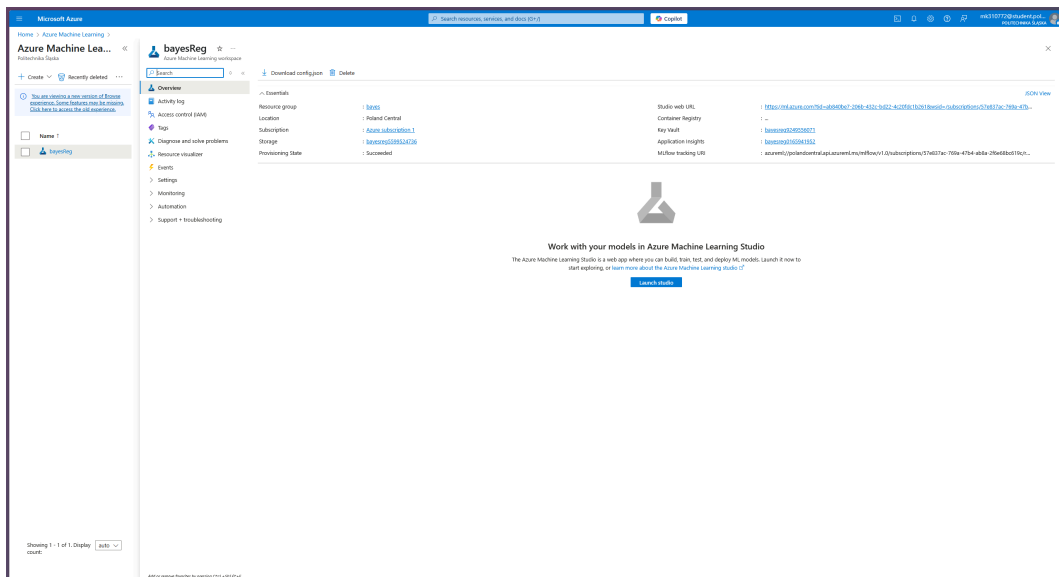
10.9 Sprint 6 - zakończenie projektu



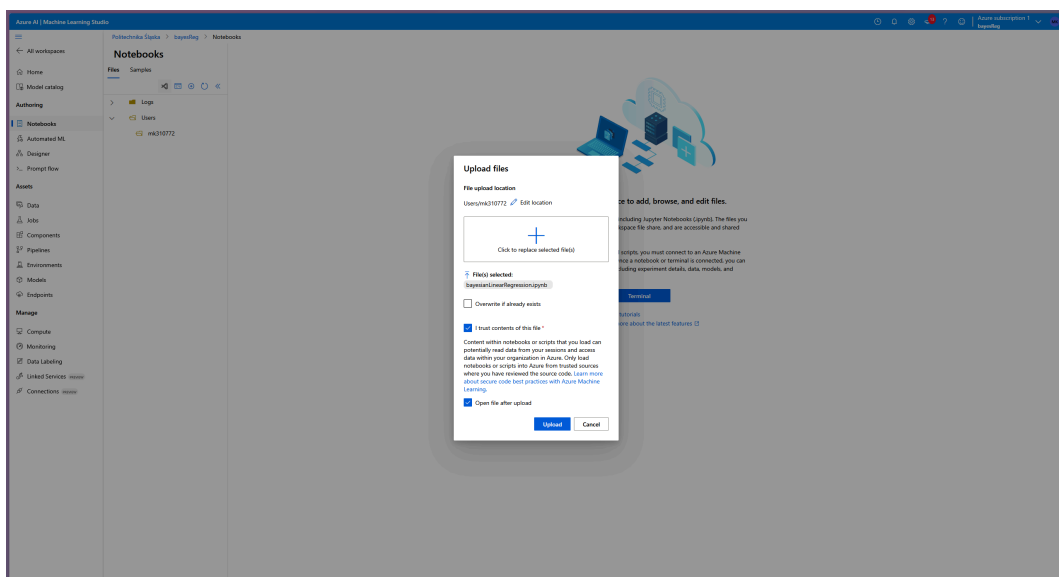
Rysunek 31: Wykres Gantt na początku szóstego sprintu

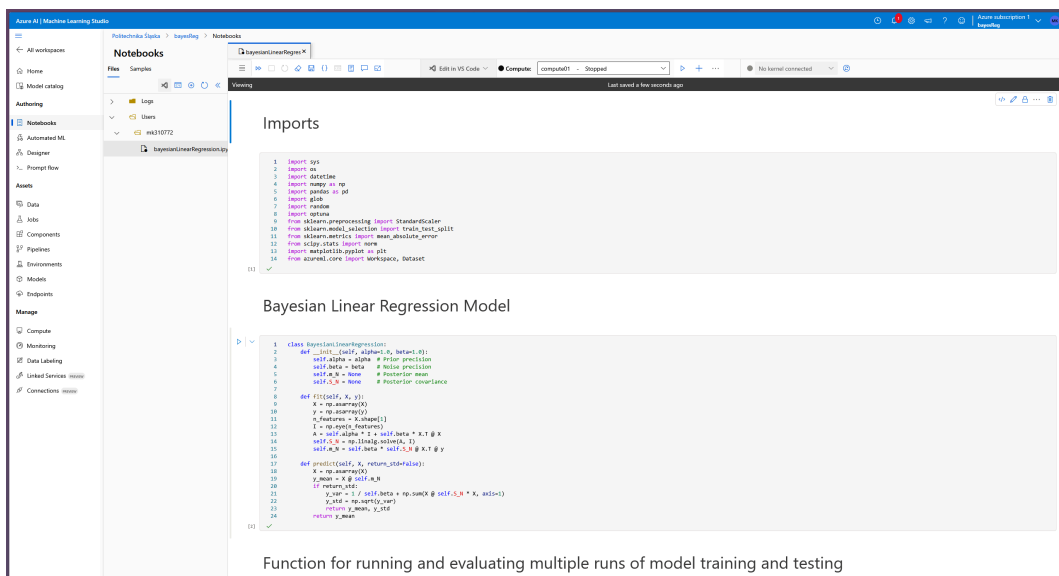
Ostatni przeprowadzony sprint objął ukończenie dokumentacji, ostatnie szczegóły bazy i aplikacji oraz umieszczenie danych i modelu w chmurze.

W chmurze Azure Machine Learning ładujemy na głównej stronie i tworzymy tutaj swój projekt *bayesReg*:

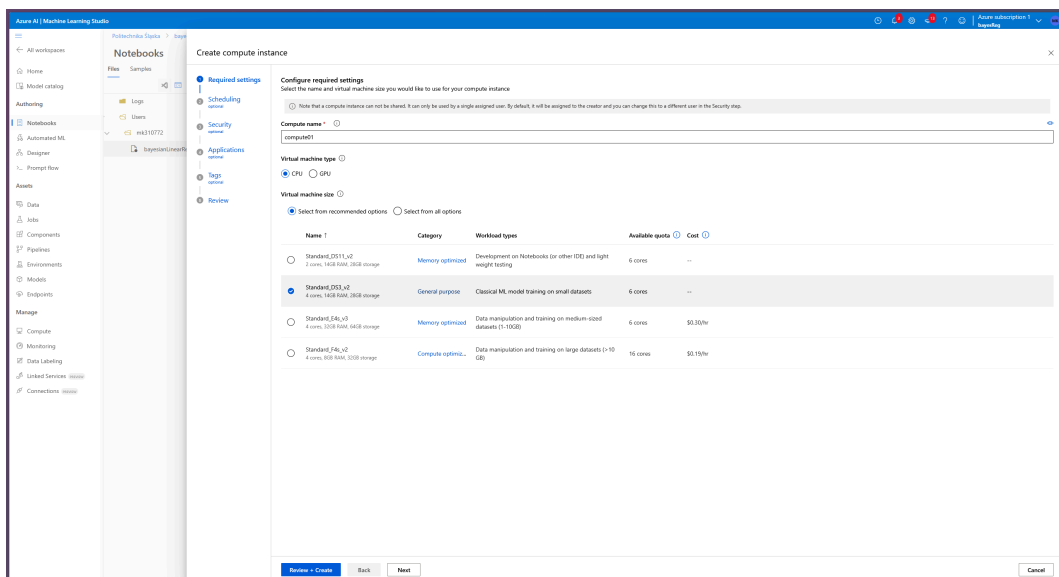


W zakładce *notebooks* dodajemy nasz wcześniej przygotowany Jupyter Notebook z modelem Bayesowskiej Regresji Liniowej:





Następnie tworzymy maszynę wirtualną, która będzie wykonywać wszystkie obliczenia:



Na nasze potrzeby i możliwości wybieramy maszynę opartą o CPU. W planie płatnym, moglibyśmy wykorzystać maszyny wykonujące obliczenia na GPU, co znacząco skróciłoby czas przetwarzania danych i szkolenia modelu:

← All workspaces

Home

Model catalog

Authenticating

Nonlocalize

Automated ML

Designer

Prompt flow

Assets

Data

Jobs

Components

Pipelines

Environments

Models

Deployments

Manage

Compute

Monitoring

Data Labeling

Linked Services (new)

Connections (new)

Microsoft Azure

Search

Compute instance

Compute instances

Choose from a selection of Compute instances

← New → Refresh

Search

None

compute001

Required settings

Scheduling

Security

Applications

Tags

Review

Configure required settings

Select the name and virtual machine size you would like to use for your compute instance

ⓘ Notes that a compute instance can not be shared. It can only be used by a single assigned user. By default, it will be assigned to this creator and you can change this to a different user in the Security step.

Compute name +
ml3107721

Virtual machine type
☐ CPU ☒ GPU

Virtual machine size
☒ Select from recommended options ☐ Select from all options

4 v4 A10 8Bv

Search by VM or GPU name...

Showing 8 VM sizes

Table with 4 columns: Name, GPU device, Available quota, Cost

Table body with 8 rows of VM size details including names like Standard_NC64ms_T4_v1, Standard_NC24ads_A100_v4, etc., and their respective costs.

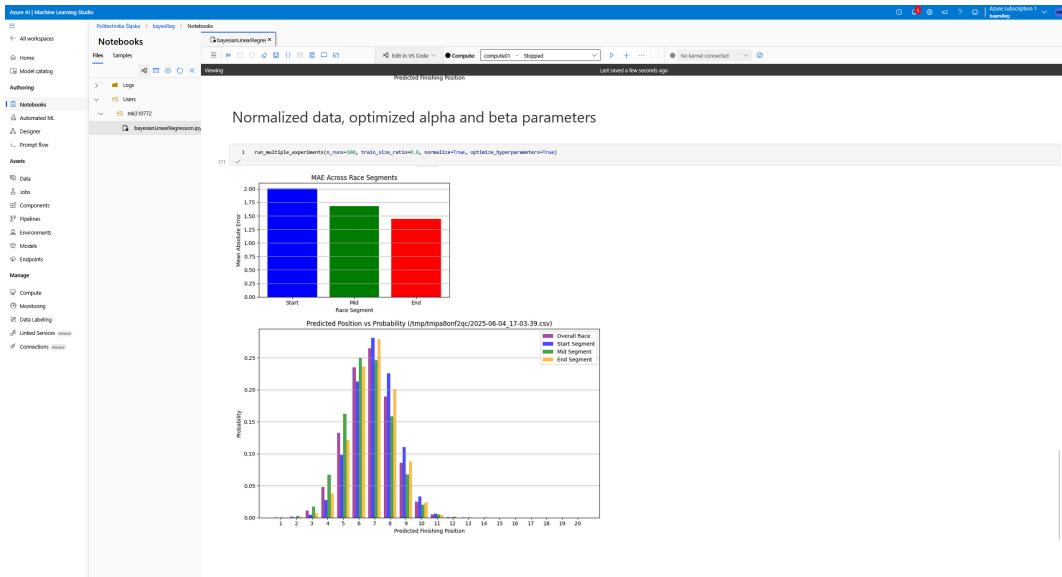
Buttons: Back, Next

Dodajemy pliki tworzące zbiór danych dla modelu:

[illegible]

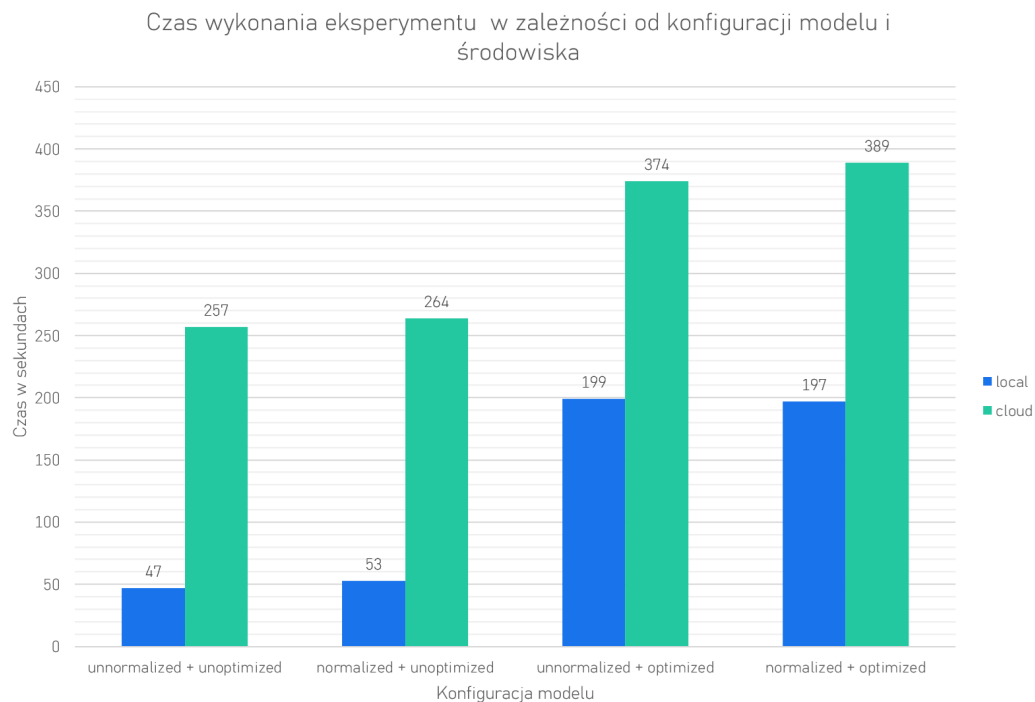
Path	File Name	Modified Time	Created Time	File Size	File Format
AV/2025-06-20_085842.UTC/2025-05-28_1..	2025-06-20_085842.UTC/2025-05-28_14..	2025-06-20 08:58:49	2025-06-20 08:58:49	395.2 KiB	csv
AV/2025-06-20_085842.UTC/2025-05-28_1..	2025-06-20_085842.UTC/2025-05-28_15_1..	2025-06-20 08:58:49	2025-06-20 08:58:49	423.2 KiB	csv
AV/2025-06-20_085842.UTC/2025-05-28_1..	2025-06-20_085842.UTC/2025-05-28_16_1..	2025-06-20 08:58:49	2025-06-20 08:58:49	438.4 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_1..	2025-06-20_085842.UTC/2025-06-04_16_1..	2025-06-20 08:58:50	2025-06-20 08:58:50	450.6 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_1..	2025-06-20_085842.UTC/2025-06-04_16_1..	2025-06-20 08:58:50	2025-06-20 08:58:50	465.6 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_1..	2025-06-20_085842.UTC/2025-06-04_17_1..	2025-06-20 08:58:50	2025-06-20 08:58:50	456.1 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_1..	2025-06-20_085842.UTC/2025-06-04_17_1..	2025-06-20 08:58:50	2025-06-20 08:58:50	440.1 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_1..	2025-06-20_085842.UTC/2025-06-04_17_1..	2025-06-20 08:58:50	2025-06-20 08:58:50	440.3 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_1..	2025-06-20_085842.UTC/2025-06-04_19_1..	2025-06-20 08:58:50	2025-06-20 08:58:50	447.7 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_2..	2025-06-20_085842.UTC/2025-06-04_20_1..	2025-06-20 08:58:51	2025-06-20 08:58:51	437.0 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-04_2..	2025-06-20_085842.UTC/2025-06-04_20_1..	2025-06-20 08:58:51	2025-06-20 08:58:51	451.1 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-07_1..	2025-06-20_085842.UTC/2025-06-07_12_1..	2025-06-20 08:58:51	2025-06-20 08:58:51	475.6 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-07_1..	2025-06-20_085842.UTC/2025-06-07_12_1..	2025-06-20 08:58:51	2025-06-20 08:58:51	456.8 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-07_1..	2025-06-20_085842.UTC/2025-06-07_13_1..	2025-06-20 08:58:51	2025-06-20 08:58:51	448.8 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-07_1..	2025-06-20_085842.UTC/2025-06-07_13_1..	2025-06-20 08:58:51	2025-06-20 08:58:51	466.3 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-07_1..	2025-06-20_085842.UTC/2025-06-07_16_1..	2025-06-20 08:58:52	2025-06-20 08:58:52	463.0 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-07_1..	2025-06-20_085842.UTC/2025-06-07_19_1..	2025-06-20 08:58:52	2025-06-20 08:58:52	442.9 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-08_1..	2025-06-20_085842.UTC/2025-06-08_18_1..	2025-06-20 08:58:52	2025-06-20 08:58:52	427.1 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-08_1..	2025-06-20_085842.UTC/2025-06-08_18_1..	2025-06-20 08:58:52	2025-06-20 08:58:52	462.1 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-08_2..	2025-06-20_085842.UTC/2025-06-08_21_1..	2025-06-20 08:58:52	2025-06-20 08:58:52	459.7 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-08_2..	2025-06-20_085842.UTC/2025-06-08_21_1..	2025-06-20 08:58:52	2025-06-20 08:58:52	446.7 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-10_2..	2025-06-20_085842.UTC/2025-06-10_06_1..	2025-06-20 08:58:53	2025-06-20 08:58:53	489.1 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-10_2..	2025-06-20_085842.UTC/2025-06-10_09_1..	2025-06-20 08:58:53	2025-06-20 08:58:53	462.8 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-10_2..	2025-06-20_085842.UTC/2025-06-10_14_1..	2025-06-20 08:58:53	2025-06-20 08:58:53	445.7 KiB	csv
AV/2025-06-20_085842.UTC/2025-06-10_2..	2025-06-20_085842.UTC/2025-06-10_15_1..	2025-06-20 08:58:53	2025-06-20 08:58:53	455.3 KiB	csv

I na koniec uruchamiamy model:



Rysunek 32: Wycinek z uruchomienia modelu w chmurze

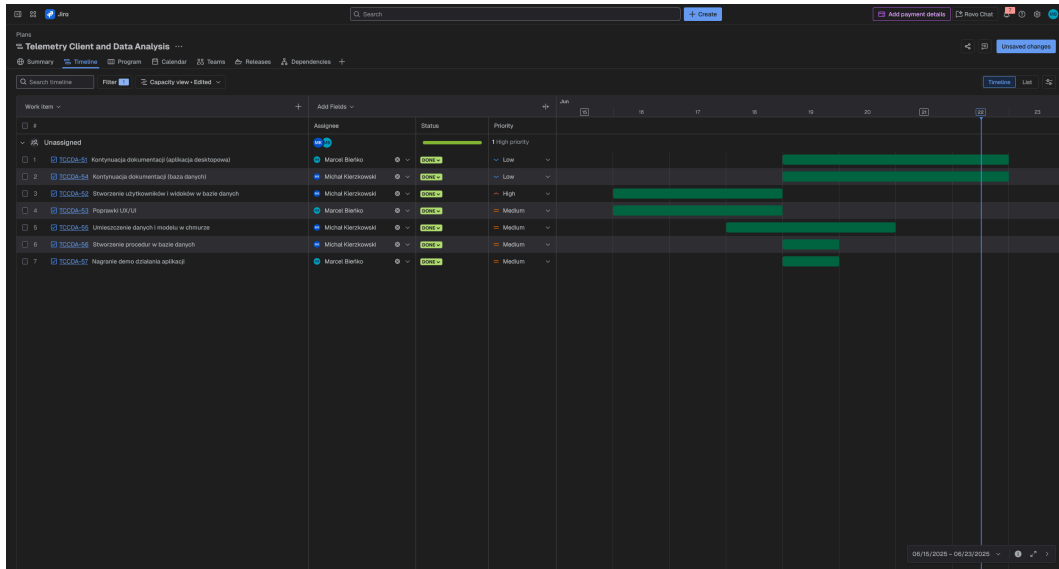
Różnica w czasie przeprowadzanych testów lokalnie i w chmurze jest ogromna, jak na tak niewielki zbiór danych. Poniżej znajduje się porównanie:



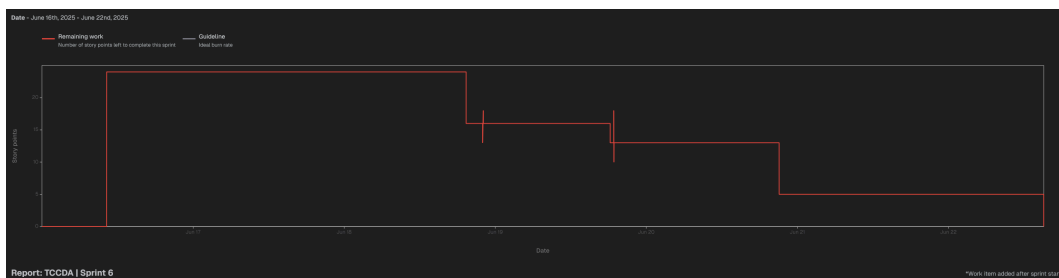
Czas wykonania eksperymentów w chmurze jest dwu- lub nawet pięciokrotnie większy, niż na lokalnej maszynie. Z tego powodu w naszym przypadku lepiej było korzystać z własnego sprzętu. Nieocenionym plusem środowiska chmurowego jest jednak koszt (zerowy) i brak potrzeby nadzorowania przebiegu działania.

Ostatnimi zadaniami wykonanymi w projekcie było nagranie demo[15] działania aplikacji oraz dokończenie tego fragmentu dokumentacji.

Ten sprint nie mógł nie zakończyć się sukcesem. Wszystkie zadania zostały wykonane terminowo i z powodzeniem.



Rysunek 33: Wykres Gantt na koniec szóstego sprintu



Rysunek 34: Burndown chart na koniec szóstego sprintu

Wszystkie pozostałe zrzuty ekranu zrobione w trakcie tworzenia projektu znajdują się na studenckiej chmurze OneDrive[16]

11 Podsumowanie projektu

Był to nasz największy projekt do tej pory, szczególnie, że obejmował w sumie 4 podprojekty składowe.

Udało nam się spełnić założenia każdego z nich i zmieścić się w wyznaczonym przez nas i studia czasie.

Byliśmy w stanie wykorzystać nowe i znane nam technologie, narzędzia i przetestować możliwości współpracy na większą skalę. Stworzyliśmy pełny ekosystem inżyniera wyścigowego, z ogromnymi możliwościami rozwoju, z którego sami będziemy korzystać w rywalizacji ze znajomymi.

Ten projekt nauczył nas wiele o istotności planowania i zarządzania projektem oraz komunikacji. Pozwolił nam znaleźć odpowiednie metodologie i ułatwił nam poruszanie się w takich przedsięwzięciach. Uświadomił nas jak kluczowa jest integracja wielu systemów do działania w jednym środowisku.

Możliwości rozwoju tego projektu stoją na każdym kroku, znajdują się w każdym jego składniku. Od zwiększenia możliwości aplikacji, przez rozwój bazy danych, znalezienie lepszych modeli i doskonalenie ich dokładności i zwiększenie udziału usług chmurowych w tym celu.

Jesteśmy dumni z tego osiągnięcia i uważamy, że projekt jest idealnym podsumowaniem naszej dotychczasowej pracy na studiach.

12 Bibliografia

- [1] „WPF”. [Online]. Dostępne na: <https://learn.microsoft.com/pl-pl/dotnet/desktop/wpf/?view=netdesktop-9.0>
- [2] „WPF-UI”. [Online]. Dostępne na: <https://wpfui.lepo.co/>
- [3] „LiveCharts2”. [Online]. Dostępne na: <https://livecharts.dev/>
- [4] „PostgreSQL”. [Online]. Dostępne na: <https://www.postgresql.org/>
- [5] „Docker”. [Online]. Dostępne na: <https://www.docker.com/>
- [6] „Jupyter”. [Online]. Dostępne na: <https://jupyter.org/>
- [7] „numpy”. [Online]. Dostępne na: <https://numpy.org/>
- [8] „pandas”. [Online]. Dostępne na: <https://pandas.pydata.org/>
- [9] „optuna”. [Online]. Dostępne na: <https://optuna.org/>
- [10] „scikit-learn”. [Online]. Dostępne na: <https://scikit-learn.org/stable/>
- [11] „matplotlib”. [Online]. Dostępne na: <https://matplotlib.org/>
- [12] „Repozytorium projektu”. [Online]. Dostępne na: <https://github.com/Byenio/engineer-dashboard>
- [13] „Jira”. [Online]. Dostępne na: <https://www.atlassian.com/software/jira>
- [14] „Dokumentacja telemetry gry F1 23”. [Online]. Dostępne na: <https://forums.ea.com/discussions/f1-23-en/f1-23-udp-specification/8390745>
- [15] „Film demonstracyjny”. [Online]. Dostępne na: <https://youtu.be/8jKv0Fvq-1U>
- [16] „Niewykorzystane zrzuty ekranu”. [Online]. Dostępne na: https://polslpl-my.sharepoint.com/:f/g/personal/mb310425_student_polslpl/El0wcA0iyBNEtq-woskYKroBb78Da1yV76Baxk3W3DyieA?e=BSVwQS